第五章　中央处理器

SEU.CSE.RGL

※主要内容

(1)CPU的组成与工作流程：功能，组成，工作流程，
指令周期的组织(执行过程/操作需求&定时)

(2)数据通路的组织：数据通路的组成(部件/互连)，μOP及其控制，
指令执行过程的组织(操作需求&控制需求)，
数据通路的设计(步骤/单周期/多周期)

(3)控制器的组成：基本结构，时序信号的形成(时序系统组织/实现)，
μOP控制信号的形成(接口信号/内部逻辑)
硬布线CU设计—设计步骤，设计示例(单周期/多周期)
微程序CU设计—微程序控制思想，CU组成及工作原理

(4)异常及中断的处理：基本概念，处理过程，中断机构组成(△)

(5)指令流水线技术：工作原理，组成要求，性能，冒险处理(△)

※总体要求：掌握CPU的组成、工作原理，可设计数据通路及CU，
理解异常的处理过程、流水线的工作原理

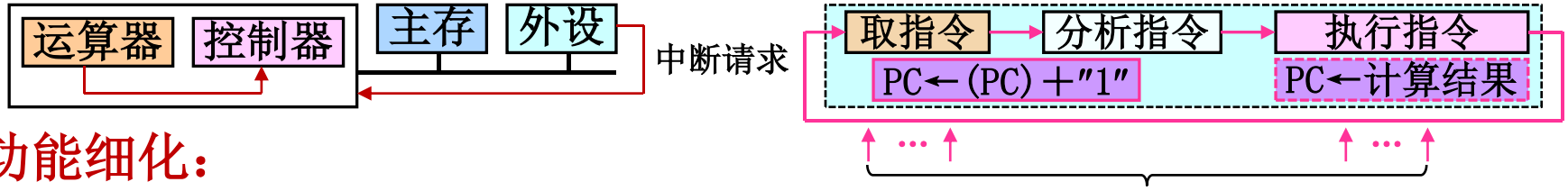
§5.1 CPU的组成与工作流程

※主要内容：CPU的功能、组成、工作流程，指令周期的组织(过程/操作&定时)

一、CPU的功能

*基本功能： ①循环地执行指令(∈指令系统)， ←实现工作方式&指令系统

②处理异常&中断请求 ←保证执行正确&I/O性能



*功能细化：

(1)指令控制—控制指令的执行顺序(含执行过程) ←工作方式需求

(2)数据加工—实现指令约定的数据运算

(3)外部访问—实现对存储器、外设的访问

} 指令系统需求

(4)异常及中断处理—实现异常及中断的检测及处理 ←正确性&I/O需求

思考：操作(如ALU做加法/写REG)的实现方法？ ＝向部件发送数据&控制信号

(5)操作控制—产生指令执行所需的操作控制信号

(6)时间控制—控制操作控制信号的时序(时长及次序)

} 功能实现需求

二、CPU的组成

1、CPU的基本组成

*基本部件：指令控制—PC、IR、ID(指令译码器)

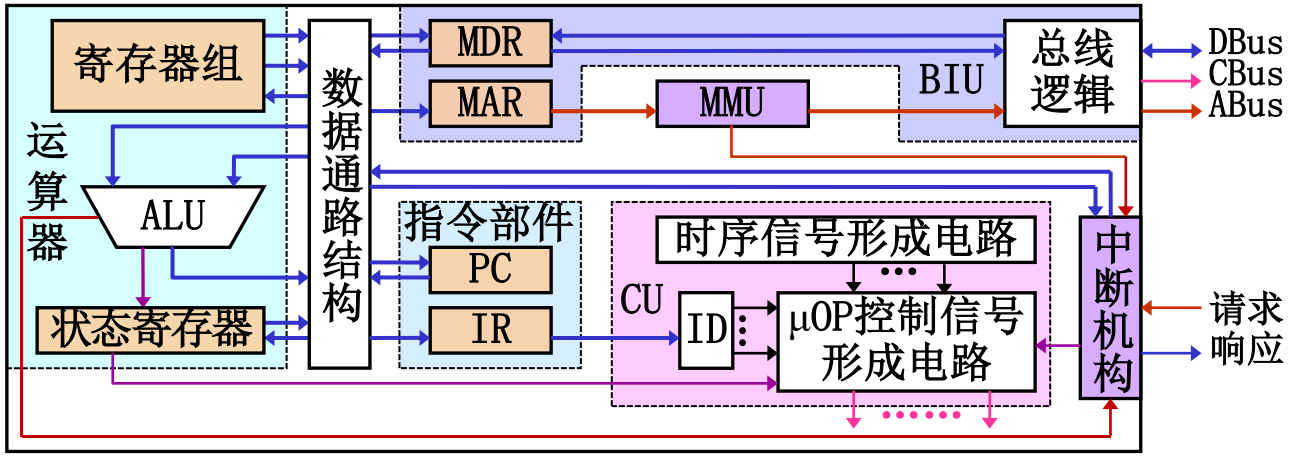
数据加工—ALU、FPU(浮点处理单元)、寄存器组、状态寄存器等

外部访问—总线逻辑电路、缓冲寄存器(内-外间)，MMU(地址变换)

异常&中断处理— 中断机构

操作&时间控制—时序信号形成电路、操作控制信号形成电路

*基本组成：



结构划分—执行部件(数据通路)＋控制部件(控制器)　←基于操作/操作控制

└←含指令部件+中断机构　└←仅含CU

数据通路(Data Path)—指令执行过程中数据所经过的路径及路径上的部件

控制器(Control Unit)—进行指令译码并产生指令执行所需控制信号的部件

2、寄存器的组织

GPR—General Purpose Register
PSR—Program Status Register

*用户可见寄存器：（程序中使用）

数据REG—存放OPD，整数、浮点数分别设置（长度＝机器字长n、32位）

地址REG—存放OPD/指令的MEM地址（长度＝逻辑地址位数m，m可≠n）

通用REG(GPR)—用作数据REG和地址REG（要求m＝n）　←主流方法

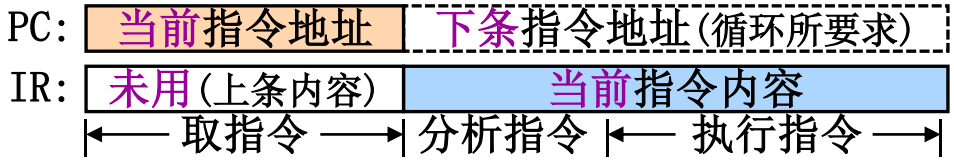
状态REG(PSR)—存放程序运行状态（又称为PSW[Program Status Word]）

①结果标志：ZF/CF/OF/SF等，常用作条件码（可修改）

②执行标志：跟踪标志TF、中断允许标志IF等（可设置）

*专用寄存器：

PC&IR—存放指令地址（用作循环变量），存放当前指令内容



MAR&MDR—存放外部访问的部件地址&数据　←内部-外部操作可并行

不设置：IR←M[(PC)]　PC←(PC)+4，…　←PC被占用、需等待

设置时：MAR←(PC)　MDR←M[(MAR)]　IR←(MDR)　PC←(PC)+4，…　←内外无关，可并行

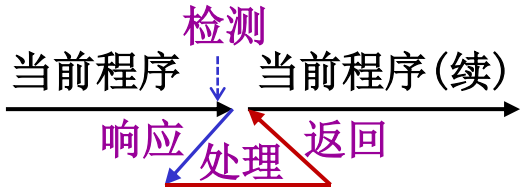
系统控制REG—

三、CPU的工作流程

*CPU基本功能分析：（循环地执行指令、处理异常&中断）

指令执行的过程—取指+分析+执行，形成下条指令地址

异常&中断处理的过程—



检测：硬件(请求随机)，采样信号线

响应：硬件(基于检测)，保存返回点、请求判优、PC←处理程序入口

处理：软件(灵活性强)，执行处理程序

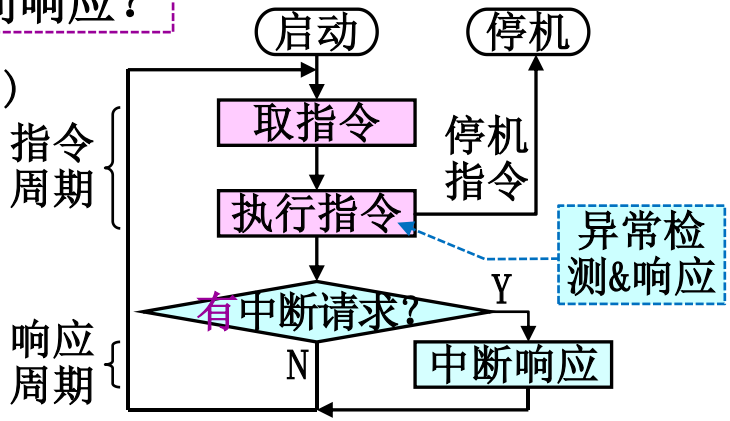
同时可能有多个请求

*CPU工作流程：

思考：为何中断可在指令间响应？

循环地执行指令＋响应中断/异常(可缺省)

指令周期—取出并执行一条指令的时间
响应周期—响应异常/中断请求的时间



细化—由多个有序的操作组成

*CPU工作原理：

CU循环地产生工作流程所需的操作控制信号

数据通路实现工作流程所需的操作功能

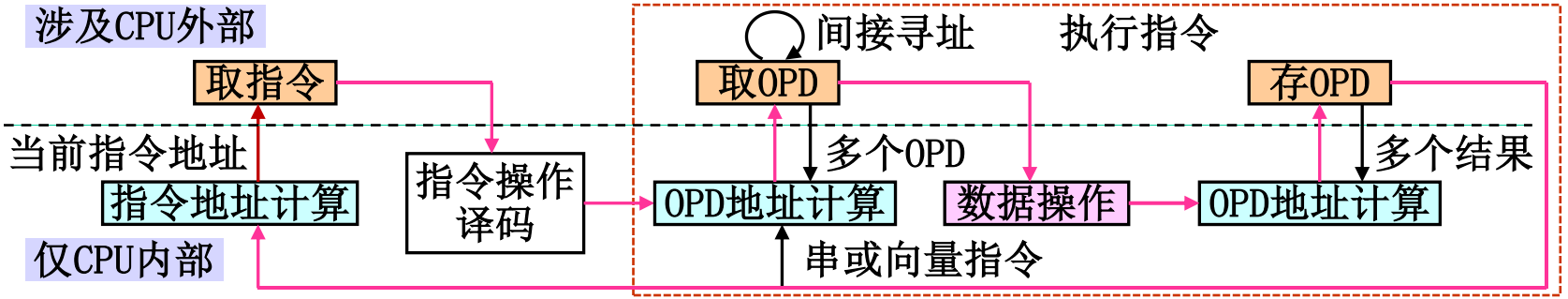
→操作控制

←操作实现

四、指令周期的组织 --执行过程&操作组织

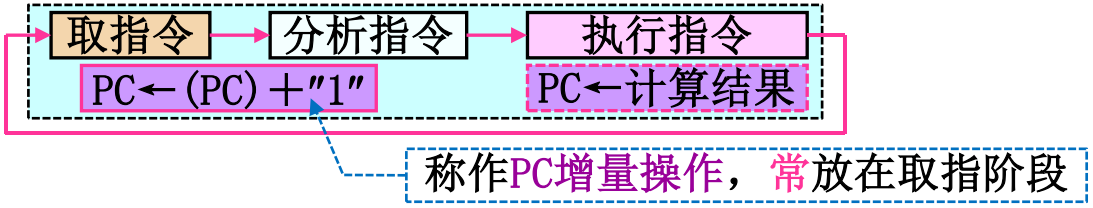
1、指令执行的步骤

***理论方案：** (基于指令格式)



思考：哪些寻址方式可缺省OPD地址计算？指令地址计算可以提前吗？

***实际方案：** (优化性能)



***任务分配：** 数据通路实现取指令、执行指令的操作

例:R2←(R2)+(R1)
R2←(R2)-M[(R1)]

- 取指令、执行指令的操作
  - 所有指令的操作相同
  - 不同指令的操作不同

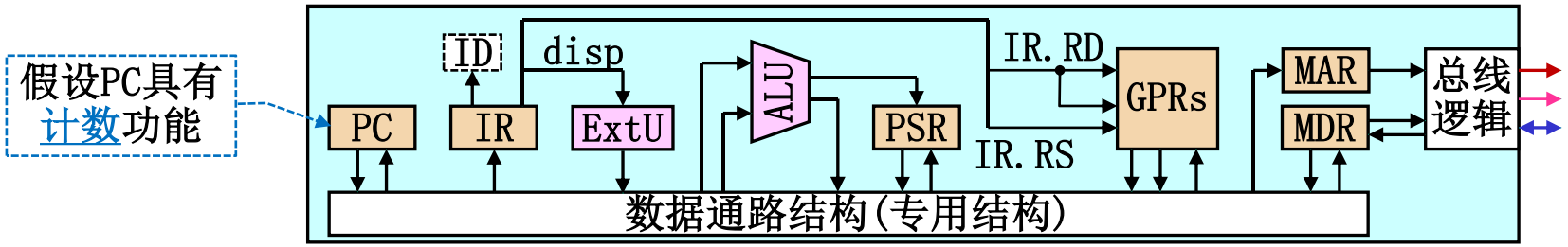
CU实现分析指令(无操作，产生相应的操作控制信号)

2、指令执行的过程

***指令系统示例：** Demo_IS指令系统(下图为部分指令)

指令功能	指令格式	指令寻址方式
取数(LD) ： RD←M[(RS)]	0010 RD RS	隐含EA=(PC)＋1
存数(ST) ： M[(RS)]←(RD)	0011 RD RS	隐含EA=(PC)＋1
加法(ADD)： RD←(RD)＋(RS)或	010 0 RD RS	隐含EA=(PC)＋1
RD←(RD)＋M[(RS)]	010 1 RD RS	隐含EA=(PC)＋1
减法(SUB)： RD←(RD)－(RS)	0110 RD RS	隐含EA=(PC)＋1
分支(JNZ)： ZF＝0时PC←(PC)＋Disp	1100 Disp	相对EA=(PC)＋Disp 或隐含EA=(PC)＋1

***数据通路示例：** 运算器由ALU及ExtU(位扩展)、GPRs及PSR组成



***基本操作的组成要求：** (操作具有独立性[不依赖于其他操作])

①源数据/结果放在时序逻辑部件中(如REG) ←否则依赖其它操作

②操作功能具有原子性(如REG→ALU→REG) ←否则不是基本操作

*程序执行环境示例：假设(PC)=10H，(R0)=20H，(R2)=30H

10H	00100100
11H	00111000
12H	01100110
13H	11011110
	…
20H	01001000

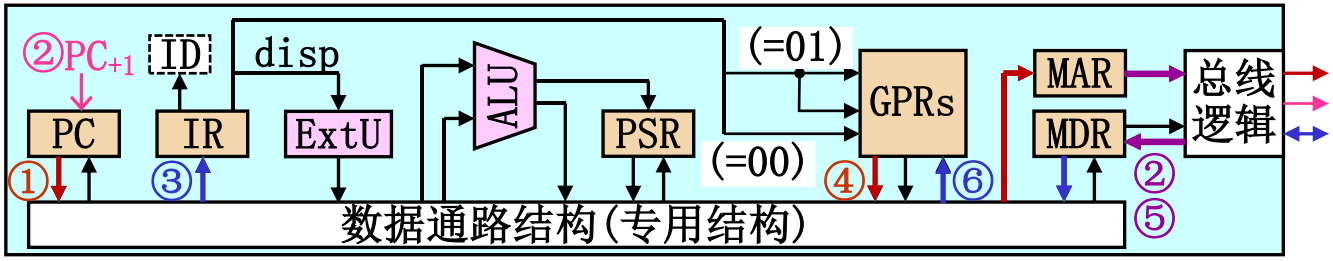
主存

*取数指令LD的执行过程：

取指令—功能为 实现IR←M[(PC)]、PC←(PC)＋"1"

操作有 ①MAR←(PC)，②MDR←M[(MAR)]，③IR←(MDR)

②PC←(PC)＋1



教材P153

结果为 (IR)＝24H，(PC)＝11H，其余不变/不关注

指令译码—功能为 识别当前指令功能、产生操作控制信号

结果为 指令为R1←M[(R0)]，控制信号为读REG-读MEM-写GPR

执行指令—功能为 实现当前指令的约定功能(数据操作&下条指令地址计算)

数据操作有 ④MAR←(R0)，⑤MDR←M[(MAR)]，⑥R1←(MDR)

数据结果为 (R1)＝48H，其余不变(数据传送指令不写PSR)

指令地址计算操作有 无操作(顺序型指令取指时已完成)

*存数指令ST的执行过程：

取指令—功能为 实现IR←M[(PC)]、PC←(PC)＋"1"

操作有 （同LD指令[①~③]）

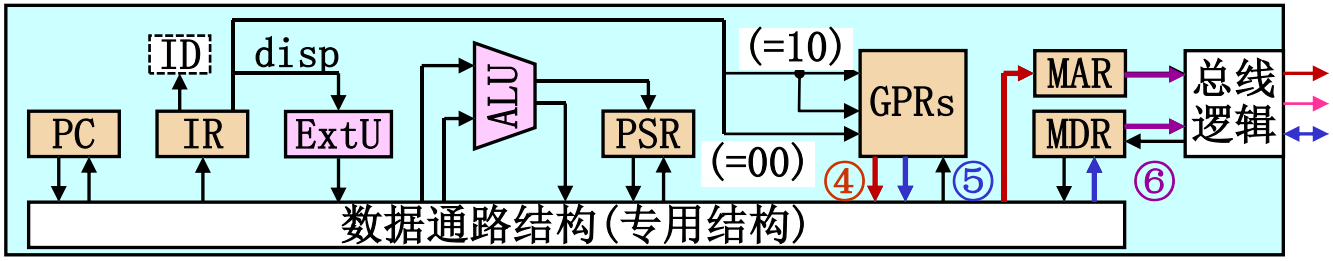
结果为 (IR)＝38H，(PC)＝12H，其余不变/不关注

指令译码—功能为 识别当前指令功能、产生操作控制信号

结果为 指令为M[(R0)]←(R2)，控制信号为读GPR-写MEM

执行指令—功能为 实现当前指令的约定功能(数据操作&下条指令地址计算)

数据操作有 ④MAR←(R0)，⑤MDR←(R2)，⑥M[(MAR)]←(MDR)



数据结果为 M[20H]＝30H，其余不变(数据传送指令不写PSR)

指令地址计算操作有 无操作(ST为顺序型指令)

*减法指令SUB的执行过程：

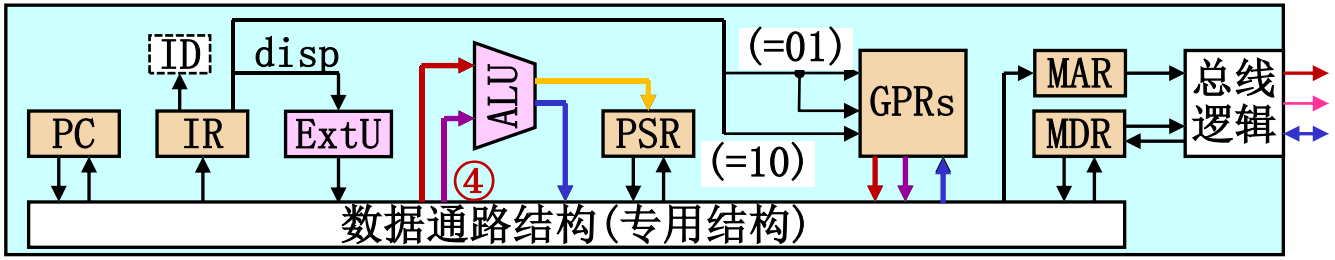
取指令—功能为 实现IR←M[(PC)]、PC←(PC)＋"1"

操作有（同LD指令）

结果为 (IR)＝66H，(PC)＝13H，其余不变

指令译码—结果为 指令为R1←(R1)－(R2)，控制信号为ALU-写GPR

执行指令—数据操作有 ④R1←(R1)－(R2)



数据结果为 (R1)＝18H，PSR.ZF/CF/SF/OF＝0/0/0/0

指令地址计算操作有 无操作(SUB为顺序型指令)

思考：若M[12H]=56H，执行阶段的操作及结果?

*分支指令JNZ的执行过程：

取指令—功能为 实现IR←M[(PC)]、PC←(PC)＋"1"

操作有（同SUB指令）

结果为 (IR)＝DEH，(PC)＝14H，其余不变

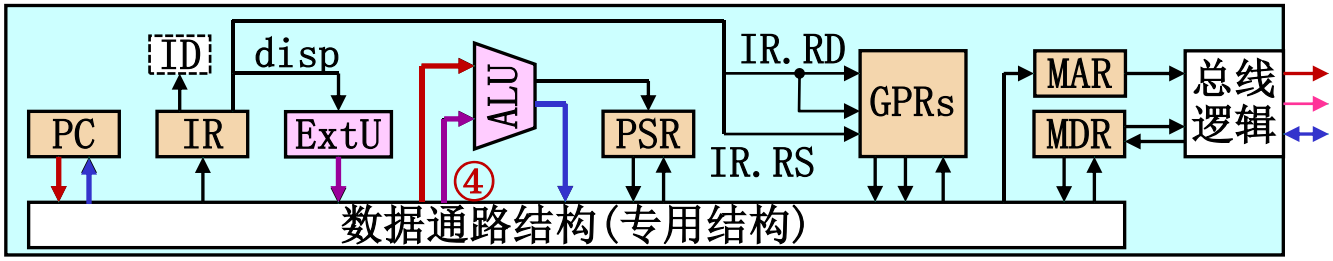
指令译码—结果为 指令为JNZ，disp＝1110，

控制信号为相对寻址-写PC （∵PSR.ZF=0）

执行指令—数据操作有 无(CU判断ZF&产生控制信号) ←CU判断更方便

数据结果为 无

指令地址计算操作有 ④PC←(PC)＋(ExtU)



指令地址计算结果为 (PC)＝12H

思考：若上条指令结果的ZF=1，执行阶段的操作及结果?

*指令执行过程的操作特征：

操作的特征—由取指令、执行指令阶段的操作组成



取指令的操作：对所有指令通用 ←如≤单字长的指令字

执行指令的操作：受操作码、寻址方式、PSW(程序状态字)影响

思考：多字长指令如何取指令？ 取指时仅取首字，其余字在执行阶段取
└←需含OP/寻址方式

基本操作的类型—

①REG间传送：$R_Y \leftarrow (R_X)$

②存储器读：$MDR \leftarrow M[(MAR)]$

③存储器写：$M[(MAR)] \leftarrow (MDR)$

④算逻运算：$R_D \leftarrow (R_{S1})$ op (R_{S2})

基本操作的组成要求：
①源数据/结果放在时序部件中
②操作功能具有原子性

微操作(μOP)—CPU内部的原子操作
微操作控制信号—实现μOP的部件控制信号，又称微操作命令(μOPCmd)

3、指令周期的组织

*指令执行过程的操作定时：

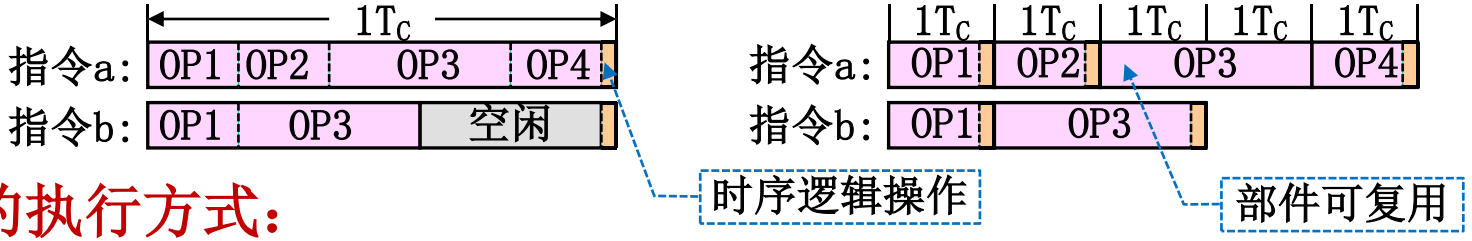
时间基准—主时钟脉冲信号，其宽度称为主时钟周期(T_C)

操作定时—时长$t_{OP}=kT_C$，k为整数变量(不同操作的时延不同)

*指令周期的组织：

单周期CPU—指令周期$=1T_C$，$T_C=$max{各指令的ΣOP时长} ←实现简单

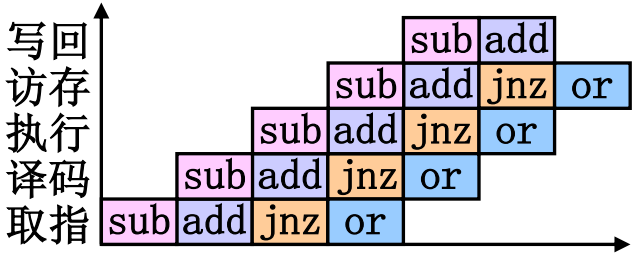
多周期CPU—指令周期$=nT_C$，$T_C=$min{指令中的各OP时长} ←性能好



*程序的执行方式：

串行方式—不同指令逐条执行

流水方式—不同指令执行过程重叠 ←相应硬件称为指令流水线



├→各个阶段的部件可同时处理不同指令的操作
|　　(不复用)
└→每条指令按序通过各个段(避免部件冲突)

※CPU的组成与原理小结：

基本功能—循环地执行指令、处理异常&中断　←实现工作方式&指令系统

细化为4个操作类功能、2个操作控制类功能

基本组成—(指令部件＋运算器＋BIU＋MMU＋中断机构)＋控制器，

划分为数据通路＋控制器CU，负责操作实现、操作控制

工作流程—循环地执行指令＋响应中断&异常，由多个有序的操作组成

└←中断在指令周期间、异常在指令周期中

工作原理—CU循环地产生工作流程所需的操作控制信号，　(5.3节讨论)

数据通路实现工作流程所需的操作　(5.2节讨论)

指令周期的组织—

指令执行的过程：由取指&执行阶段的基本操作(即μOP)序列组成



└←有REG间传送/MEM读写/算逻运算

微操作的定时：基于主时钟脉冲信号，时长可不等(基于操作)

指令周期的组织：可为1个/多个时钟周期，有串行/流水执行方式

§5.2 数据通路的组织

※主要内容：数据通路的组成，μOP及其实现，指令执行过程的组织，
数据通路的设计(部件/互连)

一、数据通路(DataPath)的组成

数据通路—指令执行过程中数据所经过的路径及路径上的部件

涉及指令/地址等←部件复用←┘

数据通路的组成—功能部件＋部件互连

(目标是实现工作流程所需的所有操作)　←相关于ISA

1、数据通路的功能部件

*数据通路部件的组成：　(仅讨论指令周期所需的所有操作)

取指阶段—PC、IR、Adder(地址加法器)、IMEM(指令MEM)等

执行阶段—ALU、GPRs、PSR、DMEM(数据MEM)等

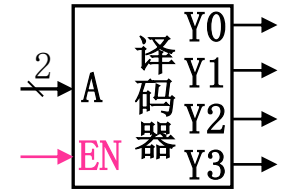
注意：①IMEM及DMEM可合并、∈存储系统

(即冯·诺依曼结构)(可放在CPU中[如Cache])

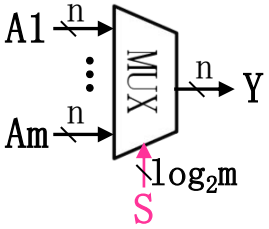
②不同ISA的部件组成不同

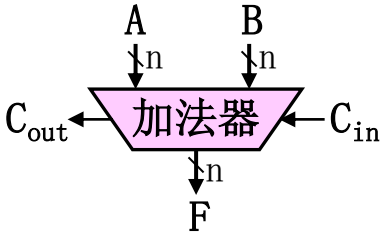
*常见功能部件回顾：

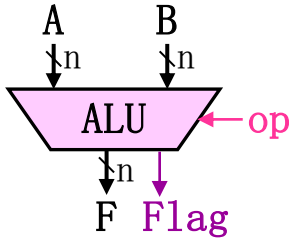
组合逻辑部件——译码器、多路选择器、加法器、ALU、扩展器等



EN=0时Y全无效





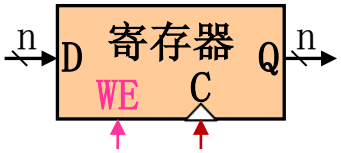




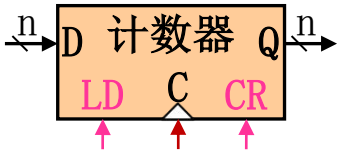
op=1常为符号扩展
仅1种功能时op缺省

时序逻辑部件——寄存器、计数器、寄存器组、存储器等

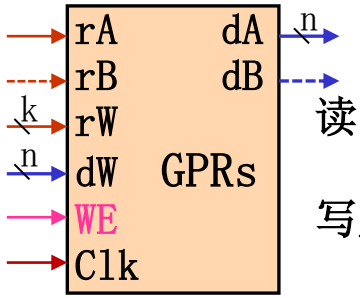
（仅写入为时序逻辑操作）　　└←有同步/异步2种



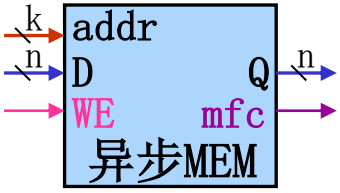
写入：WE=1&C=↑时
读出：不受C影响



异步清零：CR=1时Q=0
置数：LD=1&C=↑时Q=D
计数：LD=0&C↑时Q=Q+1



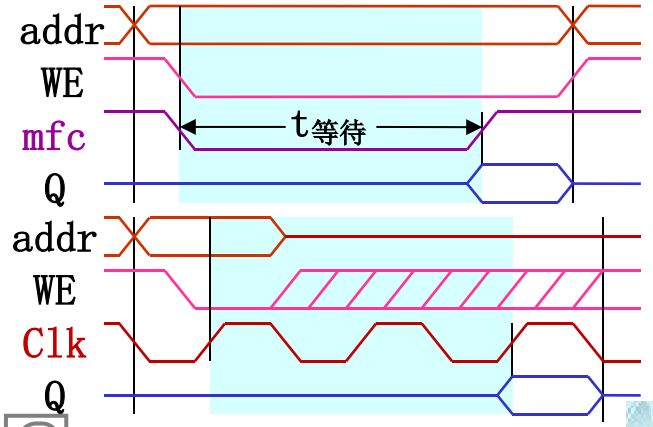
读出：dA=R[rA]，
　　dB=R[rB]
写入：WE=1&Clk=↑时，
　　R[rW]=D



写入：WE=1时M[addr]=D
读出：WE=0时Q=M[addr]
（异步控制，时延用mfc表示）



写入：WE=1&Clk=↑时M[addr]=D
读出：WE=0&Clk=↑时Q=M[addr]
（同步控制，时延用约定实现）



*数据通路部件的组成示例：（以Demo_IS为例）

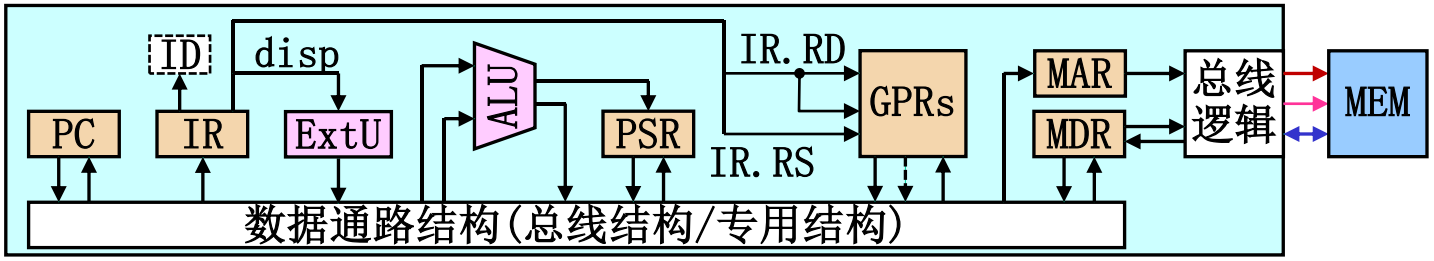
所支持指令—

（右图为部分指令）

指令功能	指令格式	指令寻址方式
取数(LD) ： RD←M[(RS)]	0010 RD RS	隐含EA=(PC)＋1
存数(ST) ： M[(RS)]←(RD)	0011 RD RS	隐含EA=(PC)＋1
加法(ADD)： RD←(RD)＋(RS)或	010 0 RD RS	隐含EA=(PC)＋1
RD←(RD)＋M[(RS)]	010 1 RD RS	隐含EA=(PC)＋1
减法(SUB)： RD←(RD)－(RS)	0110 RD RS	隐含EA=(PC)＋1
分支(JNZ)： ZF＝0时PC←(PC)＋Disp	1100 Disp	相对EA=(PC)＋Disp 或隐含EA=(PC)＋1

所需部件—取指阶段： PC、IR，MAR、MDR、MEM(冯·诺依曼结构)

执行阶段： ALU、ExtU(符号扩展)，GPRs、PSR



部件接口组织— 满足指令系统、部件互连结构的需要

如：ExtU仅一种功能，无需控制引脚；

总线结构时，GPRs仅1个读端口 ←稍后讨论

2、数据通路的部件互连

互连结构(数据通路结构)类型—有总线结构、专用结构(有称点点结构)

(1)总线结构的部件互连

*互连方式回顾：部件入端通过同一信号线(总线)连接其他部件出端

连接要求—部件出端增设三态门 ←总线上信号无冲突(同时发送≤1个)

部件入端增设暂存器 ←入-入、出-入信号无干扰(未锁存≤1个)

*部件互连方法：各部件均连接到总线上 (以Demo_IS为例)

用作缓冲器

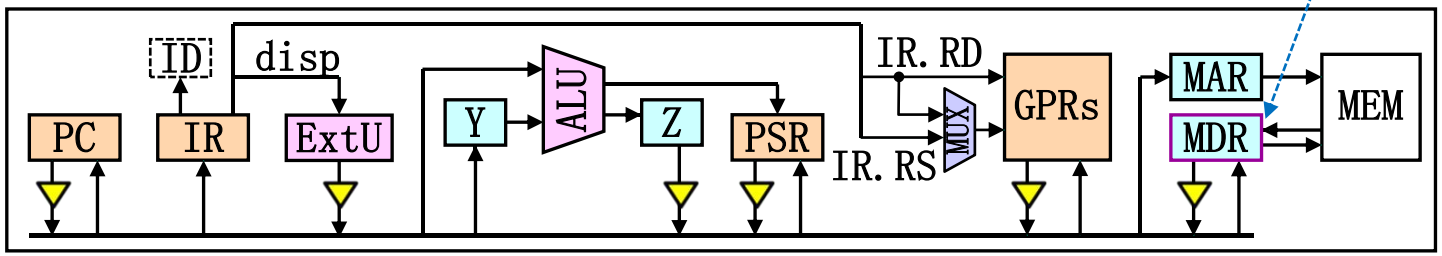


图5.7勘误：补上PSR

思考①：GPRs为什么只配置一个读端口？GPRs地址引脚前为何设置MUX？

没必要(同时发送≤1个) 源OPD有2个、地址来源不同

思考②：不设置Y有何问题？不设置Z有何问题？Z可放在ALU另一入端吗？

无法实现A+B，无法保存结果(反馈到入端) 可以(未锁存≤1个即可)

思考③：Y→ALU、IR→ExtU为何不设置三态门？ 非共用信号线

*数据传送特性：同时仅传送一个数据

思考④：R0←(R0)+(R1)所需操作？

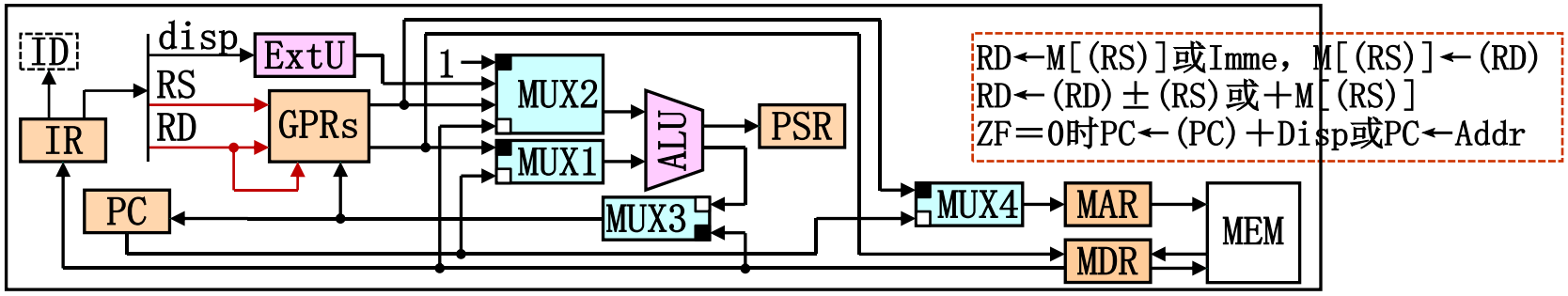
(2)专用结构的部件互连

*互连方式回顾： 部件入端通过不同信号线连接其他部件出端

连接要求—部件入端增设选择器、出端无要求

*部件互连方法： 数据路径满足各指令功能的要求 (以Demo_IS为例)

如：MAR←(PC)或GPRs，MDR←GPRs，GPRs←MDR或ALU，PC←ALU或MDR，…



思考①：GPRs为什么配置2个读端口？ 同时传送性能好

*数据传送特性： 同时可传送多个数据 思考②：R0←(R0)+(R1)需几个操作？

※数据通路组成小结： (目标是实现工作流程所需的所有操作)

部件组成—取指阶段部件＋执行阶段部件，不冲突时部件可复用

└←与指令功能相关

部件互连—有总线结构(入端/出端均连接总线)、专用结构(基于指令功能连接)

└←与指令功能无关

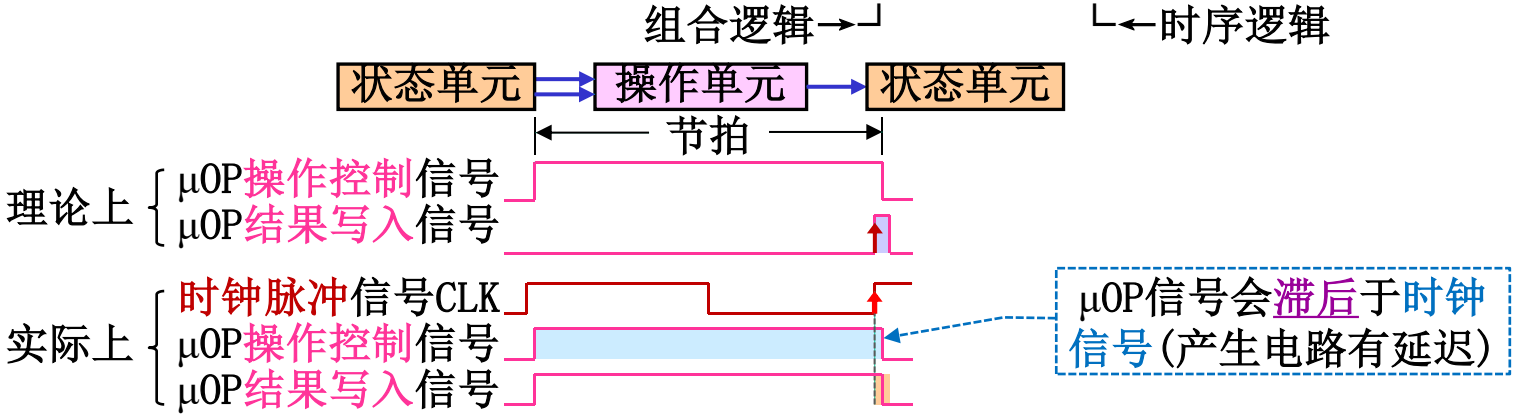
二、数据通路的微操作及其控制(实现)

微操作(μOP)—CPU内部的原子操作
微操作控制信号—实现μOP的部件控制信号，又称微操作命令(μOPCmd)
节拍—完成1个μOP所需的时间

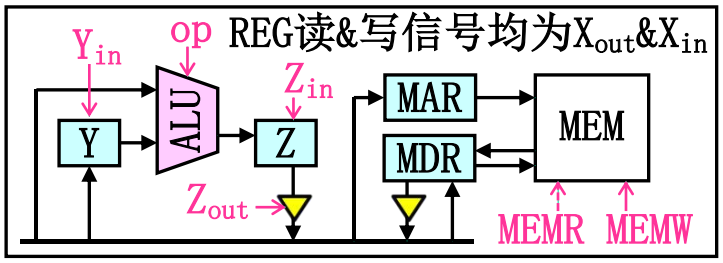
*μOP的组成要求：①源数据&结果都放在时序逻辑部件中 ←同基本操作
②操作为原子操作

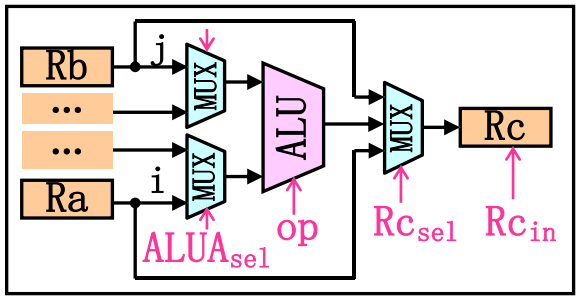
*μOP的种类：基本操作(REG间传送、MEM读、MEM写、算逻运算)，
特殊操作(PC增量、信号置位/复位等)

*μOP的实现方法：电位-脉冲制(对应于操作控制-结果写入)



*μOP的实现示例：（以基本操作为例）





寄存器间传送μOP—功能为Rc←(Ra)

总线结构时的实现：Ra_{out}、Rc_{in}　　如：Z_{out}、MAR_{in}

专用结构时的实现：Rc_{sel}=i、Rc_{in}

μOPCmd书写约定：写出的信号都有效(多种取值用=描述)，未写出的信号都无效

算逻运算μOP—功能为Rc←(Ra) op_n (Rb)

总线结构时的实现：Rb_{out}、op=op_n、Z_{in} [功能为Z←(Y) op_n (Rb)]

思考：Rc←(Ra) ⊕ (Rb)的μOPCmd序列是什么？

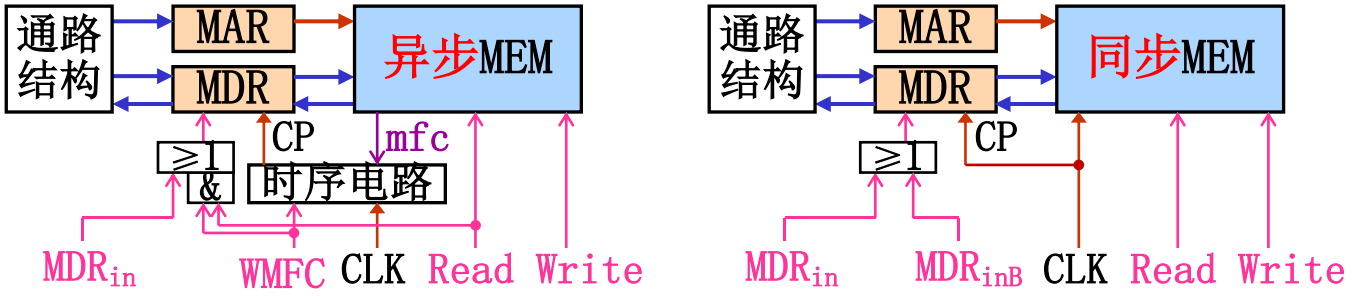
①Ra_{out}、Y_{in}　②Rb_{out}、op=⊕、Z_{in}　③Z_{out}、Rc_{in}

专用结构时的实现：$ALUA_{sel}$=i、$ALUB_{sel}$=j、op=op_n、Rc_{sel}=k、Rc_{in}

MEM读&MEM写μOP—（下页）

MEM读&MEM写μOP——功能为MDR←M[(MAR)]、M[(MAR)]←(MDR)

部件连接：仅与MEM控制方式(同步/异步)有关 ←与数据通路结构无关



取指令μOP序列：
①MAR←(PC)
②MDR←M[(MAR)]
③IR←(MDR)

思考：如何表示μOP是同步/异步方式？ 需设置专用uOPCmd(如WMFC)

操作控制：异步—读时自动写MDR(mfc及WMFC触发CP↑) ←1组μOPCmd

写时mfc指示完成(每个节拍由CP↑触发) ←1组μOPCmd

同步—读时第m个CLK写MDR(用MDR_{inB}控制) ←2组μOPCmd

写时第m个CLK完成 ←1组μOPCmd

μOP实现：异步—读为Read、WMFC；写为Write、WMFC

等到何时？CU中讨论

如：取指令①PC_{out}, MAR_{in} ②Read, WMFC ③MDR_{out}, IR_{in}

同步—读为①Read，②MDR_{inB}(第m个CLK时)；写为Write

如：取指令①PC_{out}, MAR_{in} ②Read ③ ④MDR_{inB} ⑤MDR_{out}, IR_{in}

三、指令执行过程的组织 —基于数据通路&指令功能

*目标：形成各条指令指令周期的控制需求(μOPCmd序列) ←基于μOP

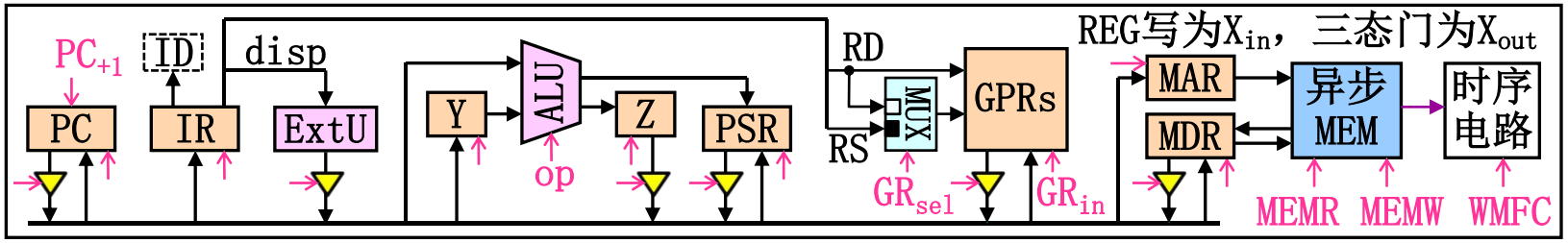
*组织要求：①μOP顺序按指令执行的步骤安排 ←保证正确性

└←如源OPD寻址+取OPD+操作+目OPD寻址+存OPD

②相邻&无关的μOP放在同一步骤中 ←缩短指令周期

*单总线结构数据通路的指令周期组织：(以Demo_IS为例)

假设：PC有计数功能，MUX中□、■表示控制信号为0、1时所选入端



思考：为何没有Y_{out}、ALU_{out}信号？

取指令阶段的μOP序列及μOPCmd序列—

t1：MAR←(PC)

t2：MDR←M[(MAR)],PC←(PC)+1

t3：IR←(MDR)

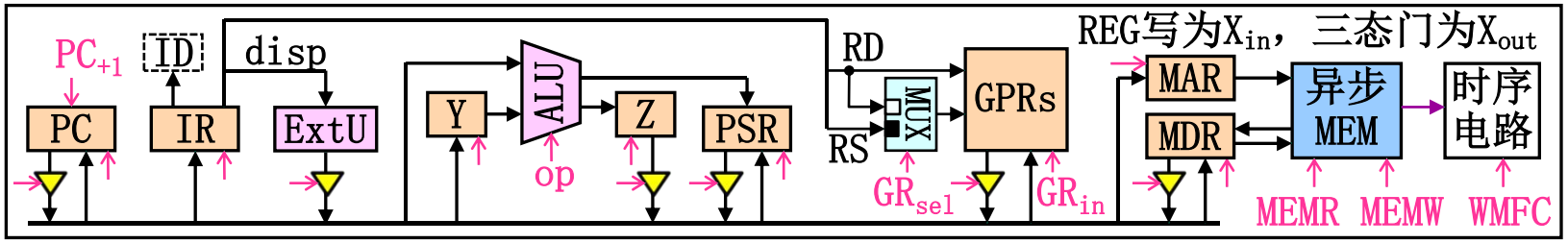
MAR在步骤结束时写，需分为2步

与MEM读无关

t1：PC_{out}、MAR_{in} ;其余均无效

t2：MEMR、WMFC，PC_{+1} ;其余

t3：MDR_{out}、IR_{in} ;后同



RD←M[(RS)]执行阶段的μOP序列及μOPCmd序列— (REG间接寻址)

t4：MAR←(RS)　　;OPD寻址　　t4：GR_{out}、GR_{sel}、MAR_{in}

t5：MDR←M[(MAR)]　;取OPD　　t5：MEMR、WMFC

t6：RD←(MDR)，End←1　　t6：MDR_{out}、GR_{in}，End

注—End信号表示指令周期是/否结束，用于触发中断请求的检测(5.6节讨论)

M[(RS)]←(RD)执行阶段的μOP序列及μOPCmd序列— (REG寻址)

t4：MAR←(RS)　　t4：GR_{out}、GR_{sel}、MAR_{in}

t5：MDR←(RD)　　t5：GR_{out}、MDR_{in}　;GR_{sel}=0

t6：M[(MAR)]←(MDR)，End←1　　t6：MEMW、WMFC，End

思考①：MDR←(RD)时，为何GR_{sel}=0？

思考②：哪些行为同步控制方式？

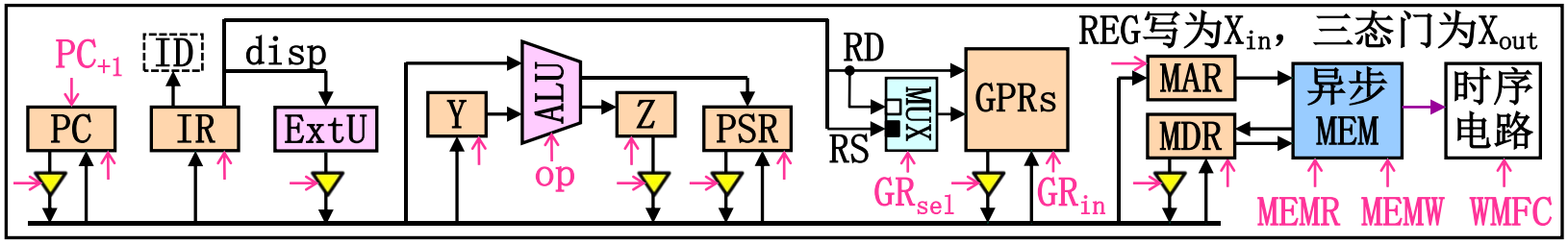
RD←(RD)－(RS)执行阶段的μOP序列及μOPCmd序列—

t4：Y←(RD)　　t4：GR_{out}、Y_{in}　;GR_{sel}=0

t5：Z←(Y)－(RS)　　t5：GR_{out}、GR_{sel}、op＝01、Z_{in}

t6：(RD)←(Z)，End←1　　t6：Z_{out}、GR_{in}，End

注—根据ALU功能送被加数/被减数(假设为Y对应端)，Y的输出无需控制



JNZ disp执行阶段的μOP序列及μOPCmd序列—（相对寻址）

ZF＝0时：

t4：Y←(PC)　　t4：PC_{out}、Y_{in}

t5：Z←(Y)＋(ExtU)　　t5：$ExtU_{out}$、op＝00、Z_{in}

t6：(PC)←(Z)，End←1　　t6：Z_{out}、PC_{in}，End

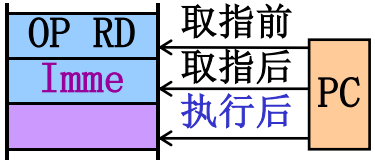
常见错误：Y_{out}、ALU_{in}、$ExtU_{out}$、ALU_{in}、op＝00、ALU_{out}、Z_{in}

ZF＝1时：

t4：End←1　　t4：End

思考：谁负责判断ZF的值？尝试在图中添线

RD←Imme执行阶段的μOP序列及μOPCmd序列—（双字长指令）



注—取指时仅取首个字，执行时取其余字；

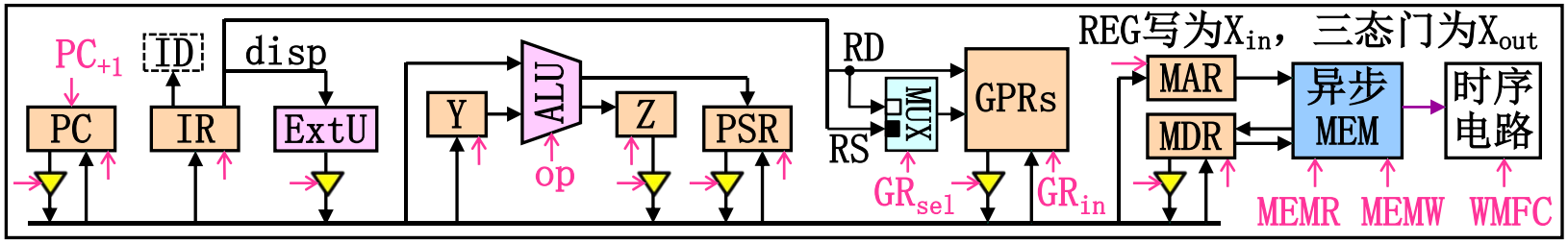
每取一个字，应及时修改PC

t4：MAR←(PC)　　t4：PC_{out}、MAR_{in}

t5：MDR←M[(MAR)],PC←(PC)＋1　　t5：MEMR、WMFC，PC_{+1}

t6：RD←(MDR)，End←1　　t6：MDR_{out}、GR_{in}，End

取数指令为RS



思考：下列μOPCmd序列的功能?

t4：GR_{out}、GR_{sel}、Y_{in}
t5：$ExtU_{out}$、GR_{out}、op=01、Z_{in}
t6：Z_{out}、GR_{in}，End

功能：RD←(RS)-未知值
(RD)与disp冲突→┘

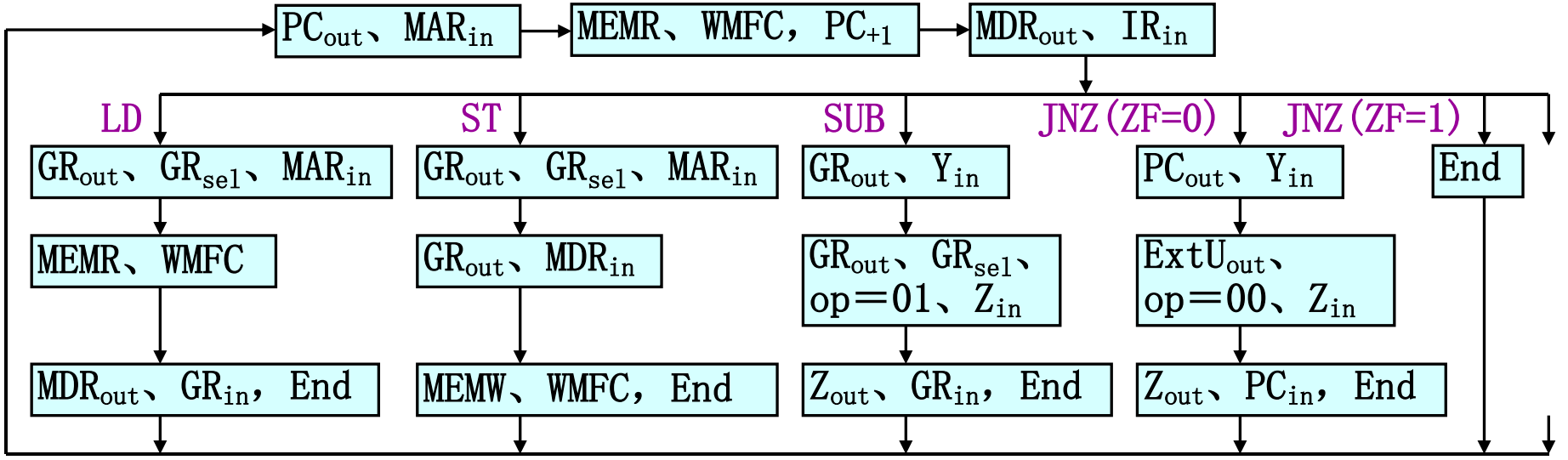
t4：PC_{out}、Y_{in}
t5：$ExtU_{out}$、GR_{in}
t6：GR_{out}、op=00、Z_{in}
t7：Z_{out}、PC_{in}，End

功能：RD←disp，
PC←(PC)+(RD)

t4：GR_{out}、GR_{sel}、MAR_{in}、MEMR
t5：WMFC
t6：MDR_{out}、GR_{in}，End

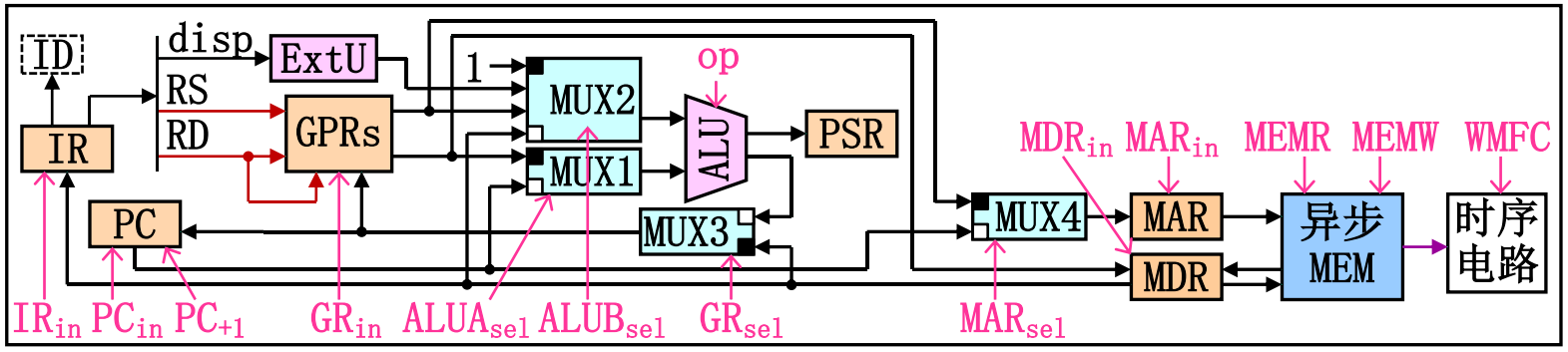
功能：RD←(MDR)，
MDR值不确定[t5无MEMR]

所有指令周期的组织结果—状态转换图(汇总各μOPCmd序列)



*专用结构数据通路的指令周期组织：（以Demo_IS为例）

假设：ALU有4种功能，PC有计数功能



取指令阶段的μOP序列及μOPCmd序列—

t1：MAR←(PC)　　t1：MAR_{sel}＝0、MAR_{in}

t2：MDR←M[(MAR)],PC←(PC)＋1　　t2：MEMR、WMFC，PC_{+1}

t3：IR←(MDR)　　t3：IR_{in}

M[(RS)]←(RD)执行阶段的μOP序列及μOPCmd序列—

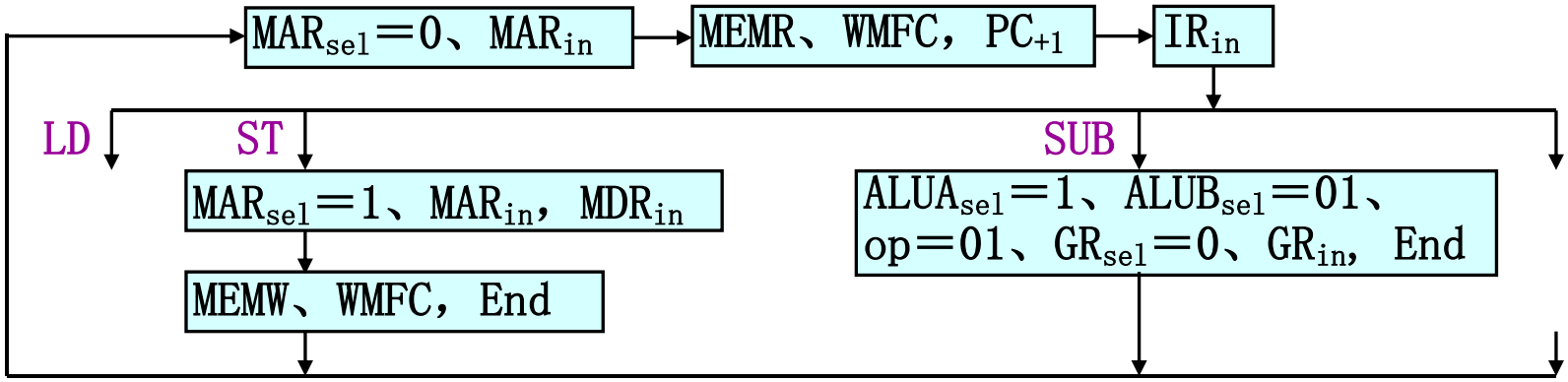
t4：MAR←(RS)，MDR←(RD)　　t4：MAR_{sel}＝1、MAR_{in}，MDR_{in}

t5：M[(MAR)]←(MDR)，End←1　　t5：MEMW、WMFC，End

RD←(RD)－(RS)执行阶段的μOP序列及μOPCmd序列—

t4：RD←(RD)－(RS)，End←1　　t4：$ALUA_{sel}$＝1、$ALUB_{sel}$＝01、op＝01、GR_{sel}＝0、GR_{in}，End

所有指令周期的组织结果—状态转换图(各μOPCmd序列的汇总)



※指令执行过程的组织小结：　(基于数据通路进行)

组织方法—每条指令单独组织(形成μOPCmd序列)；　←指令串行执行

μOP顺序按指令执行步骤安排，　←～互连结构/指令功能/PSW

相邻&无关的μOP放在同一步骤中

组织结果—各指令周期μOPCmd序列构成的状态转换图

CU的功能需求

(CU按序产生所需的μOPCmd→数据通路即可实现CPU功能)

作业5-1：P236— 3、5、6(1)和(4)

四、数据通路的设计

1、指令周期与数据通路结构

指令周期— $=\mathrm{CPI}\times T_C$

*单周期数据通路：CPI＝1，$T_C=\max\{T_{指令i}\}$



部件需求—部件不能复用(需要时重复设置)　←同一CLK内信号值不变

互连需求—专用结构　←减少结果暂存

*多周期数据通路：$T_C=\max\{T_{\mu OPj}\}$，CPI＝n(可变[取决于指令功能])

└←实际上$T_C=\max\{T_{基本\mu OPj}\}$　←稍后讨论

部件需求—部件可以复用(需要时暂存结果)　←不同CLK可改变信号值

互连需求—专用结构或总线结构

*特点：多周期CPU性能好(实际应用)，单周期CPU实现简单(用于教学)

2、数据通路的设计步骤

(1)指令系统分析

内容—数据表示、OPD存放、寻址方式、指令功能-指令格式等

结果—操作类型的种类、数据寻址/指令寻址的计算方法(含参数)，
寄存器的位数及个数，存储器的编址单位、地址空间等

(2)功能部件设计 --基于指令系统分析结果

数据操作单元— 实现操作类型、数据寻址需求

地址计算单元— 实现指令寻址需求 ←单周期DP不可复用操作单元

寄存器组— 实现REG需求(含PSR)， 读端口数～DP结构(单总线为1个)

存储器— 实现MEM需求(配置容量)， 数据位数＝CPU数据引脚数

特殊寄存器— 实现指令部件/BIU接口，个数～DP结构、指令周期

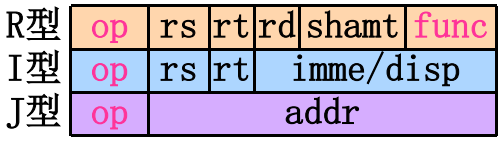
(3)部件互连设计 --基于功能部件设计结果、指令功能

内容—建立各条指令的数据路径(～指令功能、DP结构、复用方案)

├→总线结构：出端设置三态门，入端设置暂存器(1个未锁存)

└→专用结构：出端直连，入端设置选择器(所连出端＞1个时)

3、单周期数据通路的设计 （掌握设计过程、部件设计）



(1)指令系统分析 （以MIPS32的7条指令为例）

指令名	指令功能	功能说明
有符号加add	rd←(rs)＋(rt)	OPD为有符号整数(补码表示) 产生OF标志(不保存)
有符号减sub	rd←(rs)－(rt)	
按位或ori	rt←(rs) | ZExt(imme)	ZExt表示零扩展
取数lw	rt←M[(rs)＋SExt(disp)]	SExt表示符号扩展
存数sw	M[(rs)＋SExt(disp)]←(rt)	
相等转移beq	if ((rs)=(rt)) PC←(PC)＋4＋SExt(disp)<<2	无符号减法
跳转j	PC←$(PC)_{高4位}$ ‖ addr<<2	‖表示拼接，<<2扩大寻址范围

*结果：指令功能操作—32位的有符号加/减、按位或、无符号减，有符号加减产生OF标志

数据寻址操作—无符号加、位扩展(零/符号)

指令寻址操作—无符号加(2次/指令)、符号扩展、<<2、拼接

寄存器—32个×32位GPR，每条指令最多2次读、1次写

存储器—按字节编址，32位地址空间

见教材P155

(2)功能部件设计

功能包括数据操作单元、地址计算单元、GPRs、MEM、特殊REG
（指令操作/数据寻址）　（指令寻址）　（OPD存放部件）　（取指/BIU）

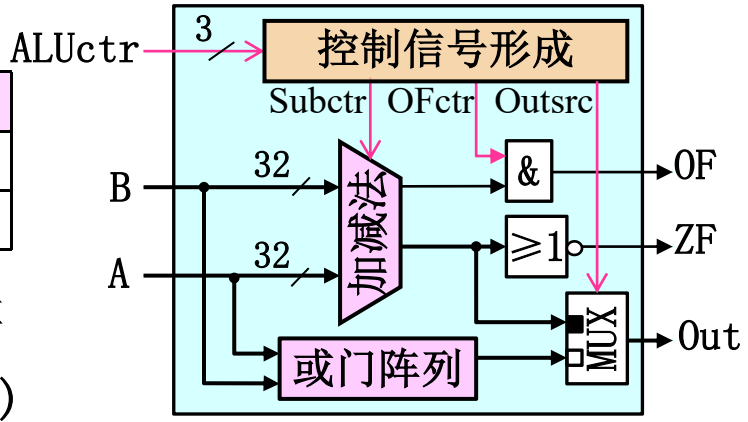
组织原则是相近功能合并(复用元器件/降低成本)，尽量模块化(简化控制)

*数据操作单元：　(基于分析结果设计)

ALU功能—支持5种运算(32位)，产生OF(有符号加/减时)、ZF

ALU设计—

ALUctr	000	001	010	011	100
功能	+有	−有	+无	−无	|
OF&ZF	√&√	√&√	×&√	×&√	×&√

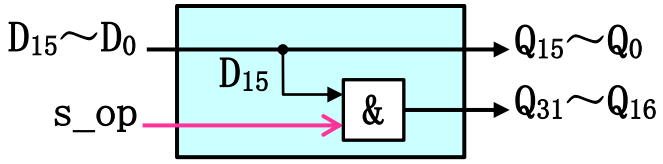


2个部件＋输出选择/标志形成逻辑

ExtU功能—支持零&符号扩展(16→32位)

ExtU设计—

D_{15} \ s_op	0(零扩展)	1(符号扩展)
0	$Q=0\cdots0D_{15}\sim D_0$	$Q=0\cdots0D_{15}\sim D_0$
1	$Q=0\cdots0D_{15}\sim D_0$	$Q=1\cdots1D_{15}\sim D_0$



*地址计算单元(ACU)：　(基于分析结果设计)

指令寻址需求—设j、beq指令的指令类型信号为Jump、Branch

Jump＝1时，NPC＝(PC)$_{高4位}$ ‖ addr<<2

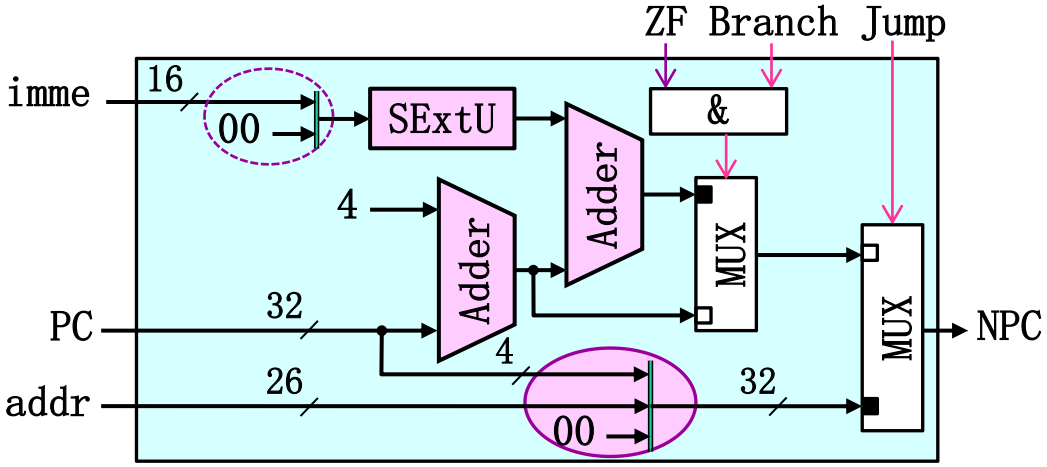
Branch•ZF＝1时，NPC＝(PC)＋4＋SExt(imme)<<2

否则，NPC＝(PC)＋4

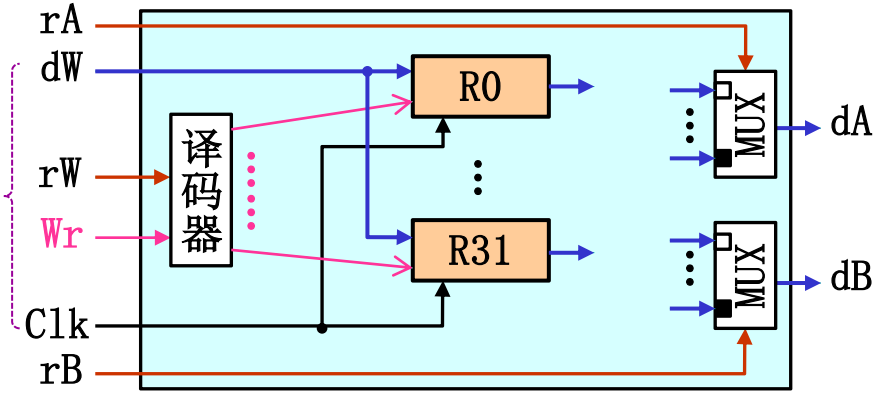
ACU功能—支持上述功能　←单周期DP不能复用ALU

ACU设计—3种功能＋输出选择　←内部不能复用部件

(SExtU可以包含<<2)

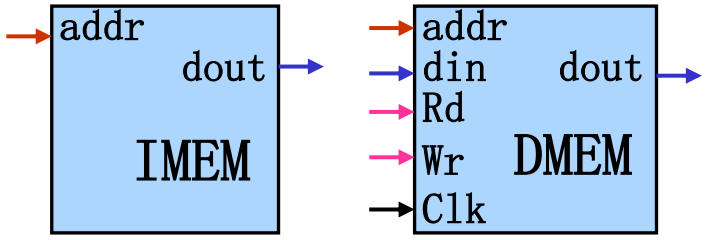


*GPRs设计：支持2个读端口、1个写端口 ←单周期DP需采用专用结构



*MEM组织：采用哈佛结构，数据宽度为32位 ←单周期DP不复用部件

└←lw/sw的MEM读/写用1个操作实现



假设：IMEM为异步RAM(降低复杂度)

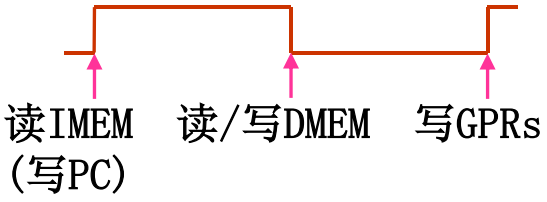
DMEM为同步RAM(无mfc)

组成：多种方案

*特殊寄存器：(指令部件/BIU接口等)

需求—尽量减少时序操作(仅有2个边沿可用)

设计—仅设置PC(缺省IR/MAR/MDR)



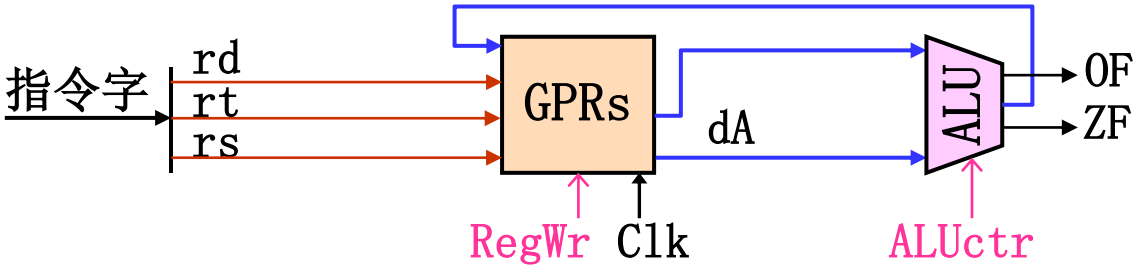
(3)部件互连设计

*设计方法：建立每条指令的数据路径，边设计边汇总(或后汇总)；
采用专用结构互连 ←单周期DP

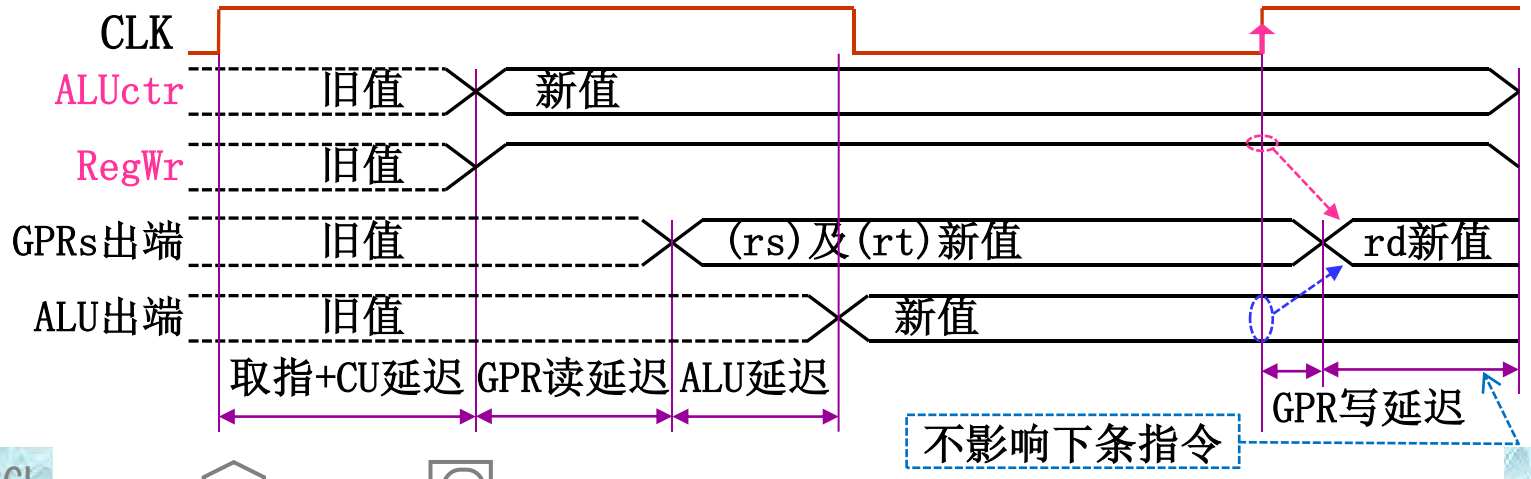
*add/sub指令：rd←(rs)＋(rt)及rd←(rs)－(rt)

数据路径设计— (暂不考虑取指)

(rs)连接ALU的A端

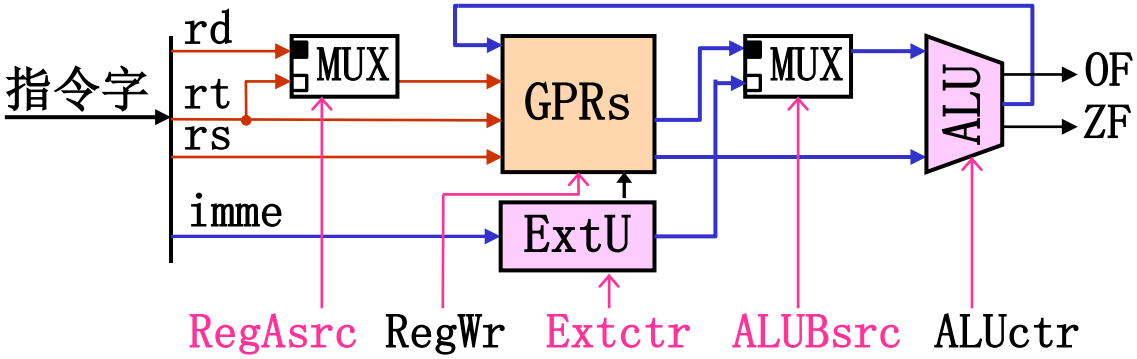


μOP时序组织— 写GPRs放在T_C结束时 ←与lw指令统一处理(lw在T_C中部读MEM)



*ori指令：rt←(rs)| ZExt(imme)

数据路径设计—增加ExtU相关路径

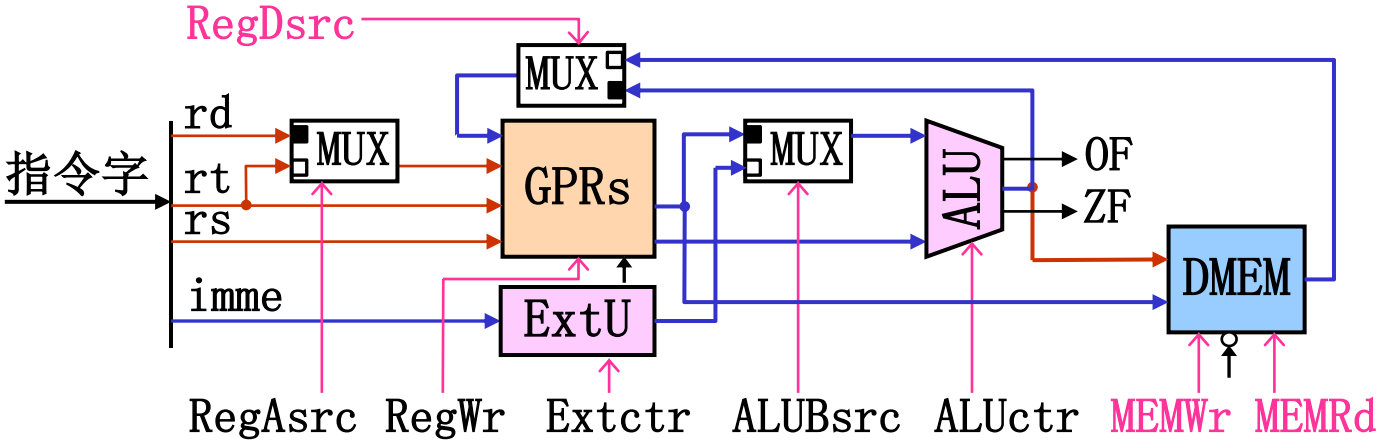


μOP时序组织—同add/sub指令

*lw/sw指令：rt←M[(rs)＋SExt(imme)]及

M[(rs)＋SExt(imme)]←(rt)

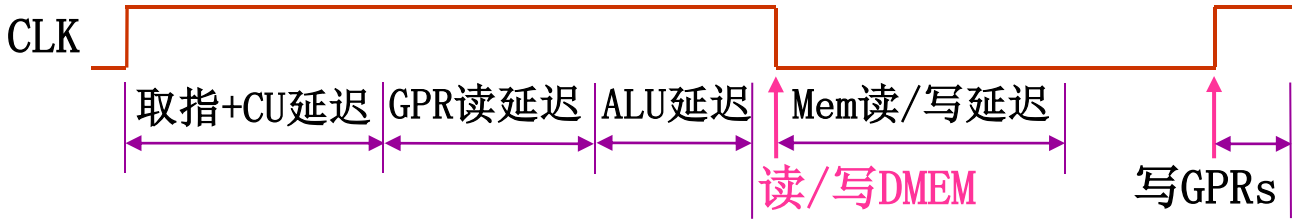
数据路径设计—增加DMEM读/写相关路径



μOP时序组织—DMEM操作放在T_C中部，　←同步MEM

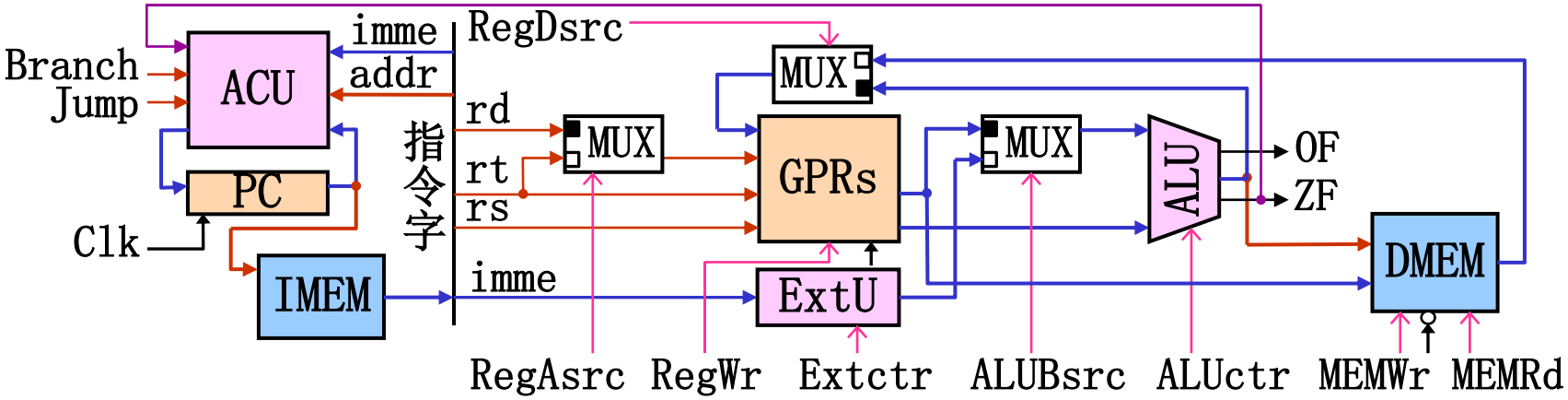
└→地址计算在前半周期完成

写GPRs放在T_C结束时　←2次时序操作

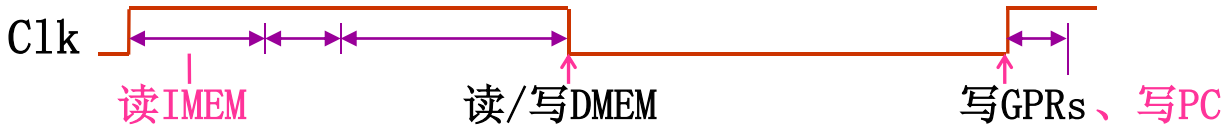


*beq指令：PC＝((rs)=(rt))？(PC)＋4＋SExt(imme)<<2 :(PC)＋4

数据路径设计—相等比较用ALU实现(产生ZF)、地址计算用ACU实现



μOP时序组织—写PC放在T_C结束时(上升沿后读)　←IMEM为异步MEM



*j指令：PC＝PC高4位 ‖ addr<<2

思考：IMEM为同步RAM时，何时写PC？

T_C中部(上升沿时读)

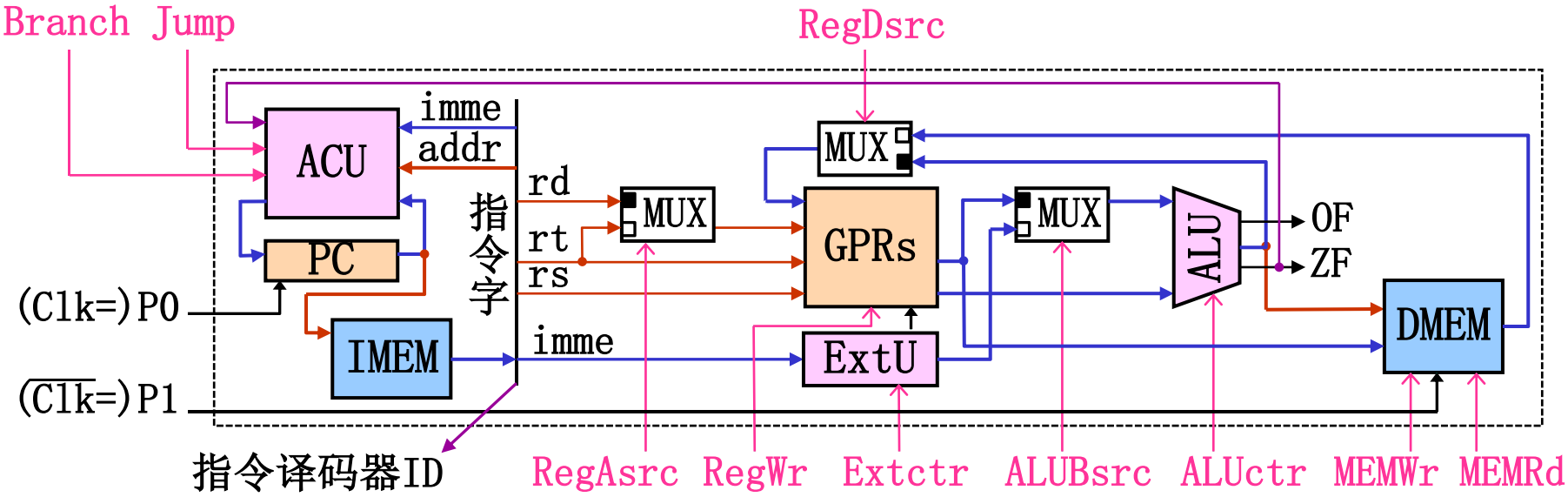
数据路径设计—同beq指令(无数据操作)

μOP时序组织—同beq指令(写PC时机相同)

*取指令数据通路：增设IMEM

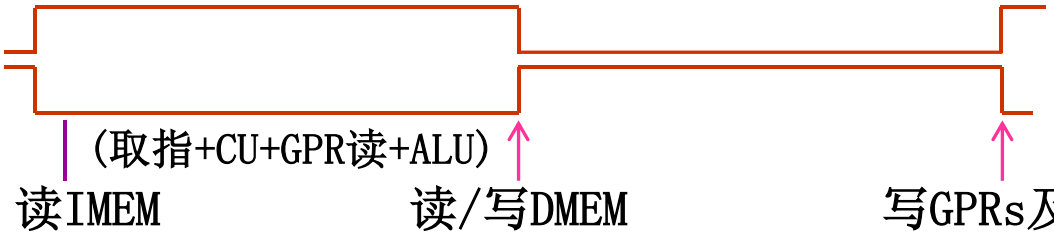
*设计结果汇总：

电路组成—



接口信号— μOPCmd(10个)，工作脉冲(2个)，操作码 ←与CU之间

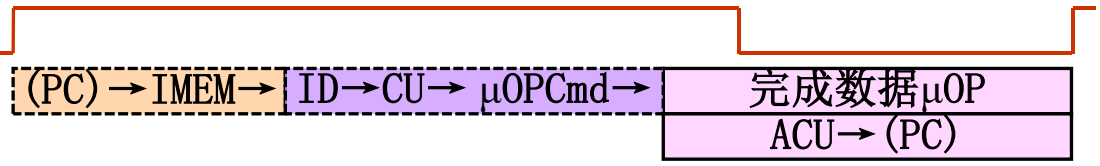
操作时序—



(4) 指令执行过程的组织　(数据通路设计的验证)

***指令执行过程的操作：** Clk

Clk时长＝max{$T_{指令i}$}

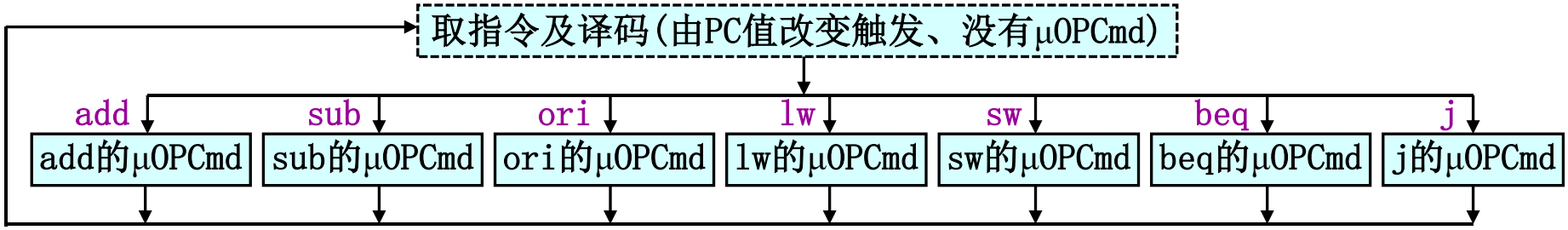


***各指令周期的μOPCmd序列：** 仅1步/指令　←单周期DP，取指/译码无μOPCmd

	Extctr	ALUBsrc	ALUctr	RegAsrc	RegDsrc	RegWr	MemRd	MemWr
add	×	1	00	1	1	1	0	0
sub	×	1	01	1	1	1	0	0
ori	0	0	10	0	1	1	0	0
lw	1	0	00	0	0	1	1	0
sw	1	0	00	×	×	0	0	1
beq	×	1	11	×	×	0	0	0
j	×	×	×	×	×	0	0	0

注意：与路径无关的信号中，时序逻辑部件的μOPCmd为无效值

***数据通路的状态转换图：** 仅1种状态/指令　←取指/译码无μOPCmd



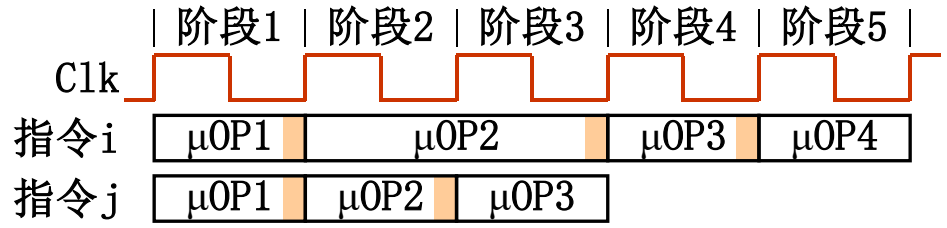
作业5-2：P237— 9、10

4、多周期数据通路的设计　（掌握设计过程）

***多周期数据通路的设计思想：**

(1)指令周期＝n个阶段，每个阶段＝1个时钟周期　←n可变

(2)其他阶段所需的结果须保存在状态部件(如附加REG)中



***时钟周期的确定：**

理论上—$T_C=\max\{T_{\mu OPi}\}$，任意μOP＝1个T_C　←T_C较大

实际上—$T_C=\max\{T_{基本\mu OPi}\}$，任意μOP＝1~*p*个$T_C$

思考：μOP时长如何确定？

同步控制方式时，*p*为常数，需≥1组μOPCmd

异步控制方式时，*p*为变量，需1组μOPCmd

***设计方法：**　（常基于单周期数据通路改进）

基于部件复用方案，组织各功能部件；

基于μOP，划分指令周期，并设置附加REG；

基于互连结构&指令功能，连接各功能部件

(1)功能部件设计　　--以MIPS为例(改进单周期数据通路)

***操作部件组织：**

部件复用方案—有多种，本例为ACU功能由ALU、ExtU等实现

功能部件组成—ALU/ExtU/GPRs/IMEM/DMEM，增加SL2、Splice

假设：IMEM及DMEM都为同步RAM、时延＝ALU(简化设计)

***指令周期阶段组织：**　(基于最复杂指令的执行步骤)



注：读GPRs含位扩展操作

***附加寄存器组织：**　(保存其他阶段所需信息)

①确定时钟周期长度—　(可完成哪些部件操作/时钟周期)　←便于操作合并

结果：max{GPRs读/写、ALU、ExtU等}，不含MEM操作(CPU外部)

②组织各个μOP的功能—　(可完成哪些部件操作/μOP)　←基于时钟周期

结果：含1个GPRs/ALU/MEM操作，包容ExtU、Splice等操作

③设置附加寄存器—　(每个μOP结束时设置[不复用时可缺省])

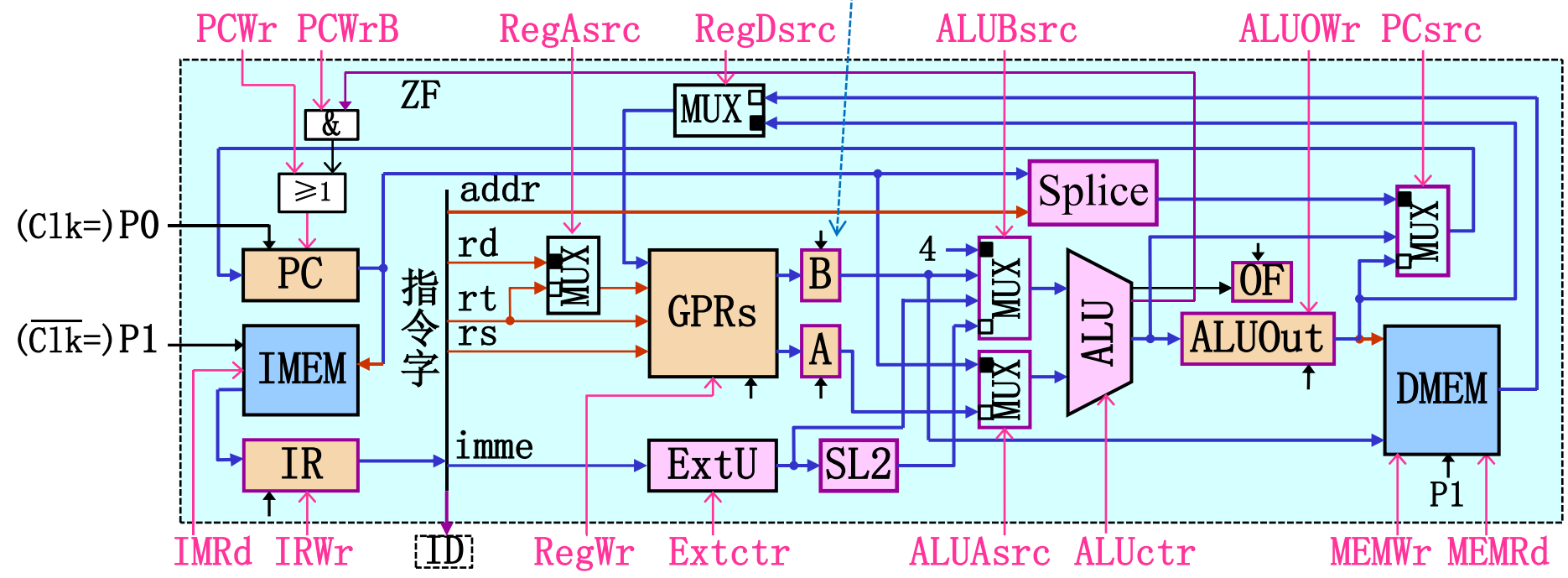
理论结果：在IMEM、GPRs、ALU、DMEM之后设置

实际方案：设置IR、A和B、ALUOut和OF，缺省DMDR(优化MEM写性能)

(2) 部件互连设计

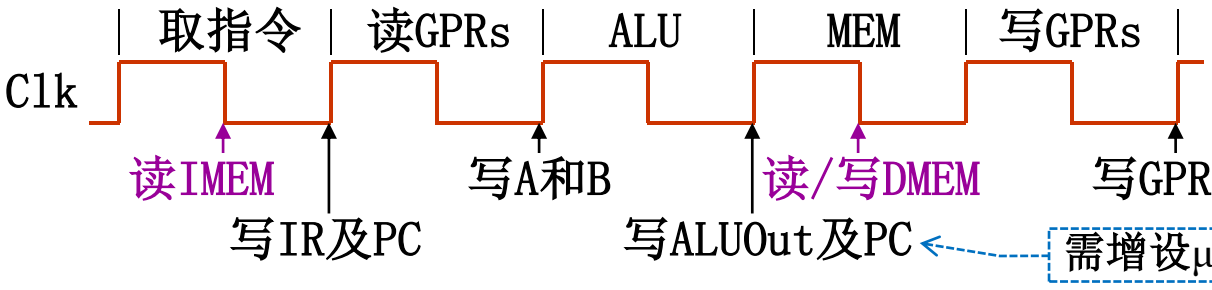
可缺省(执行阶段不变)，留便于理解uOP

***数据路径的修改：** 增加SL2、Splice、附加REG，及相关数据路径



***μOP时序的组织：** 附加REG在T_C结束时写入，MEM在T_C中部操作

与PC、ALUOut错开



需增设μOPCmd

(3)指令执行过程的组织　(数据通路的控制需求/设计验证)

***取指令阶段的μOPCmd序列：** μOP为IR←M[(PC)]、PC←(PC)＋4

t1：IMRd、~~IWMFC、~~IRWr，　←勘误：删除IWMFC(同步控制方式)

ALUAsrc＝1、ALUBsrc＝3、ALUctr＝0、PCsrc＝1、PCWr

***执行指令阶段的μOPCmd序列：**

add/sub—μOP为A&B←GPRs、ALUOut←(A)±(B)、GPRs←ALUOut

t2：无　；μOP为A←(rs)、B←(rt)　←A、B无控制信号

t3：ALUAsrc＝0、ALUBsrc＝2、ALUctr＝0/1、ALUOWr

t4：RegAsrc＝1、RegDsrc＝1、RegWr，End

ori—μOP为A←GPRs、ALUOut←(A)|(ExtU)、GPRs←ALUOut

t2：无　；μOP为为A←(rs)

t3：ALUAsrc＝0、ALUBsrc＝1、ALUctr＝2、ALUOWr

t4：RegAsrc＝0、RegDsrc＝1、RegWr，End

***分析指令阶段的安排：**　(稍后讨论)

lw—μOP为A←GPRs、ALUOut←(A)＋(ExtU)、GPRs←M[(ALUOut)]

未设置MDR→┴→μOP占2T_C

t2：无　　；μOP为为A←(rs)

t3：Extctr、ALUAsrc＝0、ALUBsrc＝1、ALUctr＝0、ALUOWr

t4：MemRd、~~WMFC~~　　←勘误：删除WMFC及MemRd(同步控制方式)

t5：~~MemRd、~~RegAsrc＝0、RegDsrc＝0、RegWr，End

sw—μOP为A&B←GPRs、ALUOut←(A)＋(ExtU)、M[(ALUOut)]←(B)

t2：无　　；μOP为为A←(rs)、B←(rt)

t3：Extctr、ALUAsrc＝0、ALUBsrc＝1、ALUctr＝0、ALUOWr　←(B)值不变

t4：MemWr~~、WMFC~~，End　　←勘误：删除WMFC(同步控制方式)

beq—μOP为A&B←GPRs、ALUOut←(PC)＋(ExtU)<<2、

(A)－(B)＝0时PC←(ALUOut)　←即ZF＝1时

t2：Extctr、ALUAsrc＝1、ALUBsrc＝0、ALUctr＝0、ALUOWr

；μOP还包括A←(rs)、B←(rt)

t3：ALUAsrc＝0、ALUBsrc＝2、ALUctr＝3、PCsrc＝0、PCWrB，End

j—μOP为PC←(Splice)

t2：无　　；等待指令译码结果

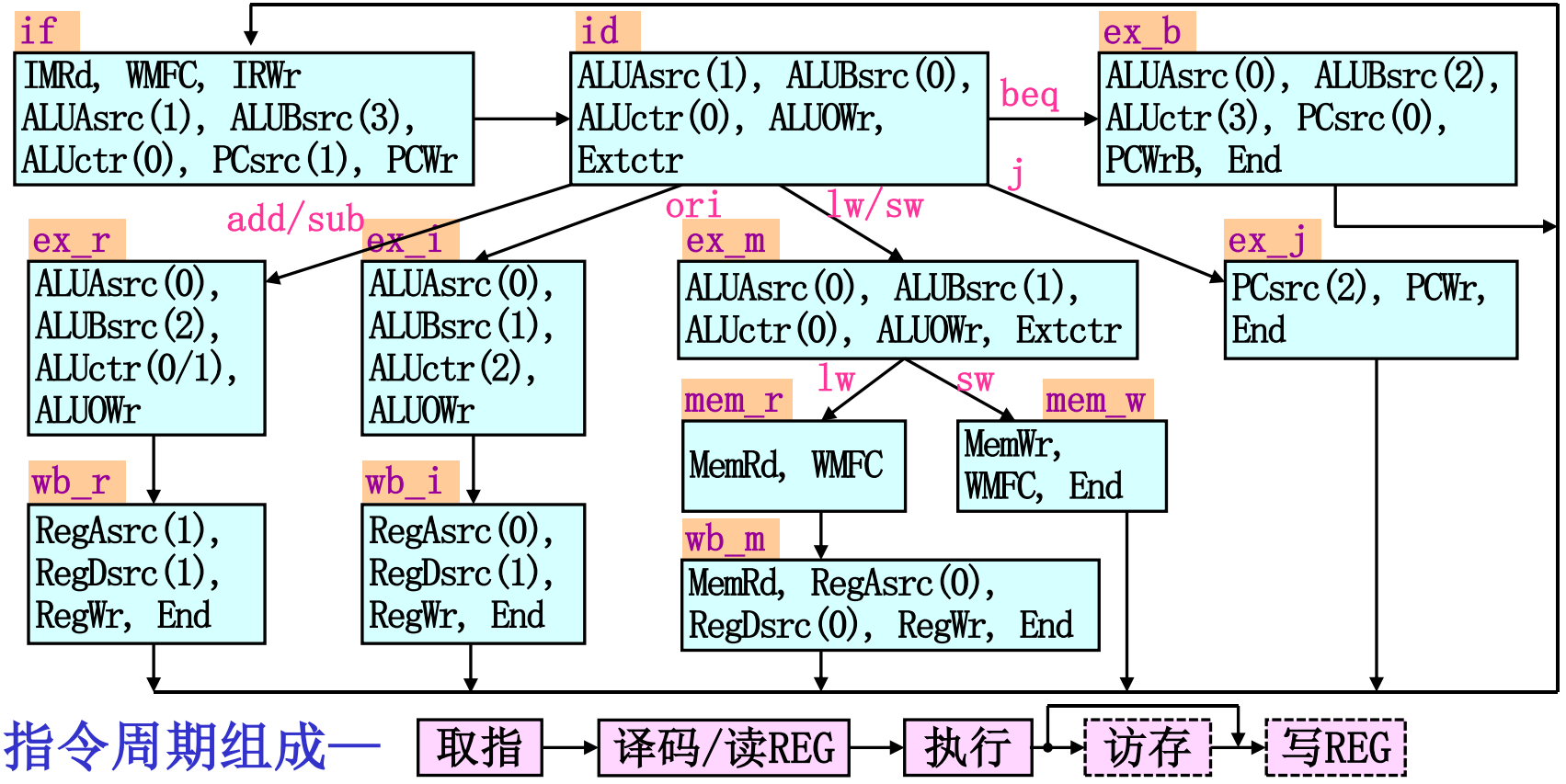
t3：PCsrc＝2、PCWr，End

*分析指令阶段的安排：适于所有指令，放在t2步(不影响其操作)

t2步功能—指令译码、A←(rs)、B←(rt)、ALUOut←(PC)＋imme<<2

t2步μOPCmd— Extctr、ALUAsrc＝1、ALUBsrc＝0、ALUctr＝0、ALUOWr

*数据通路的状态转换图：汇总各指令的μOPCmd序列



指令周期组成— 取指 → 译码/读REG → 执行 → 访存 → 写REG

※数据通路的组织小结：

目标—实现CPU工作流程所需的操作

└←指令执行＋中断响应(可缺省)

数据通路的组成—　(工作流程的硬件需求[基于执行过程&指令功能])

部件：取指部件(PC/IR等)＋执行部件(对应于指令功能)

互连：结构有总线(增设三态门&暂存器)、专用(增设选择器)，

方法为逐条增加各指令的数据路径

μOP及其控制—　(操作的实现方法[基于部件功能+控制])

μOP的类型(基本操作+特殊操作)，μOP的实现(发送相应μOPCmd)

指令执行过程的组织—　(工作流程的控制需求[基于硬件&指令功能])

基于已有数据通路，设计各指令周期的μOPCmd序列

数据通路的设计—

步骤(ISA分析→部件设计→互连设计)，设计&指令执行过程组织

CU的功能需求

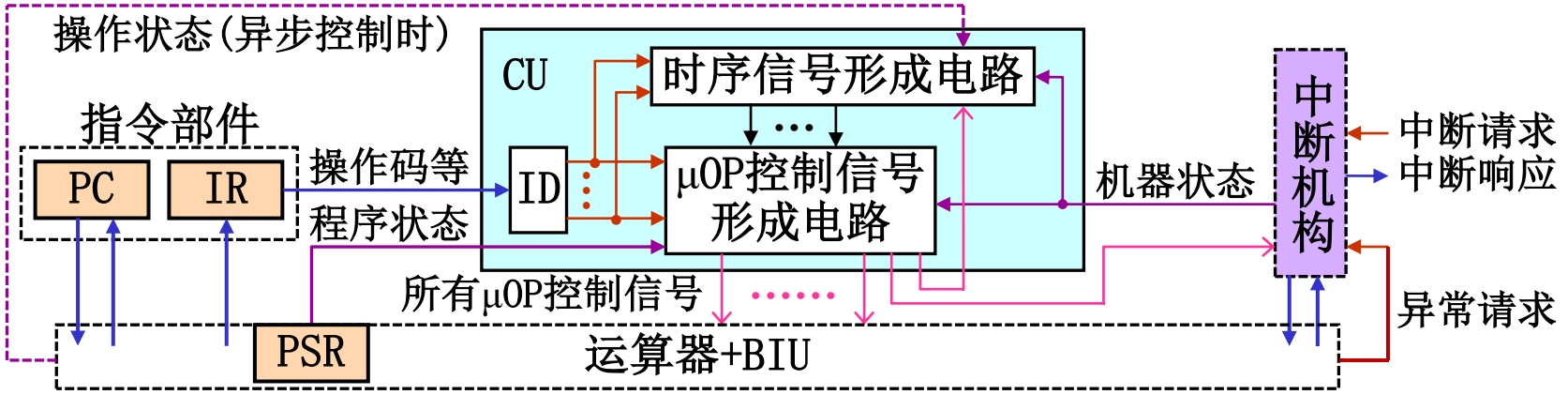
§5.3 控制器的组成

※主要内容：基本结构，时序信号的形成(组织/实现)，μOP控制信号的形成

一、控制器的基本结构

*目标：实现CPU工作流程所需操作的操作控制(有序地产生μOP控制信号)

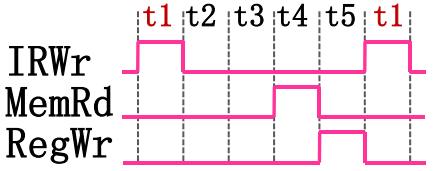
*基本组成：仅控制单元CU，指令部件&中断机构划入数据通路(路径重叠)



*工作原理：

时序信号形成电路循环&有序地产生各个时序信号

表示不同的步骤→┘

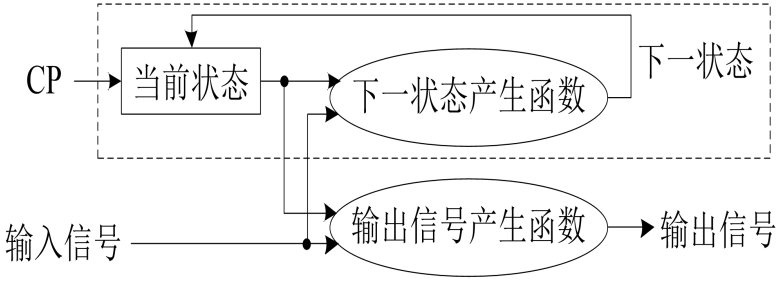


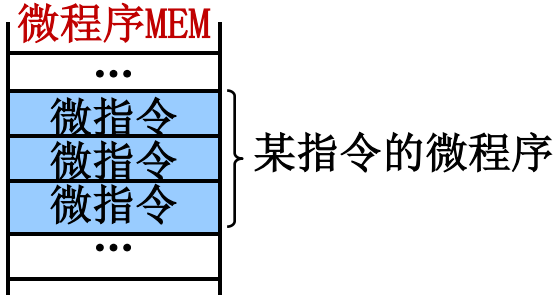
μOP控制信号形成电路实现所有μOP控制信号各个步骤的逻辑函数

└→与时序信号绑定←┘

*控制器的类型：（基于μOP控制信号的产生方法）

属性 / 类型	μOP控制信号的描述方法	时序信号的循环周期	μOP控制信号的产生方法	特点
硬布线控制器	有限状态机	1个指令周期	组合逻辑	速度快
微程序控制器	微程序	1个微指令周期	存储逻辑	实现简单
注：(1)1个微程序＝1条指令的μOPCmd序列　(2)1个微指令周期＞1个μOP的时延				





二、时序信号的形成　　--时序信号形成电路的组织

时序系统—表示时间组成的各种时序信号序列(时序信号形成电路的输出)

1、时序系统的组织　(时序系统的设计)

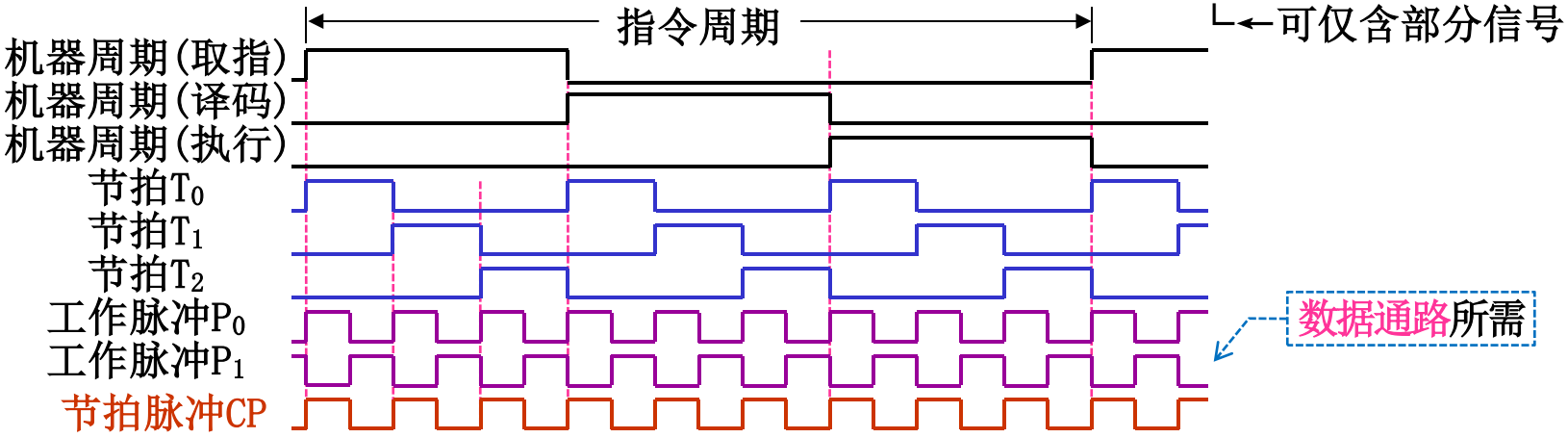
***时序系统的组成：**

时序信号类型—有机器周期、节拍(对应1个μOP)、工作脉冲(μOP内部的↑)

└←对应1个基本功能(含多个μOP)

信号同步：机器周期＝Σ节拍$_i$，节拍＝Σ工作脉冲$_j$ ←无间歇/重叠

时序信号序列—每级有多个信号，各级时序信号按一定次序组成



时序信号的定时—基于节拍脉冲CP，CP宽度＝节拍周期

节拍周期—节拍信号的宽度，＝1个μOP的时延

*早期计算机的时序系统组织：

信号个数—三级时序，各级按最复杂情况设置 ←面向各种指令需求

例：若指令周期＝ x个机器周期，x＝{3, 5}；

机器周期＝m个节拍周期，m＝{3, 4}；

节拍周期＝ k个工作脉冲， k＝2(μOP需使用CLK的↓及↑)

则机器周期、节拍、工作脉冲信号各为≥6个(含中断周期)、4个、2个

信号功能—表示操作次序、操作类型，常为后者(可简化电路)

信号定时—基于节拍脉冲 ←节拍脉冲宽度～uOP时延

信号序列组成—由各级信号组成，可有多种信号序列

满足不同指令需求→┼←权衡性能&成本

└→每级信号的个数可不同

信号的循环周期：有定长型(每级含所有信号)、变长型(每级含部分信号)

└←1种序列时 └←＞1种序列时

注意：工作脉冲均为定长型(数量少/简化实现)

*现代计算机的时序系统组织：

信号个数— 两级时序(节拍/工作脉冲)， 各级按最复杂情况设置

信号功能— 常表示操作类型 ←可简化电路

信号定时— 基于节拍脉冲 ←节拍周期＝uOP时延

信号序列组成— 常有多种序列(节拍为变长型&工作脉冲为定长型)

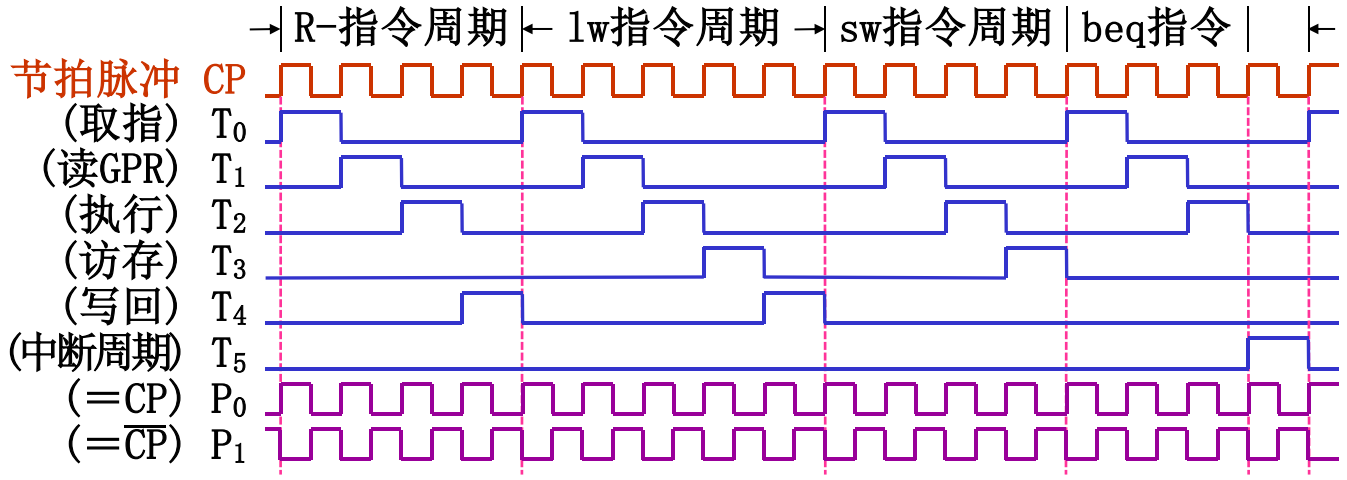
└←～uOP序列长度 └←简化实现

例：若时序系统支持7条MIPS指令(P47状态转换图)及中断处理，则

信号个数为：≥6个节拍(含中断处理)、2个工作脉冲(DP仅使用↑和↓)

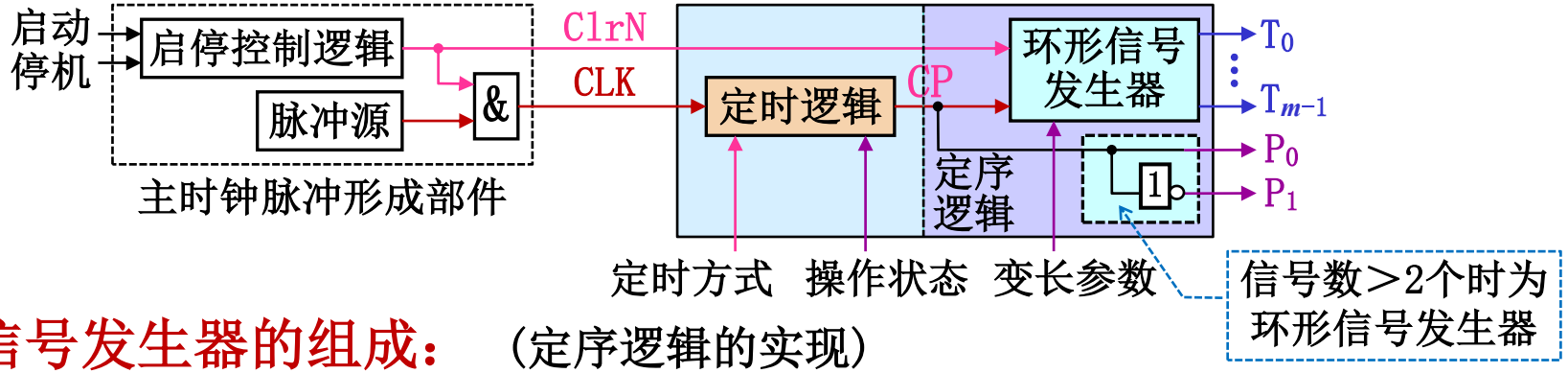
信号功能为：操作类型　　信号定时为：节拍脉冲CP

信号序列为：4种序列(节拍信号可变&全部工作脉冲)　←有多种方案



2、时序信号形成电路的组成　(时序系统的实现)

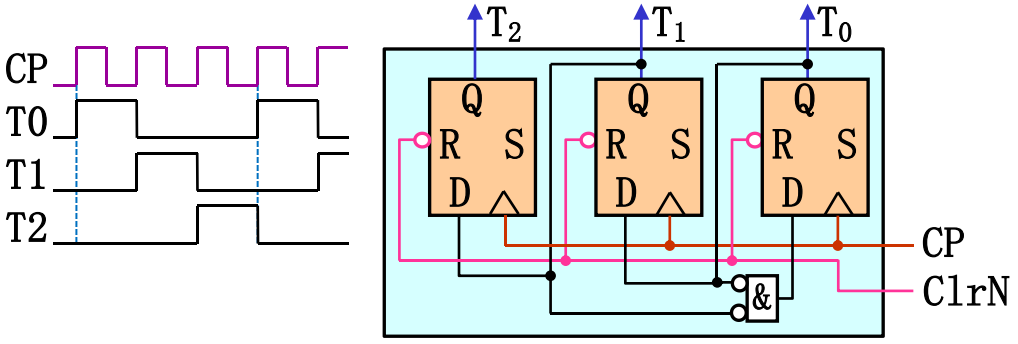
***电路组成：** 定序逻辑(时序信号次序控制)＋定时逻辑(时序信号时长控制)

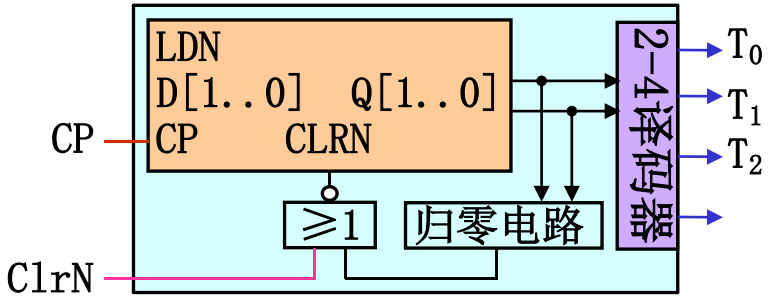


***环形信号发生器的组成：** (定序逻辑的实现)

基本方法有移位REG、计数器＋译码器　←数电讲过(不懂就看数电)

多种序列可通过改变次态函数(如引入指令类型)实现





复位时(T_0=T_1=T_2=0)—　$T_0^* = \overline{T_2}\,\overline{T_1}\,\overline{T_0}$,

轮转时(T_0=T_1=0, T_2=1)—$T_0^* = T_2\overline{T_1}\,\overline{T_0}$,

思考： 若左图产生序列T_0+T_1+T_2及T_0+T_2，选择信号为S(=1为前者)，则如何实现？

*μOP的定时方式：（定时逻辑的实现）

思考：CP宽度与μOP时长的关系？μOP序列中μOP的定时方法？ 相等，每个μOP与节拍绑定

即节拍脉冲CP的时长控制方式(受μOP类型&状态影响)

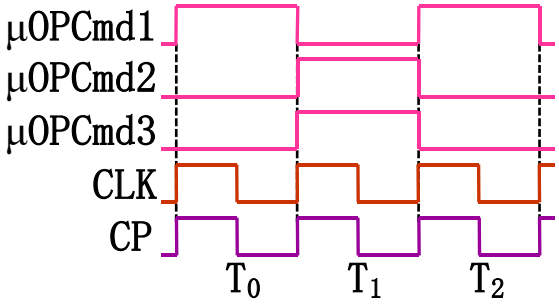
同步控制方式—

各μOP时长只受统一的基准时钟信号(主时钟信号)控制

定时原理：CP同步于CLK，即节拍周期$T_{CP}=1T_C$；

各μOPCmd同步于节拍信号 ←节拍信号基于CP

示例：



μOPCmd形成逻辑：

μOPCmd1＝T0+T2，

μOPCmd2＝T1

定时逻辑：CP＝CLK



特点：控制简单，适合CPU内部的μOP定时(时延固定/相差较小)

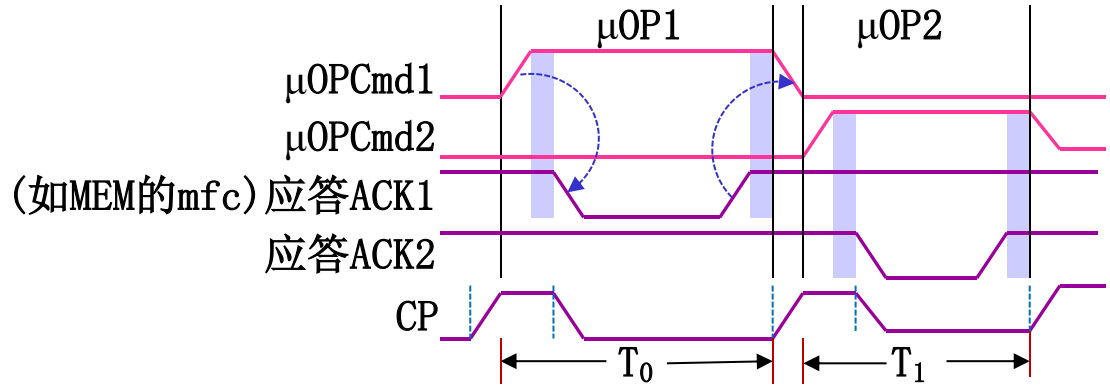
异步控制方式—（又称应答方式或握手方式）

各μOP的时长只受专门的联络信号(应答信号)控制

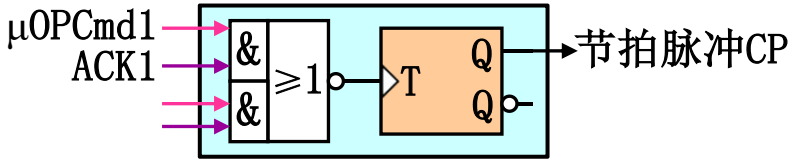
└←发出μOPCmd后，收到应答信号后本μOP完成

定时原理： CP同步于应答信号，$T_{CP} = T_{收到应答} - T_{发出命令}$；

各μOPCmd同步于节拍信号 ←节拍信号基于CP产生



定时逻辑： CP在$\mu OPCmd_i \bullet ACK_i$下降沿时改变



特点： 控制复杂，适合CPU与外部的μOP定时(时延可变/相差较大)

联合控制方式—　（又称半同步方式）

各μOP的时长受基准时钟信号及联络信号共同控制

定时原理： 基础为同步控制方式，支持异步控制方式

即 $T_{CP}=kT_C$，k为常量/变量[同步/异步方式时]

定时逻辑： CP同步于时钟信号、当前方式相关信号，

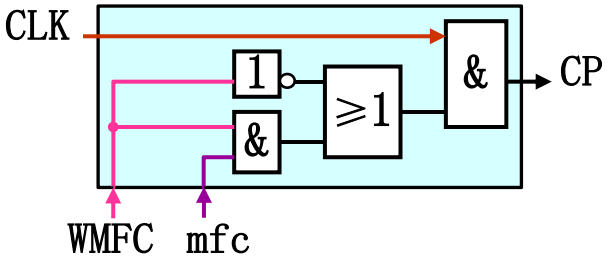
←用μOPCmd表示(如C)

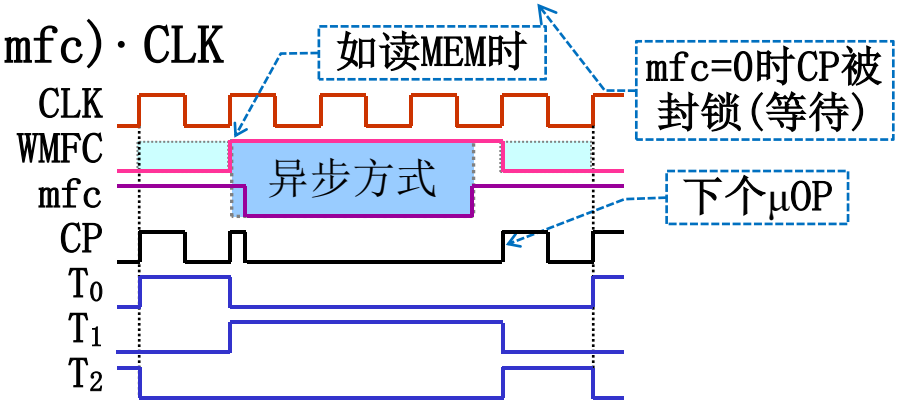
即$CP=f(C, CLK, ACK_i)$　　←不同μOP对应ACK可不同

示例： 当前方式用WMFC指明(0为同步)，MEM联络信号为mfc(0为未完成)，

即 WMFC＝0时CP＝CLK ， WMFC＝1时CP＝mfc · CLK；

则 $CP=(\overline{WMFC}+WMFC \cdot mfc) \cdot CLK$





应用： 访问异步MEM

特点： 控制较简单，适合CPU的所有μOP定时

三、uOP控制信号的形成 --μOP控制信号形成电路的组织

*电路功能： 产生工作流程中当前节拍所需的μOP控制信号

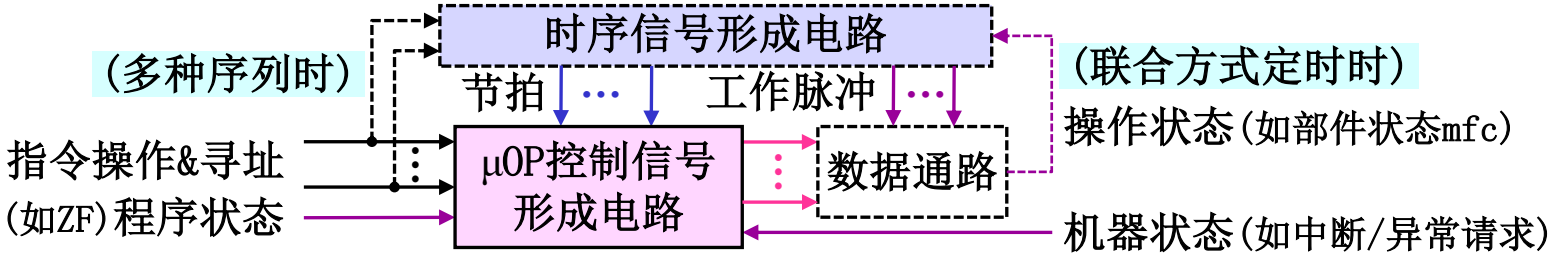
└→按序产生←┘←时序电路自动改变

*I/O信号：

输入— 节拍信号，指令操作&寻址方式、程序状态、机器状态

└←表示当前节拍 └←表示指令需求类型←┘ └←表示请求类型

输出— 所有的μOP控制信号(即状态转换图中所有的μOPCmd)



*内部逻辑： 状态转换图中每个μOP控制信号的逻辑函数；

有2种实现方法 (细节稍后讨论)

硬布线CU—组合逻辑，实质是编码器[输出=f(in)]

微程序CU—存储逻辑，实质是微主机[输出=微指令执行结果]

※控制器的组成小结：

功能—循环&有序地产生工作流程所需的μOP控制信号

（源自状态转换图）

控制器的组成—CU＝ID＋时序信号形成电路＋控制信号形成电路

ID：表示当前轮需求类型

时序信号形成电路：产生当前轮所有节拍

控制信号形成电路：产生当前节拍信号

时序信号形成电路的组成—

时序系统组织：信号的个数/功能/定时，信号序列组成

基于状态转换图

满足μOP时延

时序系统实现：定时逻辑(CLK→CP)＋定序逻辑(CP→信号序列)

μOP控制信号形成电路的组成—

I/O信号：输入为节拍、ID等信号，输出为所有μOP控制信号

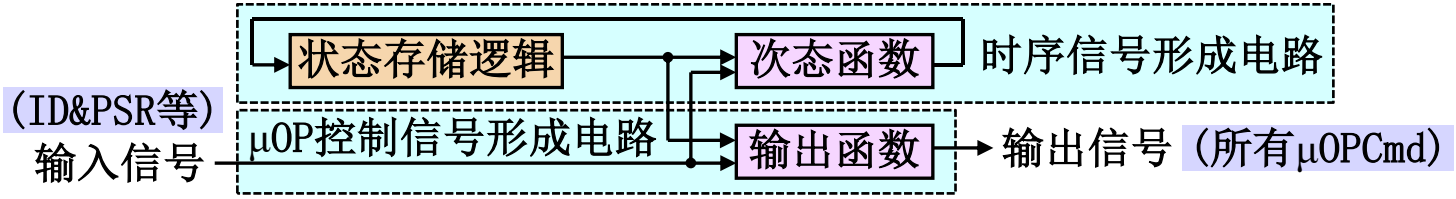
内部逻辑：状态转换图的逻辑函数；

组织方式有组合逻辑、存储逻辑

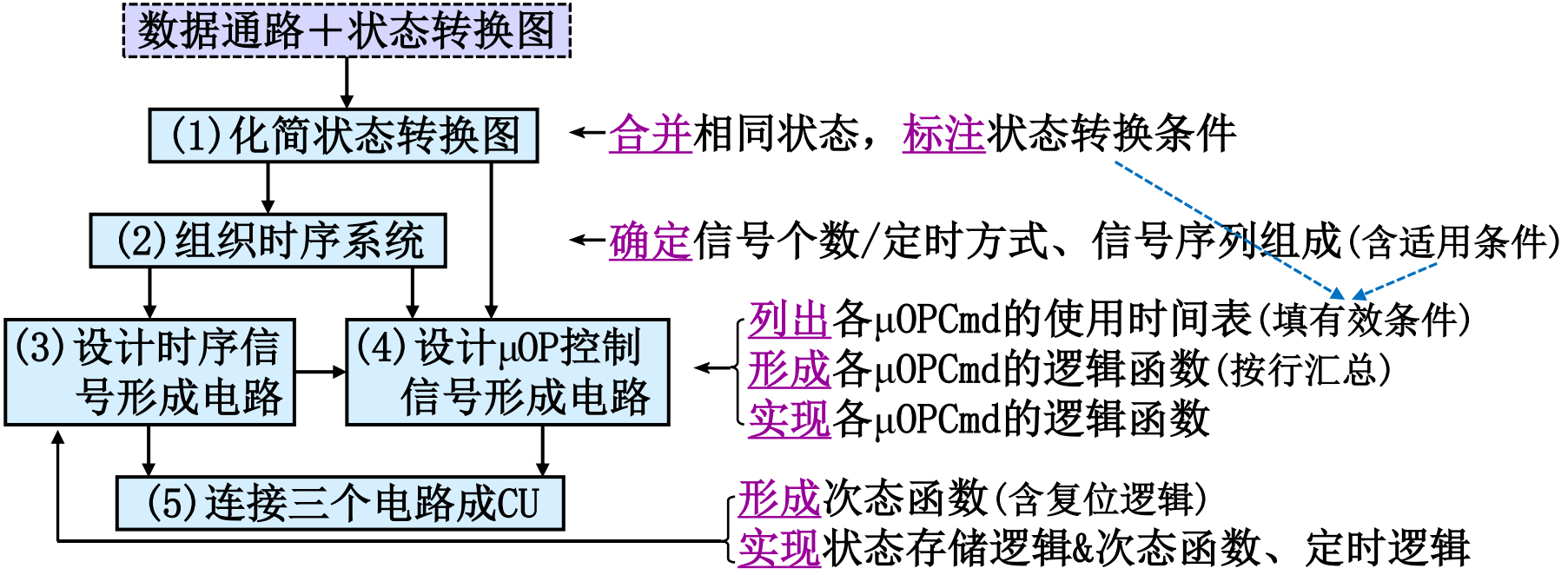
§5.4 硬布线控制器的设计

※主要内容：实现方法，设计步骤，CU的设计过程(△)

*硬布线CU的实现：基于有限状态机模型

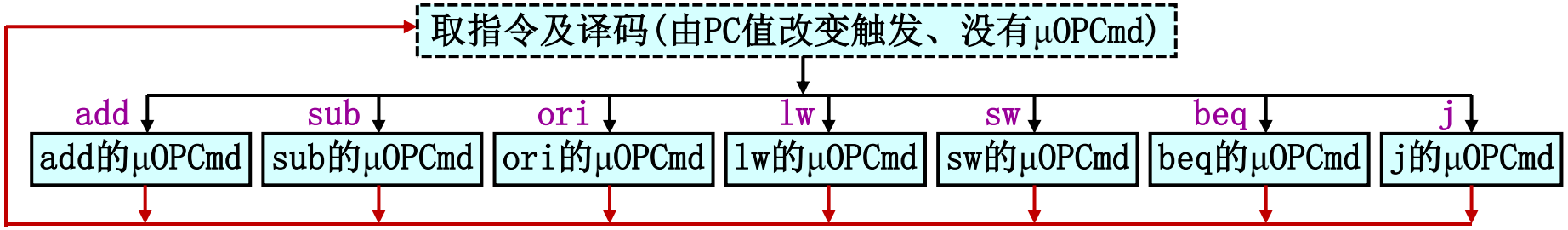


一、CU的设计步骤



二、单周期CU的设计　　（不考，掌握过程）

*设计背景：支持7条MIPS指令的数据通路＋状态转换图



*化简状态转换图：每条指令仅1个状态，无重复状态；

状态转换条件为操作码

*组织时序系统：仅2个工作脉冲信号(0个节拍信号)，同步方式定时；

└←理论上有1个，但一直有效

工作脉冲信号序列仅1种(轮流有效)、适于所有指令

*设计时序信号形成电路：

次态函数— P0＝CP·ClrN、P1＝$\overline{CP}$·ClrN　←ClrN为复位信号(低电平有效)

电路实现— 信号存储逻辑，次态函数，定时逻辑

(无需触发器)　(如上)　(CP＝CLK)

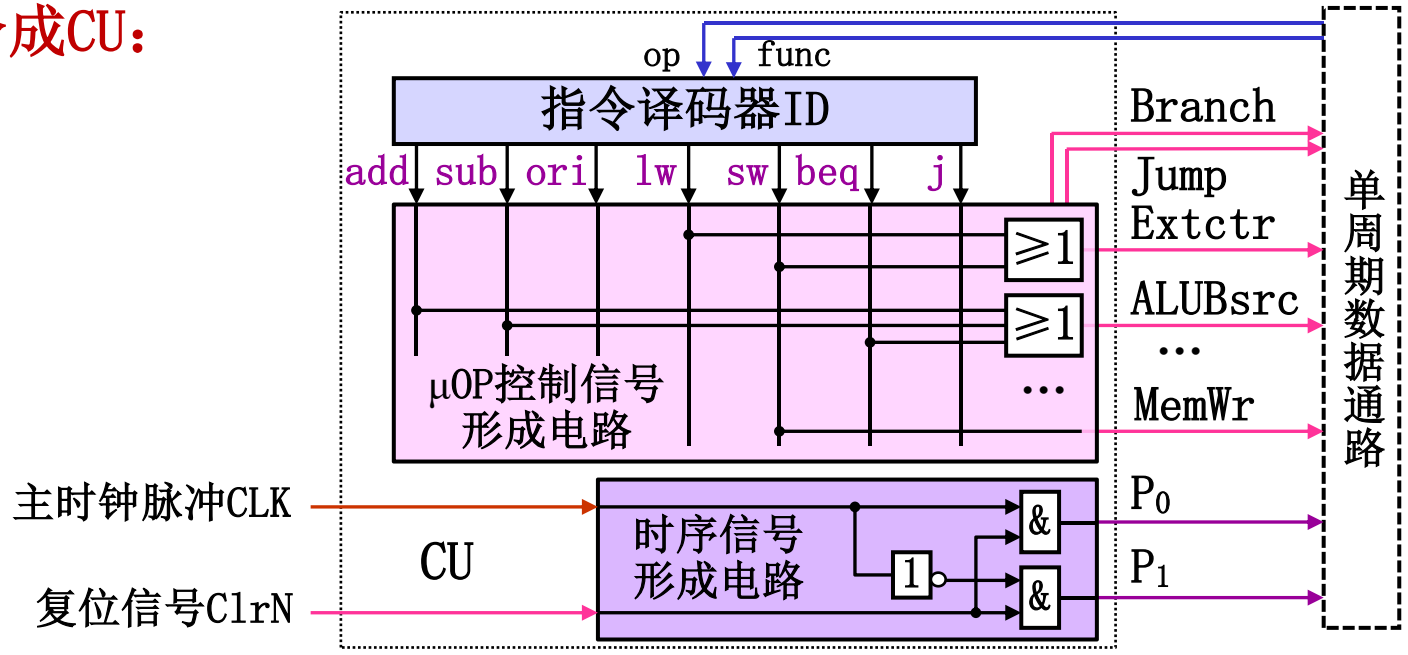
*设计μOP控制信号形成电路：

(1)μOPCmd使用时间表只有1列，无时间戳(时序系统无节拍信号)

Extctr	lw＋sw	RegAsrc	add＋sub
ALUBsrc	add＋sub＋beq	RegDsrc	add＋sub＋ori
ALUctr	[1]=ori＋beq,[0]=sub＋beq	RegWr	add＋sub＋ori＋lw
MemRd	lw	MemWr	sw

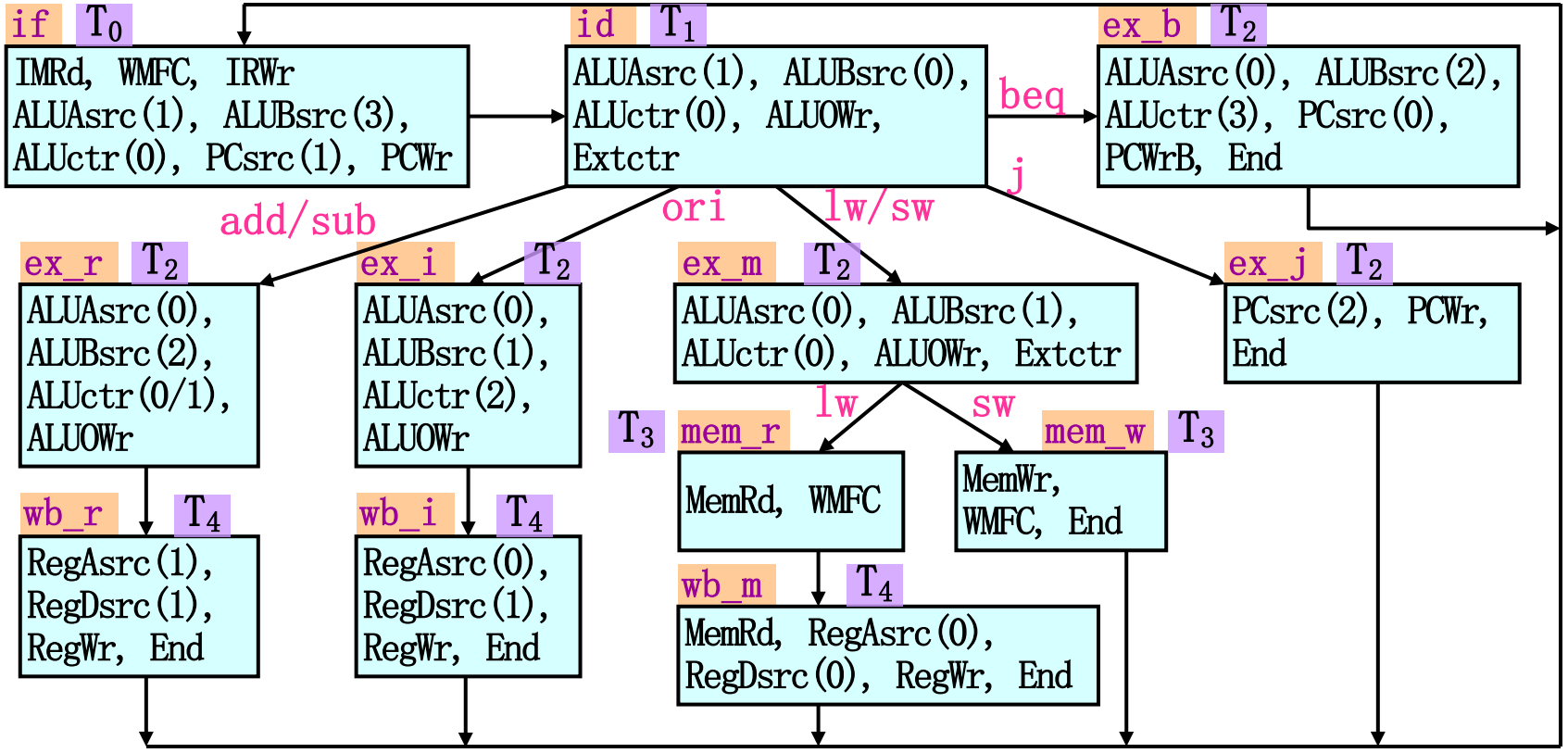
(2)(3)各μOPCmd的有效逻辑如上表，用门电路实现

*整合成CU：



三、多周期CU的设计 （不考，掌握过程）

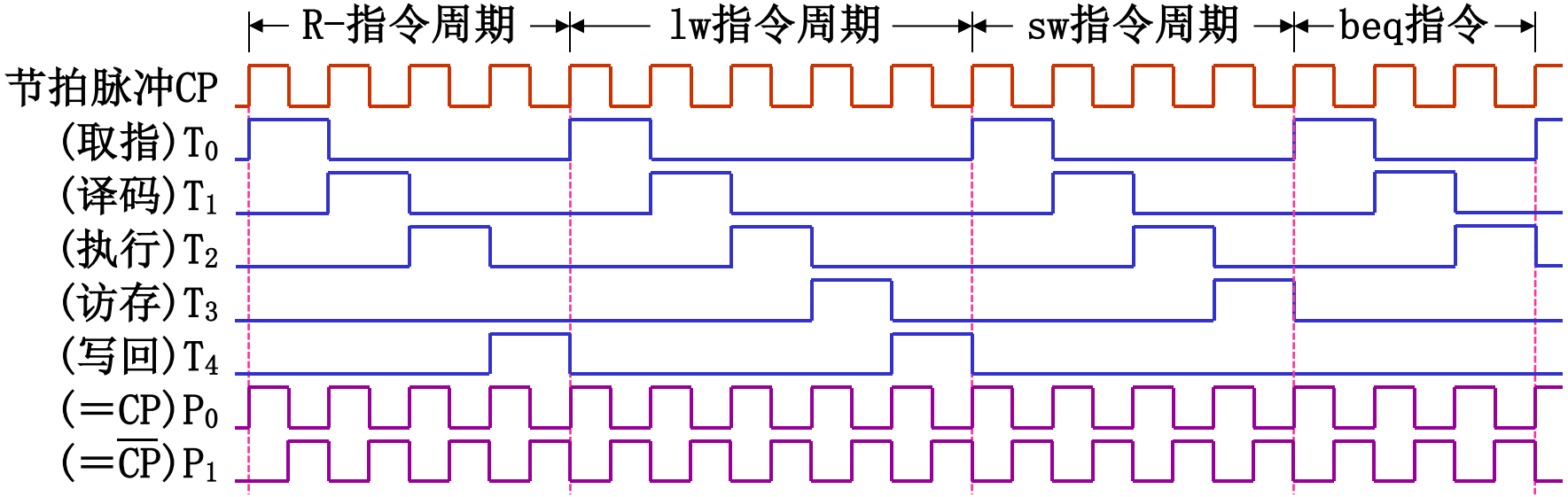
*设计背景：支持7条MIPS指令的数据通路＋状态转换图



*化简状态转换图：无重复状态，

状态转换条件为操作码或次序(≤5个)

*组织时序系统： 5个节拍(表示操作功能)、2个工作脉冲，μOP定时采用联合控制方式；4种时序信号序列(适用条件如下图)



*设计时序信号形成电路：

(1)确定次态产生函数：(T_i*表示T_i的次态)

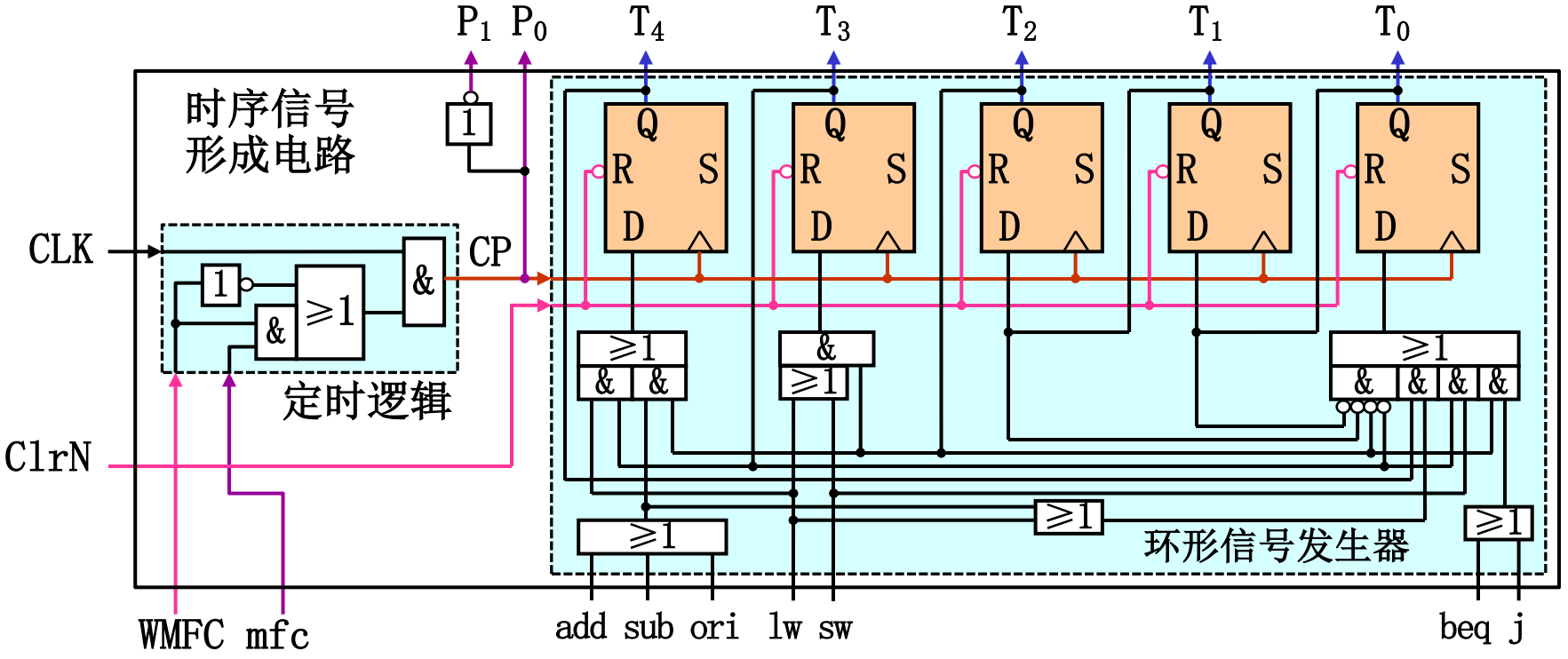
$T_1^*=T_0$，$T_2^*=T_1$，$T_3^*=(lw+sw)\cdot T_2$，$T_4^*=(add+sub+ori)\cdot T_2+lw\cdot T_3$，

$T_0^*=\underbrace{(add+sub+ori+lw)\cdot T_4+sw\cdot T_3+(beq+j)\cdot T_2}_{\text{循环逻辑}}+\underbrace{\overline{T_0}\cdot\overline{T_1}\cdot\overline{T_2}\cdot\overline{T_3}}_{\text{复位逻辑}}$

(2)电路实现：状态存储逻辑(5个触发器)，

次态函数(5个组合逻辑电路[含复位])，

定时逻辑(同步/异步分类定时)



*设计μOP控制信号形成电路：

(1)状态转换图打时间戳，μOPCmd使用时间表中填有效条件

	T_0	T_1	T_2	T_3	T_4
PCsrc	All(1)		beq(0)+j(2)		
PCWr	All		j		
PCWrB			beq		
IMRd	All				
IRWr	All				
RegAsrc					add|sub(1)+ori|lw(0)
RegDsrc					add|sub|ori(1)+lw(0)
RegWr					add|sub|ori|lw
Extctr		All	lw|sw		
ALUAsrc	All(1)	All(1)	add|sub|ori|lw|sw|beq(0)		
ALUBsrc	All(3)	All(0)	add|sub|beq(2)+…		
ALUctr	All(0)	All(0)	add|lw|sw(0)+sub(1)+…		
ALUOWr		All	add|sub|ori|lw|sw		
MemRd				lw	lw
MemWr				sw	
WMFC	All			lw|sw	
End			beq|j	sw	add|sub|ori|lw

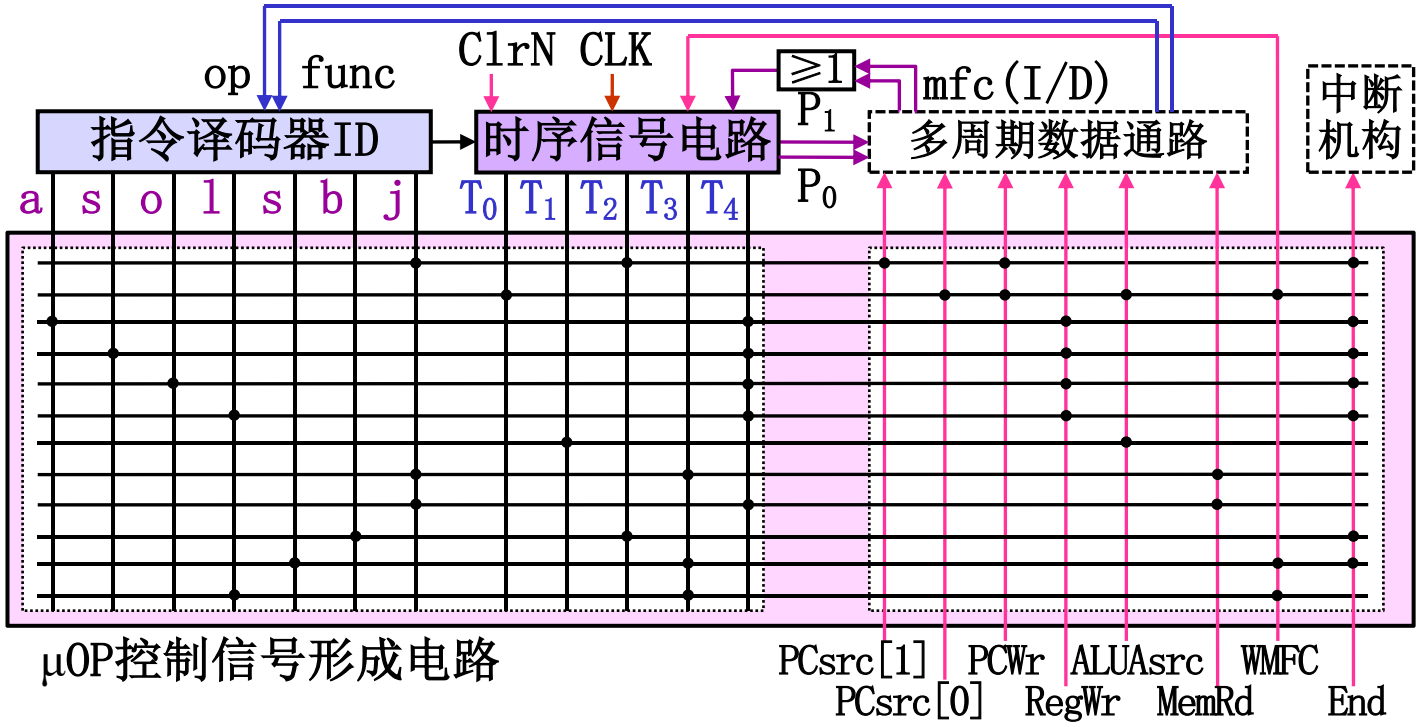
(2)按行汇总使用时间表，μOPCmd的逻辑表达式为：

$PCsrc[1]=T_2\cdot j$，$PCsrc[0]=T_0$，$PCWr=T_0+T_2\cdot j$，

$RegWr=T_4\cdot(add+sub+ori+lw)$，…，

$End=T_2\cdot(beq+j)+T_3\cdot sw+T_4\cdot(add+sub+ori+lw)$

(3)用组合逻辑电路实现每个μOPCmd



作业5-3：P237— 17(仅画出信号序列)、19(仅填μOPCmd使用时间表)

§5.5 微程序控制器的设计

※主要内容：微程序控制思想，CU的组成、工作原理

1、微程序CU基本思想

一条微指令相当于μOPCmd序列的一步

CS—Control Storage

*微程序控制思想：　—存储逻辑(CU相当于微主机)

①每条指令的μOPCmd序列用微程序表示，存放在专用MEM中

└←微指令序列　└←称为控制MEM(CS)

②CU自动、逐条取出并执行微指令，有序产生μOP控制信号

简化CU的实现

└←用一定格式表示、可同时执行的一组微命令

微指令格式：操作控制字段 | 顺序控制字段 (定长指令字结构)

*微程序类型：

多个公用微程序(如取指/中断)＋

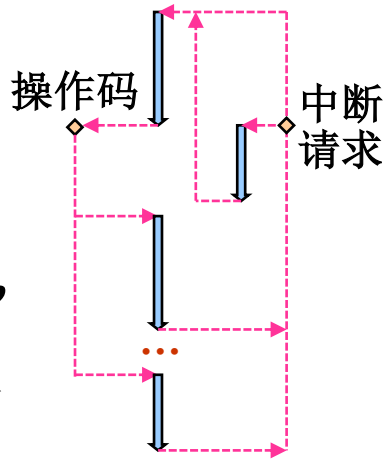
各指令功能微程序

*CPU工作流程的表示：

由多个微程序(公用+某指令)组成，

微程序的末条微指令为转移型

(判断操作码/中断请求)



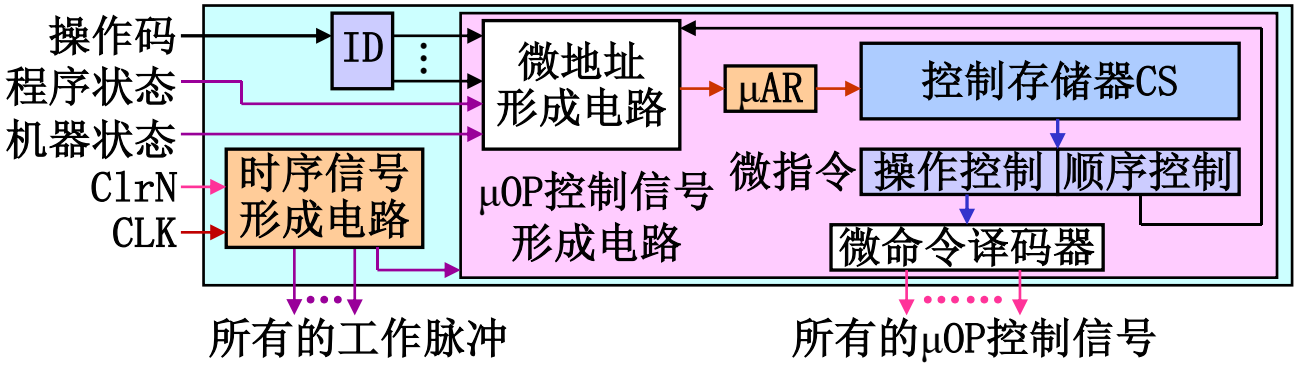
控制存储器CS

M+0	微命令组	M+1	取指令微程序
M+1	微命令组	M+2	
M+2	微命令组	aaa	
N+0	微命令组	M	中断响应微程序
…	…		
P+0	微命令组	P+1	add指令微程序
P+1	微命令组	P+2	
P+2	微命令组	bbb	
…	…	…	
Q+0	微命令组	Q+1	beq指令微程序
Q+1	微命令组	bbb	

2、微程序CU组成及工作原理

***基本组成：**

（类似硬布线CU）



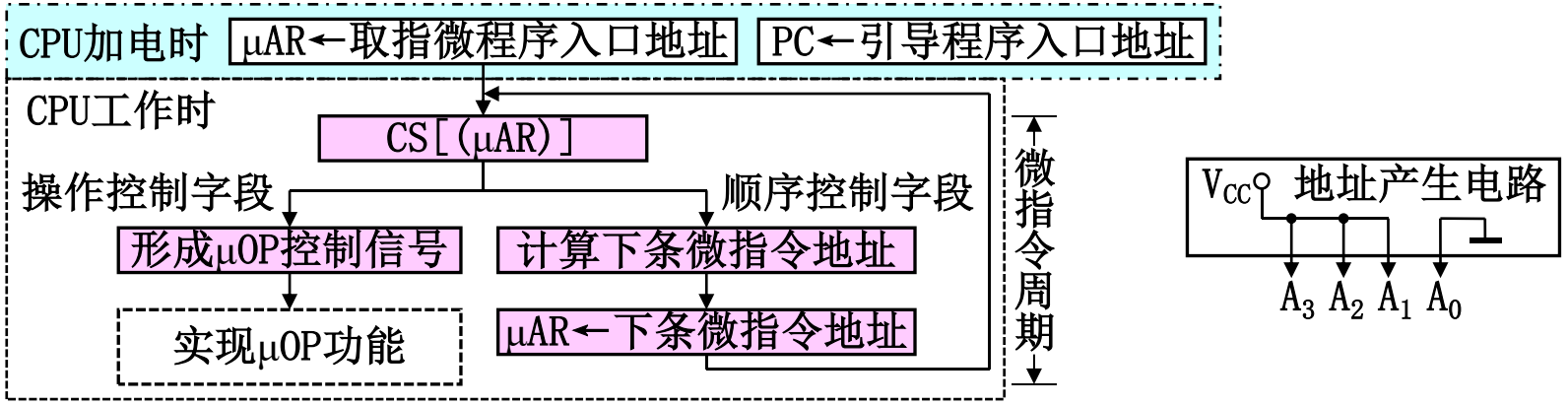
时序信号电路—1级时序，1种信号序列（含复位） ←硬布线为2级时序

└←工作脉冲　　└←信号个数＝数据通路所需+1

μOP控制信号电路—微指令执行部件＋CS ←硬布线为组合逻辑

└←μAR、微命令译码器、微地址形成电路

***微程序CU工作原理：** 循环地取出并执行微指令 ←微指令周期≈节拍周期



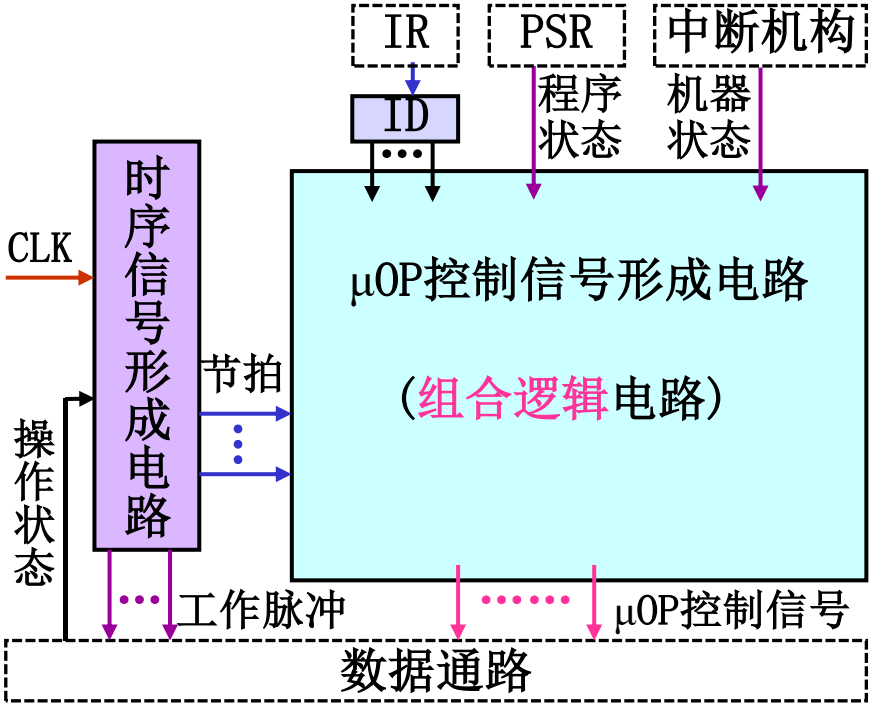
3、CU组成的对比

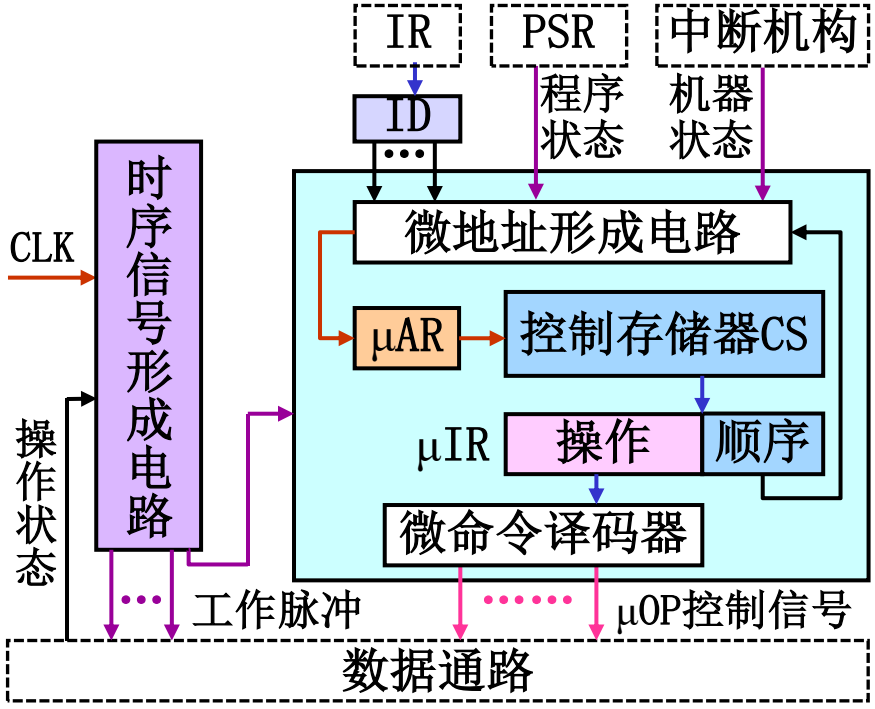
指令译码器—输入为OP码&寻址方式位，输出为操作类型

时序信号形成电路— 2级时序(硬布线)， 1级时序(微程序)

注：时序信号中，工作脉冲用于数据通路

μOP控制信号形成电路— 组合逻辑电路， 微主机





§5.6 异常及中断的处理

※主要内容：异常与中断概念，事件的处理过程，中断机构组成(△)

一、异常及中断的基本概念

*事件(Event)： 指改变程序正常执行顺序的特殊情况

(指令约定的顺序)

└→CPU必须处理

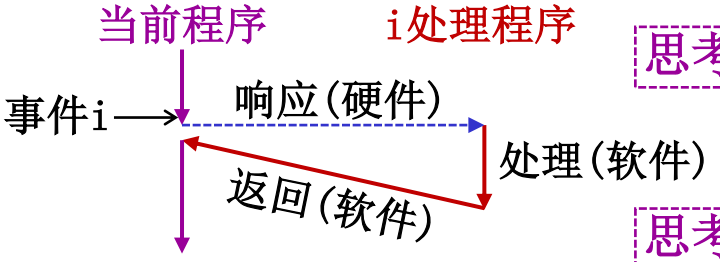
类型— 异常(Exception)、中断(Interrupt) ←发生在CPU内部和外部

*事件的处理方法： 暂停现行程序的执行，转去执行事件处理程序，

└→称为事件响应 └←称为事件处理

返回被暂停的程序继续执行

└→称为事件返回



思考①：响应为何由硬件实现、返回由软件实现？

响应基于检测，检测由硬件实现

处理程序各不相同，硬件不知何时结束

思考②：程序想继续执行，哪些信息不能被破坏？

PC/PSR/GPRs

1、异常　　--又称内部异常(程序性异常)

由CPU内部执行指令所引起的意外事件(如除零、断点)

*异常的分类：　按请求类型(报告方式)及返回方式划分

异常分类	故障(Fault)	陷阱(Trap)	终止(Abort)
	可能修复①的异常	预先安排②的异常	不可修复的异常
	缺页，除零、溢出③	单步，系统调用、断点	无效表、硬件故障
返回方式	当前指令 或 终止程序 (可修复时) (无法修复时)	下条指令	终止程序 或 关机 (能/否定位到程序)
处理时机	立即(执行过程中)	立即(执行结束时)	立即(执行过程中)

说明：①Falut由程序本身捕获，Abort只能由系统捕获
②有两种方式：满足条件触发(如判PSR标志)、执行特殊指令触发(如系统调用)
③有些ISA将溢出作为可选择的异常(如用陷阱指令INTO[非运算时自动]检测)

*返回方式：　即断点值，由请求类型决定

*处理时机：　立即

```
如：add  R1,R2,R3    (不检测OF)
    INTO             (检测OF)
    …
```

2、中断　　--又称外部中断

由外设产生的请求事件，又称为异步事件(指与指令执行无关)

*中断的分类/返回方式/处理时机：　INR—Interrupt Request　NMI—Non Maskable Interrupt

中断分类(按紧急程度)	不可屏蔽中断(NMI)	可屏蔽中断(INTR)
	须立即处理的中断	可暂不处理的中断
	校验错、电源故障等	键盘、打印机中断等
处理时机	当前指令周期结束时①	当前或多个指令周期结束时
返回方式	下条指令② 或 终止程序	下条指令
注：①两条指令之间，易实现、不影响响应速度[$T_{指令}$(ns级)<<请求间隔(μs级)] ②处理时机(指令间)所致		

*可屏蔽的实现方法：　CPU支持允许中断/屏蔽中断2种状态

状态表示—PSR中设置“中断允许”标志IF(=1允许)　←Interrupt Flag

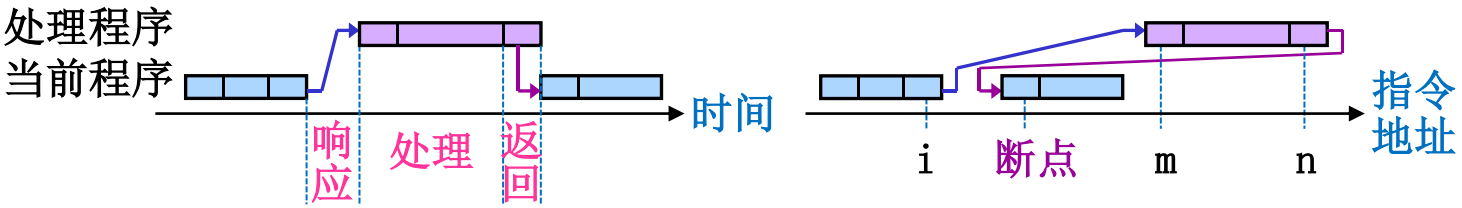
状态转换—用开中断(IF←1)、关中断(IF←0)指令实现　←指令∈ISA

※异常及中断的内涵：包括事件类型、程序控制流改变

本教材的叫法—　(异常/中断)　(中断)　→如中断机构

二、异常及中断的处理过程

*处理过程回顾： 响应(硬件)、处理＋返回(软件)



断点—事件处理后的返回地址

思考：异常、中断的断点各为什么？

*应用需求与处理策略：

应用需求— ①同时可能产生1个异常、多个中断，仅处理1个事件

有多种类型→ ←≤多个INTR、1个NMI(概率小)

②事件的紧急程度、处理时机不同(随时或指令周期结束时)

处理事件的选择— 事件具有优先级＋检测时判优

思考①：检测时判优如何实现？ 分类检测，选择时异常＞NMI＞INTR

处理时机的实现— 随时检测＋立即响应 ←满足异常需求+便于实现

思考②：中断响应时机如何实现？ 检测增加约束条件！

如NMI有效＝INTR•END，INTR有效＝INTR•IF•END (End表示指令周期将结束)

实现方法：μOPCmd序列最后一行添加End

1、异常及中断的响应

任务	目标
①保存断点及程序状态(PSW)	返回后，被暂停的程序可继续执行
②关中断	处理程序的执行过程不被打断(新事件)
③识别事件类型并转入处理程序	选择最紧急事件，获得程序入口地址，程序入口地址写入PC

思考①：程序继续执行需满足的条件？ PSR/GPRs未破坏，PC＝断点(下条/当前)

思考②：GPRs如何保存？ 处理程序保存/恢复(使用个数少)，硬件时需保存全部

*保存断点及PSW的实现： 中断机构增设后援REG保存 ←返回时写回PC/PSR

思考③：任务③需使用事件类型信息，这些信息放在哪？

事件信息的保存—中断机构增设异常原因REG保存异常信息，

CPU中实现代价大 → 中断源增设状态REG保存中断请求信息

*关中断的实现： PSR中IF←0 ←新事件不包含异常/NMI(必须打断)

思考④：任务①和②可对调吗？ 不行！IF∈PSW，对调后返回的是0(≠原来值)

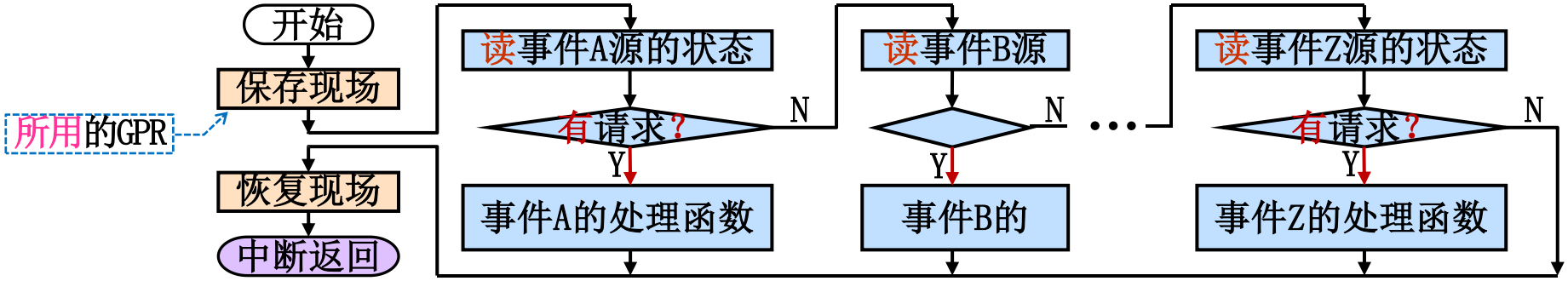
*识别事件类型并转入处理程序的实现：

*识别事件类型并转入处理程序：

识别方法—有向量方式(硬件识别)、非向量方式(软件识别)

非向量方式— (所有事件共用1个处理程序)

实现： PC←固定的入口地址 ←识别事件类型由软件(处理程序)完成



思考①：上图最高、最低优先级事件是什么？

思考②：指令的OPD放在REG/MEM中，异常/中断源的状态放在哪？

为减少保存开销，异常状态(编码形式)放在异常原因REG中，
中断源状态放在其状态REG中(无传送开销)

向量方式— (每个事件有1个处理程序)

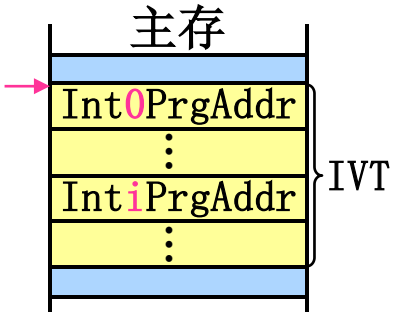
向量方式— （每个事件有1个处理程序）

处理程序入口地址管理： （软件[OS]负责）

保存在中断向量表(IVT)中，每个事件1行；

IVT放在主存中，首地址放在CPU的相应REG中

└←类似页表管理　　└←便于表项地址计算



实现： （全部由硬件完成）

基于事件在IVT中的位置

判优—按事件优先级查询事件源，形成事件类型号(i)

(分类检测)
(异常＞NMI＞INTR)

├←读异常类型REG(CPU内)　←1个/同时

├←值固定　←仅1个

└←读中断源状态REG(CPU外)　←n个/同时

查IVT—根据事件类型号访问IVT，获得处理程序入口地址

└←类似查页表

需产生外部操作！
1个操作如何从多个部件读取状态？
第7章讨论

转入处理程序—PC←所获得处理程序入口地址

(2)异常及中断的返回

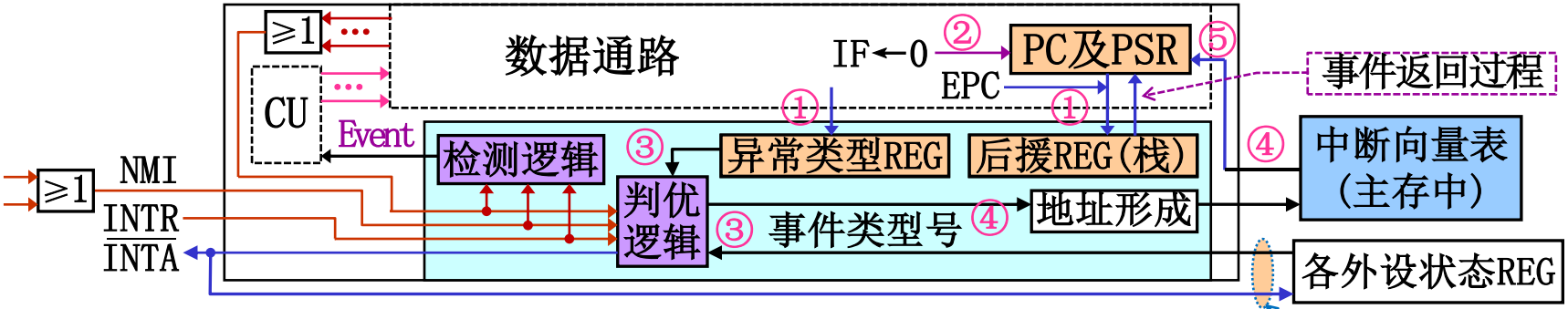
*任务：恢复断点及程序状态

*实现：使用中断返回指令，指令功能为PC及PSR←(后援REG)

3、中断机构的基本组成 （了解）

*功能：异常&中断事件的检测及响应（以向量方式为例）

*组成：请求分类接入(分类检测所需) 中断请求共用信号线(可扩展性好)



*检测逻辑：分类检测(满足优先级/时机要求)，输出3种事件(响应操作不同)

*响应过程：（CU根据Event产生相应μOPCmd序列） ←不会被打断

①保存断点及PSW—保存到后援REG中，异常类型保存到异常类型REG中

②关中断—PSR中IF←0(避免处理过程被打断)

中断请求在外设状态REG中

③判优—按优先级查找相应REG，形成事件类型号(并撤销所选请求)

④查IVT—根据事件类型号访问主存中IVT，获得处理程序入口地址

⑤转入处理程序—PC←处理程序入口地址

*返回过程：恢复断点及PSW

三、支持异常处理的CPU设计　　（不考，了解过程）

*设计需求：基于支持7条MIPS指令的数据通路＋CU，
支持溢出、非法操作码异常的处理(返回下条、当前指令)，
异常识别采用非向量方式(入口地址为8000 0180H)

*异常检测的设计：

检测时间—OF在T_2拍检测，ErrCode(非法操作码信号线)在T_1拍检测

有效逻辑—＝(add+sub)·T_2·OF+T_1·ErrCode

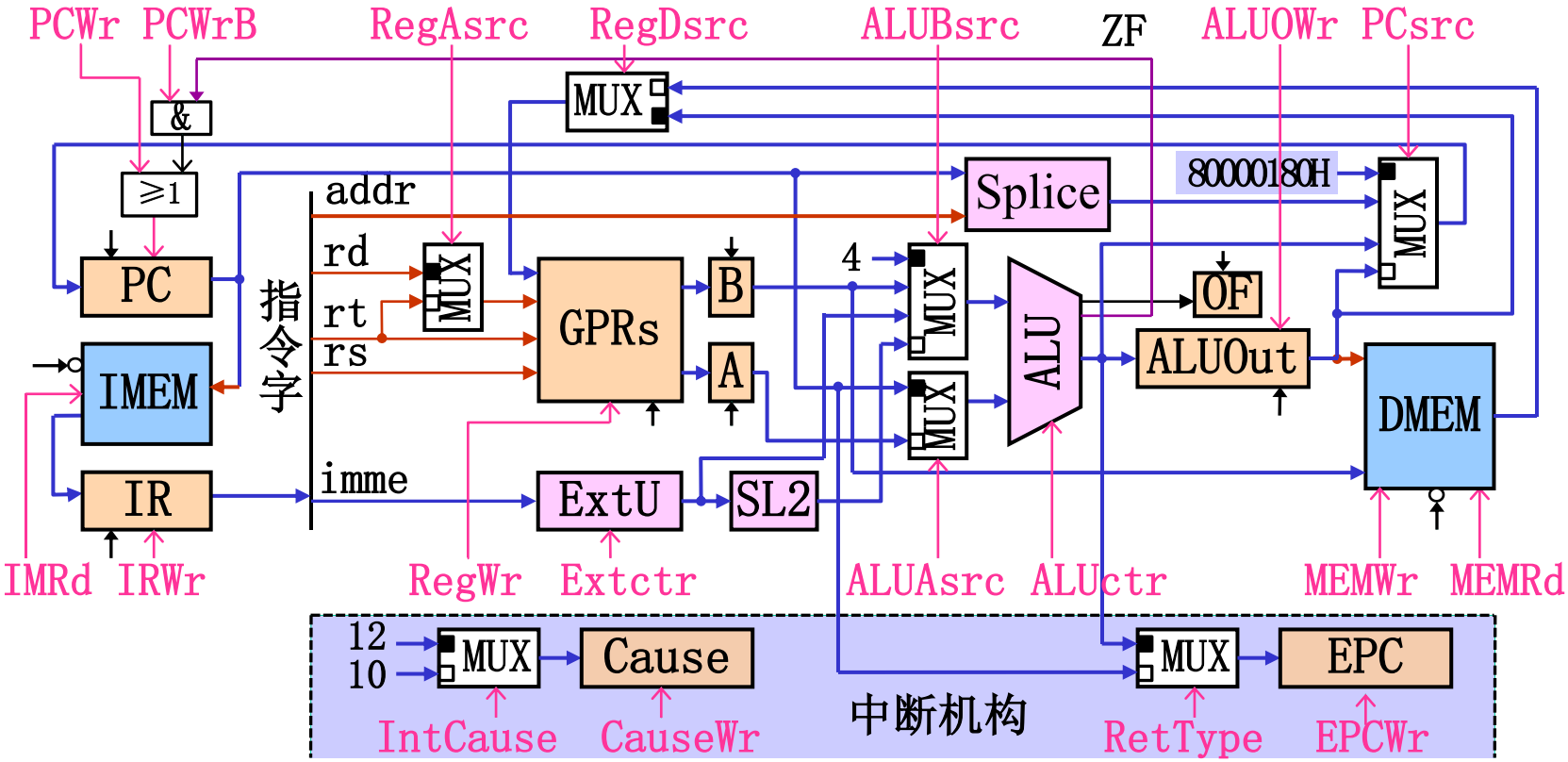
*异常响应的设计：

所需部件—用EPC保存断点，用Cause保存异常类型(12或10)
└←暂不讨论PSW的保存　　└←1个请求/同时

响应时间—检测到异常的下个节拍　　←下拍产生μOPCmd

响应过程—EPC←(PC)－4、Cause←12或10、PC←8000 0180H

*数据通路的组织：增设EPC、Cause及其路径(输出路径略)

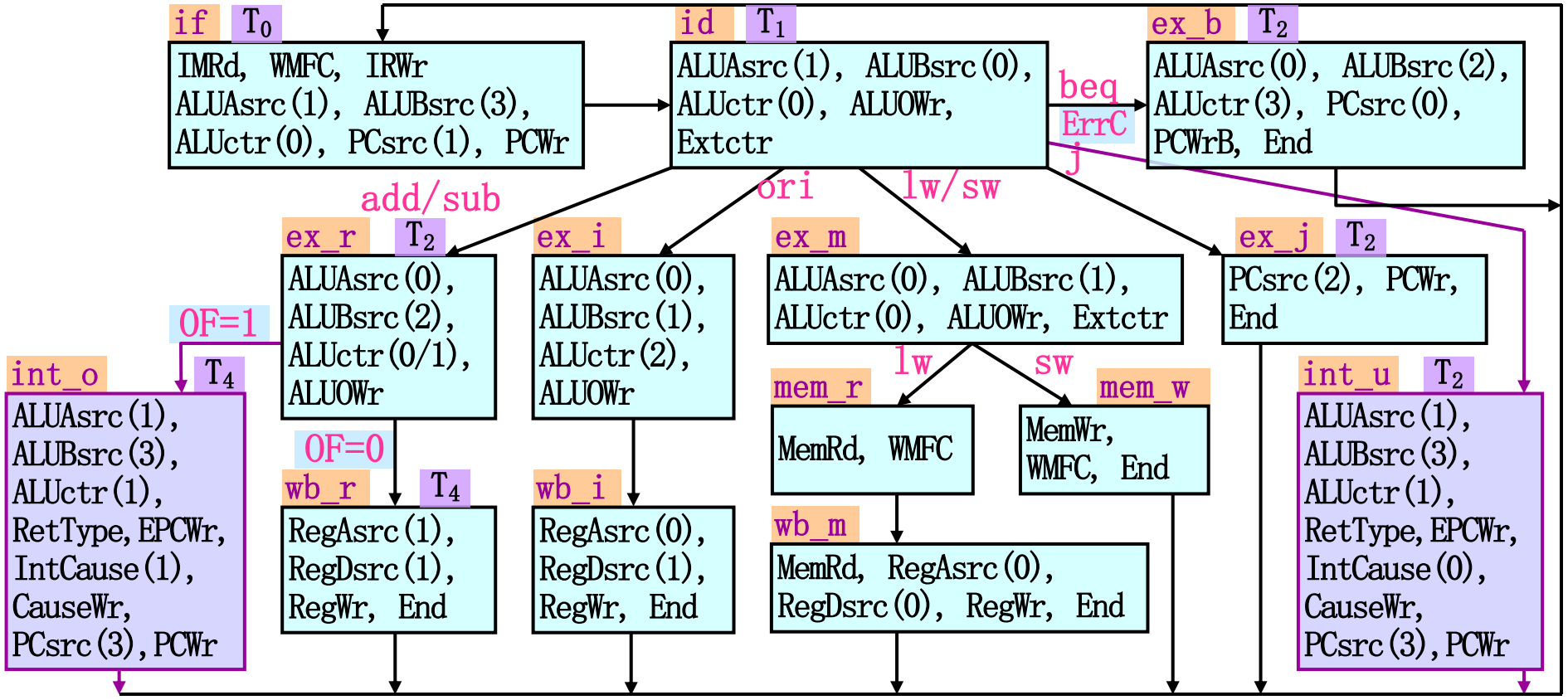


响应过程μOPCmd序列— 2种异常略有差别，均为一个节拍

ALUAsrc＝1、ALUBsrc＝3、ALUctr＝1、RetType＝0或1、

EPCWr，IntCause＝1或0、CauseWr，PCsrc＝3、PCWr

CPU工作流程的状态转换图— 增加异常响应状态



*CU的组织： 时序信号形成电路— 不变

μOP控制信号形成电路— 增加异常处理逻辑

作业5-4：P238— 25

§5.7 指令流水线技术

※主要内容：流水线的工作原理、基本组成、性能，冒险处理

一、指令流水线概述

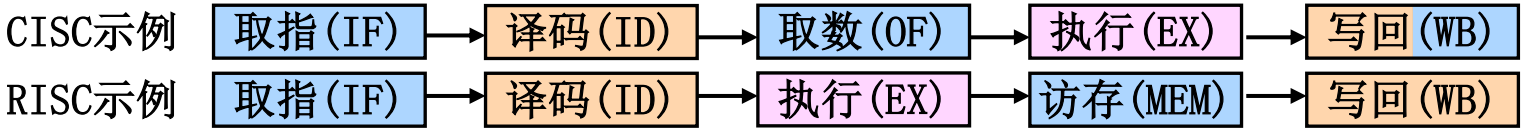
1、指令流水线的工作原理

*基本思想：指令执行过程分为多个阶段，　←基础(分割)

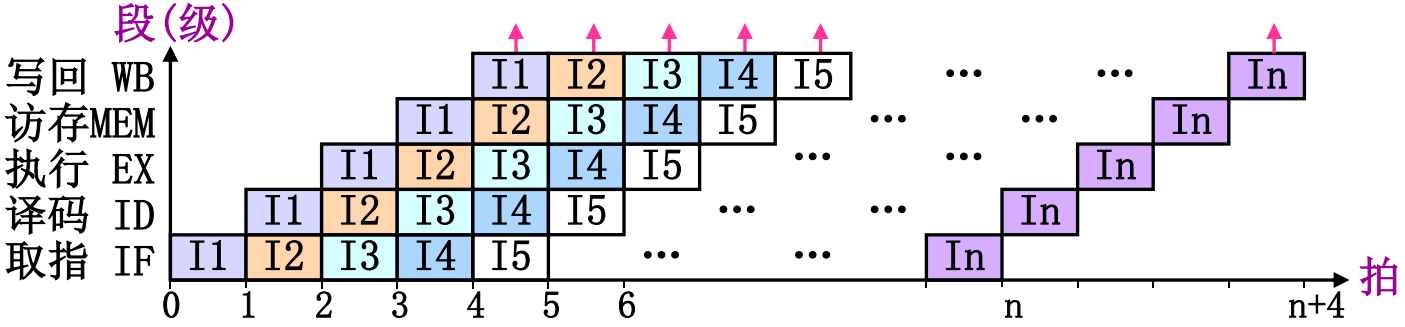
各个阶段可同时处理不同指令的操作　←效果(重叠)

多周期CPU为分时处理同一指令

*基本组成：所有功能段按序组成



*工作原理：每条指令按序通过各个段，不同指令执行的过程重叠



程序执行时间— $T_{串行}=n\times(m\Delta t)$，　$T_{流水}=m\Delta t+(n-1)\Delta t$

2、指令流水线的基本组成

(1)流水线组成基本要求

指来自其他段

*要求1：各流水段的操作相互独立 ←可以重叠所需

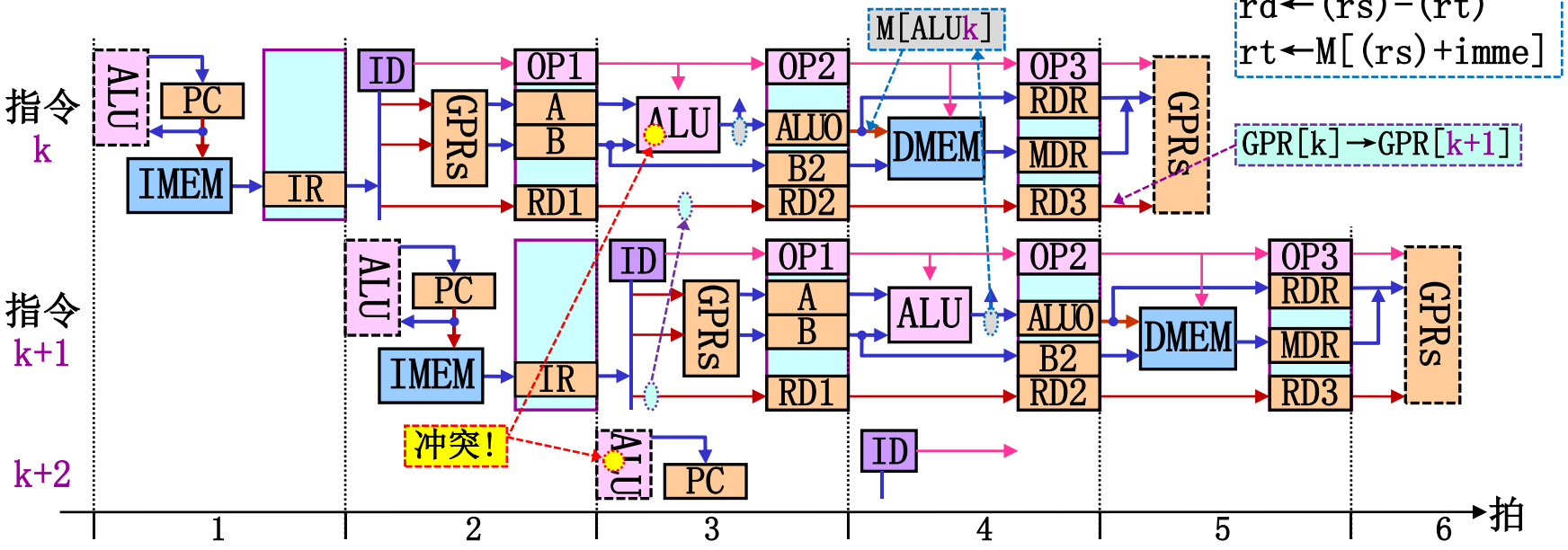
目标—①部件不能复用 ②操作的数据放在时序部件中(结果也如此)

实现—各个段的部件仅本段使用，各个段增设段间寄存器(末段除外)

思考：哪些信息需保存到段间REG？在哪些段保存？

数据/地址/命令，
产生→使用前的所有段

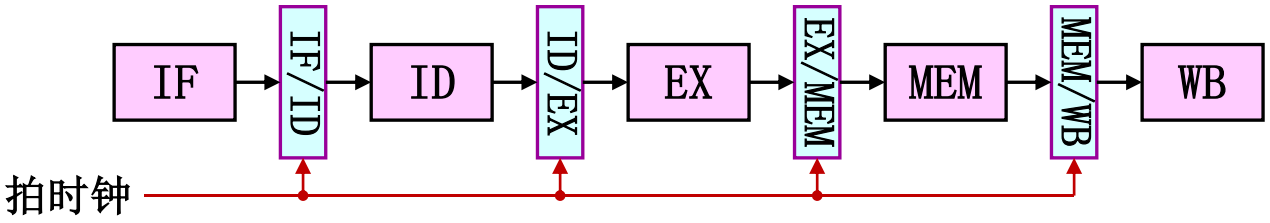
示例—MIPS32多周期数据通路的重叠现状(省略ExtU)



*要求2：各流水段的操作同步　　←连续重叠所需

实现—设置公共时钟，同时写段间寄存器(即写结果同步)

└←拍时钟周期＝max{各段延迟}



*要求3：各流水段的操作无冲突　　←保证结果正确

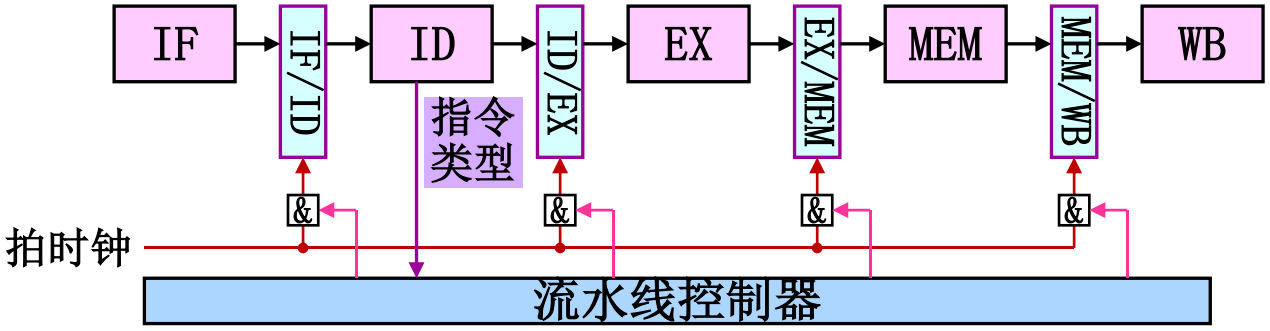
冒险(冲突)—指流水线因某些原因无法正确执行后续指令的现象

冒险类型—结构冒险、数据冒险、控制冒险

(如部件复用)　(源-目OPD相关)　(分支指令)

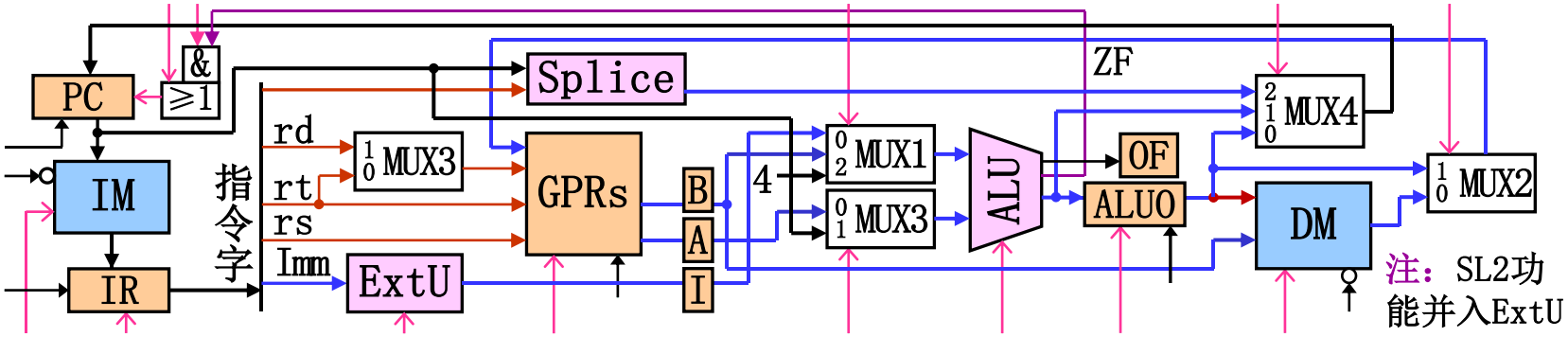
I1: R2←(R1)+5
I2:(R2)=6时PC←I1
I3: R3←(R1)+5

实现—增设部件及控制机制，解决冒险(如阻塞后续指令)



(2)流水线组成示例　(暂不考虑数据/控制冒险)

示例—支持7条指令的MPIS32多周期CPU，改为流水线(支持6条)



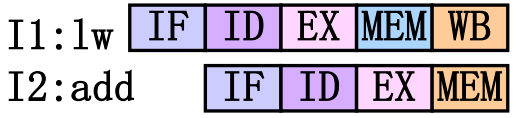
	add/sub	ori	lw	sw	beq
IF	IR←M[(PC)]，PC←(PC)+4				
ID	A←R[rs]，B←R[rt]，I←Ext(Imm)				
	/				ALUO←(PC)+(I)
EX	ALUO←(A)OP(B)或(I)				(A)-(B)＝0时PC←(ALUO)
MEM	R[rd]←(ALUO)		MDR←M[(ALUO)]	M[(ALUO)]←(B)	
WB			R[rt]←(MDR)		

***组成要求1的现状分析：**　(部件不复用,操作的数据在REG中)

①部件复用：(PC)+4、A=B?复用ALU，PC写2次

部件跨段使用：PC+Ext(Imm)，add/lw的写GPR

②操作数据在REG中：设置段间REG可解决



*指令周期调整：（部件复用问题由数据通路负责）

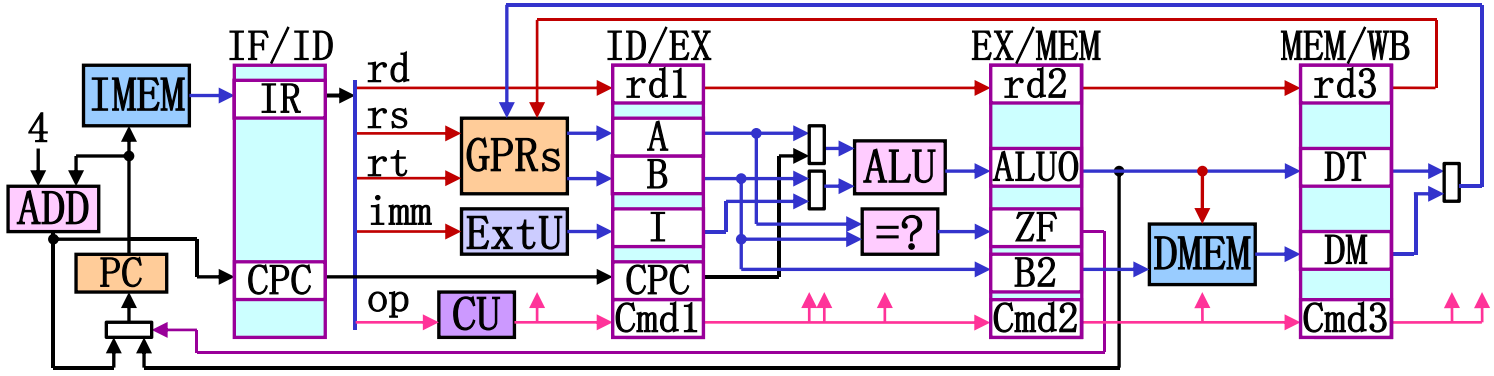
①相对寻址移至EX段(使用PC+4结果)，写GPR移至WB段

②写PC统一放在IF段(前驱指令为beq&转移时写目标、否则写PC+4)

③beq指令周期为4个T_C(写PC的数据须来自EX段的REG)

	add/sub	ori	lw	sw	beq
IF	IR←M[(PC)]，CPC←(PC)+4，PC←(EX段的ZF=1)？EX段的ALUO ：(PC)+4				
ID	A←R[rs1]，B←R[rs2]，I←Ext(Imm)				
EX	ALUO←(A)OP(B)或(I)				ALUO←CPC+(I) ZF←(A)-(B)
MEM	/	/	MDR←M[(ALUO)]	M[(ALUO)]←(B)	/ (写PC放在IF段)
WB	R[rd]←(ALUO)		R[rt]←(MDR)		

*数据通路修改：增设加法器/比较器/段间REG，及相关路径



思考：流水线常基于单周期CPU组成，有哪些不同？

无部件复用问题，
需确定T_C、组织指令周期

3、流水线的性能

假设—流水线有m个段、拍长为Δt，共执行n条指令

*吞吐率：单位时间内完成/输出的指令条数或结果数量

$$T_P = n/T_{流水} = n/[m\Delta t+(n-1)\Delta t]$$

最大吞吐率—当n>>m时，$T_{Pmax} = 1/\Delta t$，即拍时钟频率

*加速比：流水方式相对于串行方式的速度比

$$S_P = T_{串行}/T_{流水} = (n\times m\Delta t)/[m\Delta t+(n-1)\Delta t]$$

$$= nm/(m+n-1)$$

最大加速比—当n>>m时，$S_{Pmax} = m$，即流水线段数

*效率：部件使用时间与整个执行时间的比值(平均值)

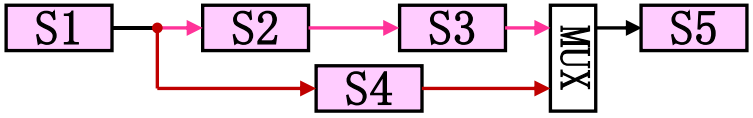
$$E = (\Sigma E_{段i})/m = \{m\times[n\Delta t/(m+n-1)\Delta t]\}/m = n/(m+n-1)$$

$$= S_P/m = T_P\times\Delta t$$

最高效率—当n>>m时，$E_{max} = 1$

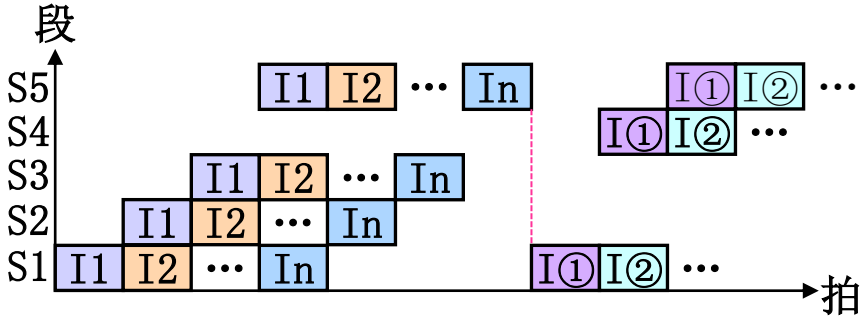
4、流水线的分类 （即流水线的属性）

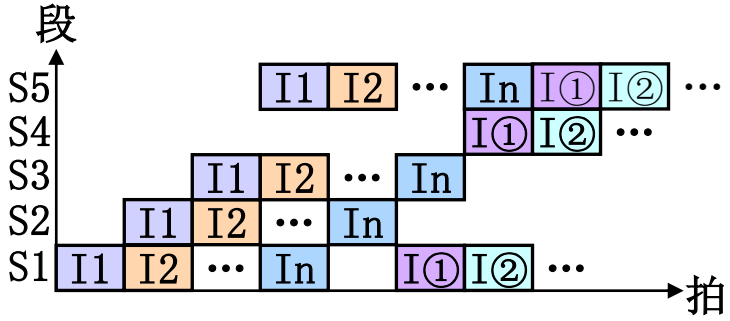
*按功能分类：单功能流水线、多功能流水线



动态→多功能
单功能→静态

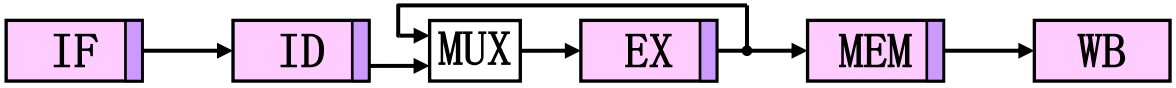
*按工作方式分类：静态流水线、动态流水线





*按结构分类：线性流水线、非线性流水线（复用流水段）

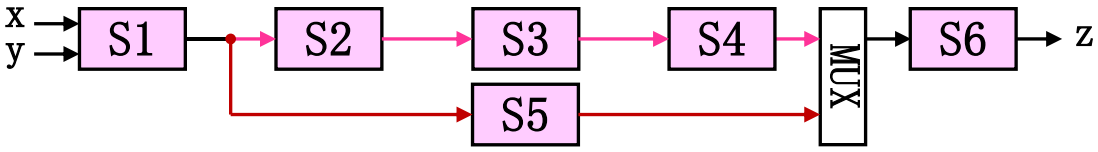
└←无法每拍流入指令



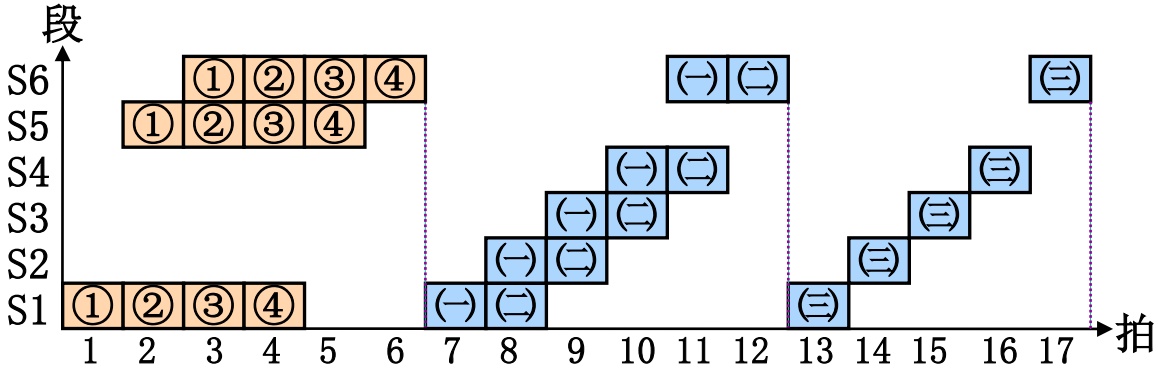
*按流入/流出次序分类：顺序流水线、乱序流水线

示例—某流水线为线性、动态、乱序、标量流水线

例：静态、加/乘双功能流水线中，加法功能仅使用3个段，计算并分析实现$\prod(a_i+b_i)$的性能，其中i=1~4。



算法—先完成所有加法、再做乘法(减少功能切换)



性能—$T_P=7/(17\Delta t)$，$S_P=1.588$，$E=26.5\%$

分析—①指令数少，填入、排空时间较多 ←仅拍3和4流水

有进无出→┘　└←有进无出

②静态流水，排空后才能切换功能 ←㈠的S1

③数据冒险，消除后才继续流水 ←㈢的S1

二、指令流水线的冒险处理

1、结构冒险处理

*产生原因：争用硬件资源(部件或路径)

如：MEM采用冯•诺依曼结构、(PC)+4复用ALU实现

*处理方法：(2种方法均可)

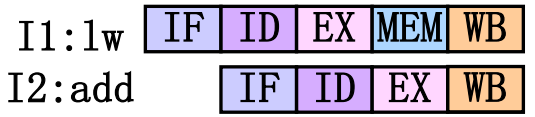
(1)重复设置资源—消除资源争用，适于高频率冲突

①部件不复用、仅本段使用 ←类似于单周期数据通路

如：MEM采用哈佛结构、PC+4增设地址加法器

又如：IF段及MEM段的写PC，可合并到IF段(选择不同数据)

再如：写GPRs统一安排在第5个段



②部件互连采用专用结构

(2)分时使用资源—避免资源争用，适于低频率冲突

争用资源的指令被暂停执行(CU实现)

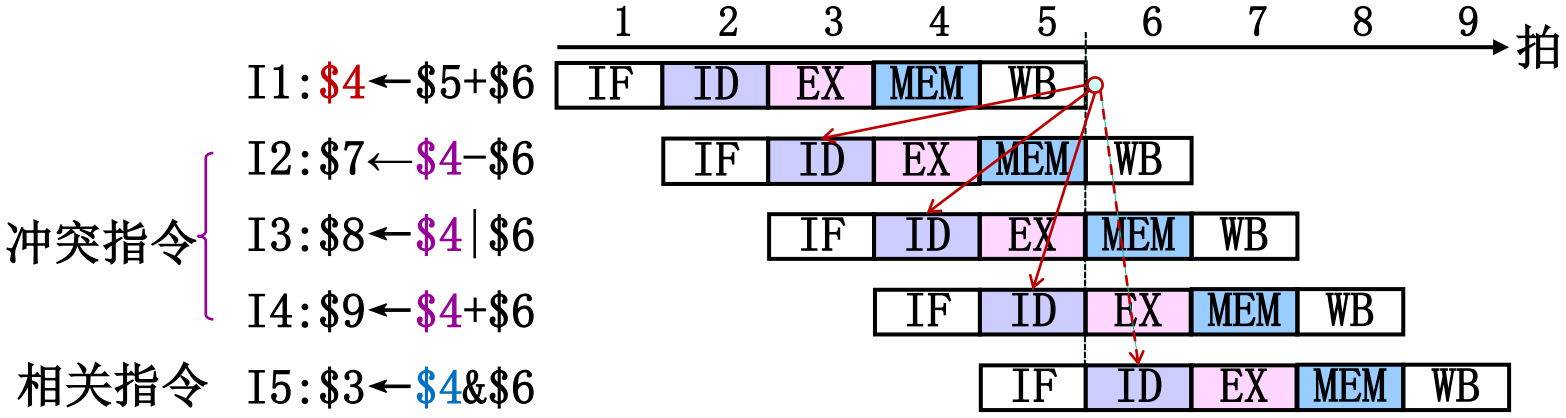
如：IF段、MEM段访存冲突时，IF段暂停(即后续指令暂停)

*应用选择：权衡性能损失～硬件成本

2、数据冒险处理

*产生原因：因重叠执行，指令所需数据不可用

*冒险类型：写后读(Read After Write, RAW)冒险等



*处理方法：阻塞法、转发法、乱序执行法

场景—食堂打饭时，甲需要借用乙(甲排在乙后)的饭卡，处理方法？

	甲	后面人	窗口状态
处理—	①原地等&拿卡后再走	一起等	有停顿(原次序)
	②往前走&途中拿卡	跟着走	无停顿(原次序)
	③原地等&拿卡后再走	绕着走	无停顿(乱次序)

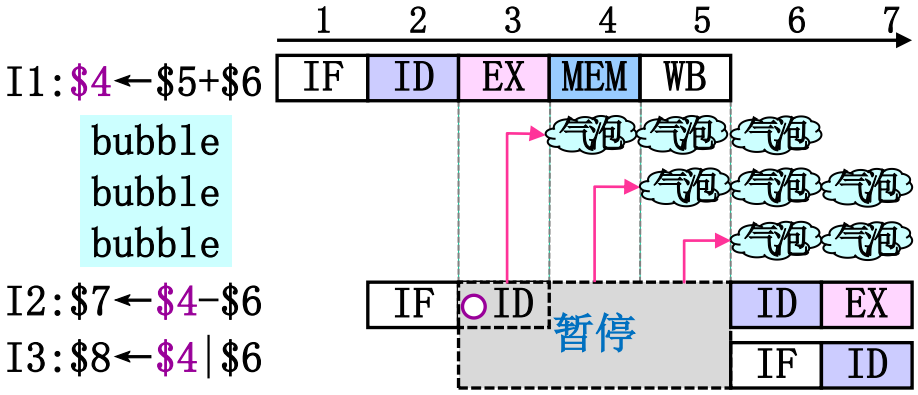
*阻塞法：冲突指令及后续指令暂停，直到冒险消除(数据就绪)

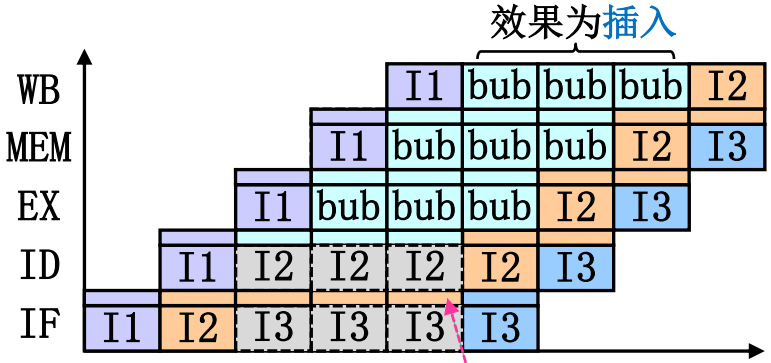
冒险消除时要求— IF/ID(段间REG)及PC不变，其余段间REG为气泡

流水线一直流动、操作基于段间REG→┘

空操作指令的uOPCmd→┘

暂停方法— ID段产生气泡，IF段暂停(不写IF/ID及PC)





思考①：ID产生气泡，IF不暂停行吗？ RAW消除时，IF/ID.IR已不是冲突指令

思考②：ID段如何产生气泡？ 用MUX选择Cmd(当前/nop指令),写入ID/EX.Cmd

CU产生→┘ └←固定值

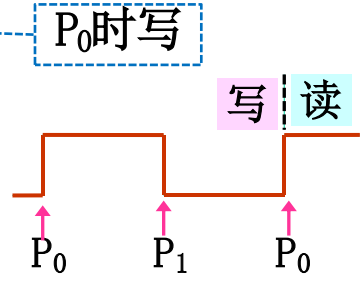
停顿拍数— 数据欲读-可读的相差拍数

思考③：如何知道停顿拍数？

ID段每拍比较(IF/ID.IR.rs～ID/EX.rd及EX/MEM.rd)，匹配失败时冒险消除

例1：MIPS流水线中，写GPRs放在WB段，在下一拍才能够读出当前拍所写的数据。现有如下MIPS指令序列：

```
I1: add $4, $5, $6      ; $4←$5+$6
I2: sub $7, $4, $6      ; $7←$4−$6
I3: or  $8, $4, $6      ; $8←$4 | $6
I4: sw  $6, 20($4)      ; M[$4+20]←$6
I5: lw  $9, 20($8)      ; $9←M[$8+20]
```



问：①哪些指令之间存在RAW冒险？

②采用阻塞法处理RAW冒险时，执行指令序列需多少拍？

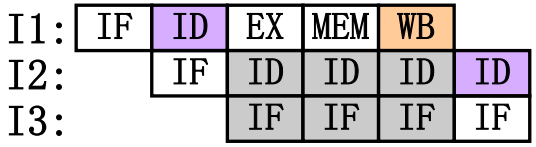
解：①RAW冒险有：I2-I1、I3-I1、I4-I1，I5-I3

②I2因I2-I1冒险需停 3 拍，I3-I1、I4-I1冒险需停 0 拍；

I5因I5-I3冒险需停 2 拍；

随I2-I1自动消除

执行时间＝[5t＋(5－1)t]＋(3＋2)t＝14t

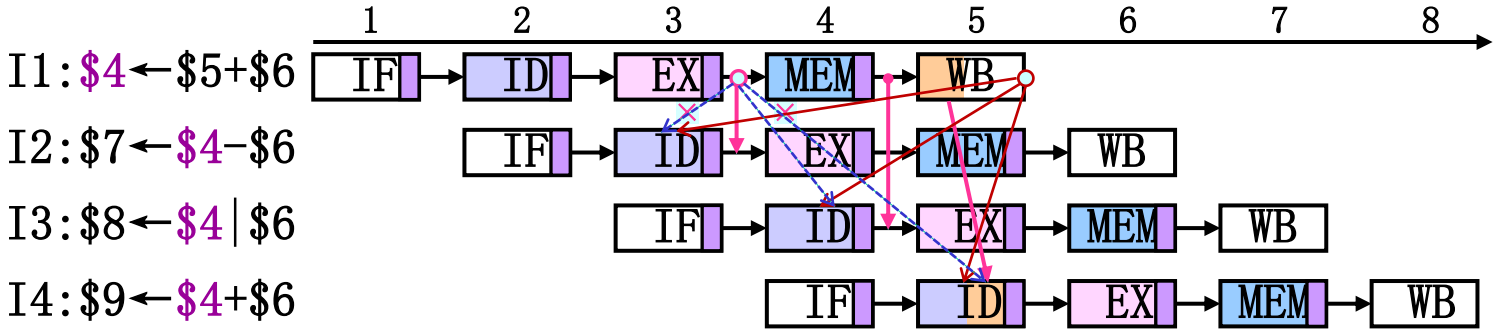


*转发法：冲突指令可从数据产生段 获取数据，来消除冒险

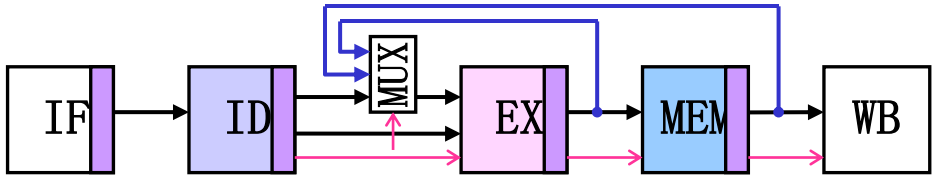
如EX段→┘ └→避免段传递时间→┘

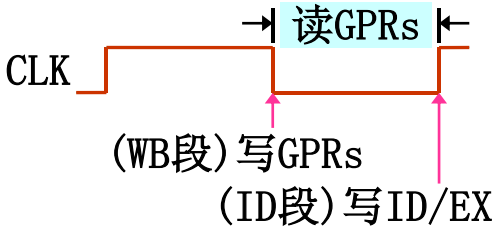
获取时机及途径—在使用段、从产生段及后续段获取

可消除冒险分析	在ID段获取	在EX段获取	在ID段/EX段获取
从EX段	下下条指令	下条指令	下条&下下条指令
从EX&后续段	下下条&后续指令	下条&后续指令	下条&后续指令



实现方法— 增设转发线路(到使用段)，同一拍中提前写GPRs





停顿拍数— 可转发时＝0拍，否则＝阻塞法

load-use冒险： lw指令与下条运算型指令间的RAW冒险

└→MEM产生～EX需要←┘ └→只能为阻塞法

例2：续例1，RAW冒险采用转发法处理，写GPRs放在前半拍完成，下列情况下，指令序列执行时间分别是多少？

P_1时写

(1)有EX→EX、MEM→EX转发线路　(2)仅有EX→EX转发线路

解：RAW冒险有： I2-I1、I3-I1、I4-I1，I5-I3

(1)I2-I1冒险停顿 0 拍；I3-I1冒险停顿 0 拍；

I4-I1冒险停顿 0 拍；I5-I3冒险停顿 0 拍；

执行时间＝[5t+(5-1)t]+(0+0+0+0)t＝9t

(2)I2-I1冒险停顿 0 拍；I3-I1冒险停顿 1 拍；（ID段可读I1的WB段）

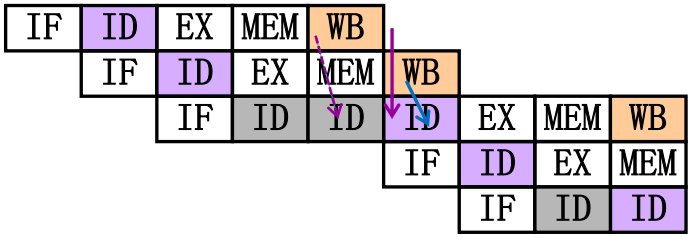
I4-I1冒险停顿 0 拍；I5-I3冒险停顿 1 拍；（同I3-I1）

执行时间＝[5t+(5-1)t]+(0+1+0+1)t＝11t

思考①：写GPRs放在后半拍时，小题(2)结果如何？

思考②：I3为$8←$4 | $7时，小题(2)结果如何？

I1:$4←$5+$6
I2:$7←$4-$6
I3:$8←$4|$7



*乱序执行法： 冲突指令停顿，后续无RAW冒险的指令可优先执行



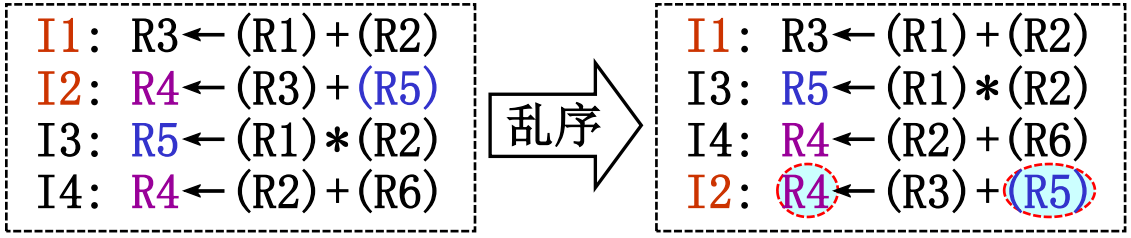
实现方法— 增设指令窗口、采用动态调度方法

（提供选择平台） （OPD就绪时流动）

停顿拍数— 0拍

新增的冒险类型— 读后写(Write After Read, WAR)冒险、

写后写(Write After Write, WAW)冒险

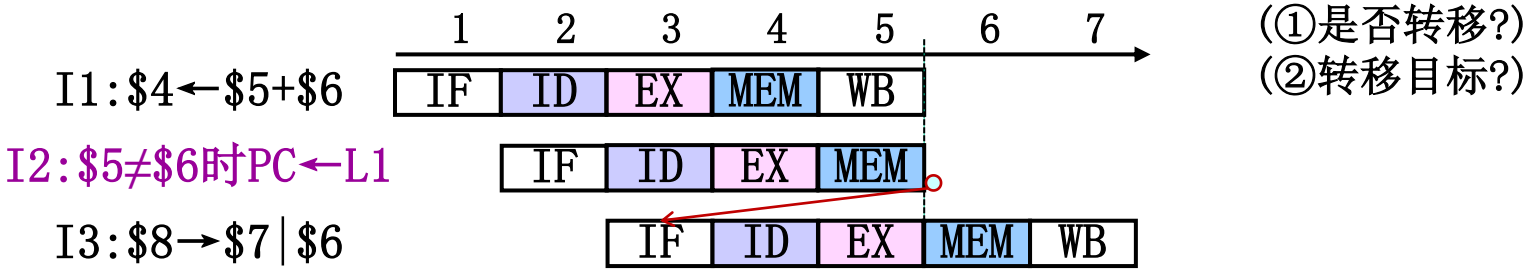


处理方法： 动态调度方法 （系统结构课程中讨论）

3、控制冒险处理

***产生原因：** 因指令执行顺序改变，引起流水线停顿

└←NextPC可能≠(PC)＋"1"　　└←IF段无法给出(①是否转移?)(②转移目标?)



***处理方法：** 阻塞法、分支预测法、延迟分支法

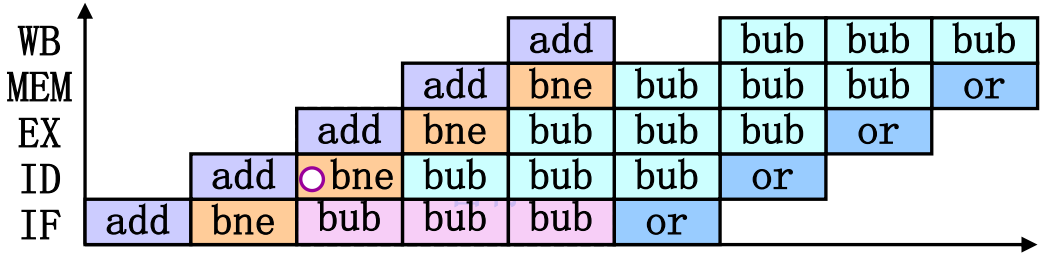
└←又称冻结法、排空法

*阻塞法：分支指令的后继指令 暂停，直到冒险消除

冒险消除时要求—PC为分支结果，各段间REG均为气泡(nop指令)

暂停方法—IF段产生气泡、PC为分支结果(最后一拍写)

└←ID段(分支指令)需继续执行



思考①：改为ID产生气泡、IF暂停，可以吗？

可以，但很麻烦(ID下拍起产生气泡/IF末拍产生气泡)

思考②：与数据冒险阻塞法的差别？

气泡产生时机(非ID)/方式(非uOPCmd)、IF(无需暂停)

停顿拍数—从ID段到指令执行完的拍数($=T_{bne}-1$)

优化方法：①尽早判断是否转移

②尽早计算转移目标地址

例3：MIPS流水线中，有EX段→EX段转发线路，bne指令在MEM段写PC。

现有如下MIPS指令序列：

```
      addi $4, $5, 100        ; I1: $4←$5＋100
L1:   add  $8, $6, $7         ; I2: $8←$6＋$7
      sw   $8, 20($6)         ; I3: M[$6＋20]←$8
      addi $5, $5, 1          ; I4: $5←$5＋1
      bne  $5, $4, L1         ; I5: $5≠$4时PC←L1
      addi $9, $9, 10         ; I6: $9←$9＋10
```

问：①哪些指令之间存在RAW冒险？

②采用阻塞法处理控制冒险，指令序列的执行时间为多少拍？

解：①RAW冒险有：I3-I2、I5-I4　(I5-I1不存在冒险)

②可用转发法处理的RAW冒险(停 0 拍)有： I3-I2、I5-I4

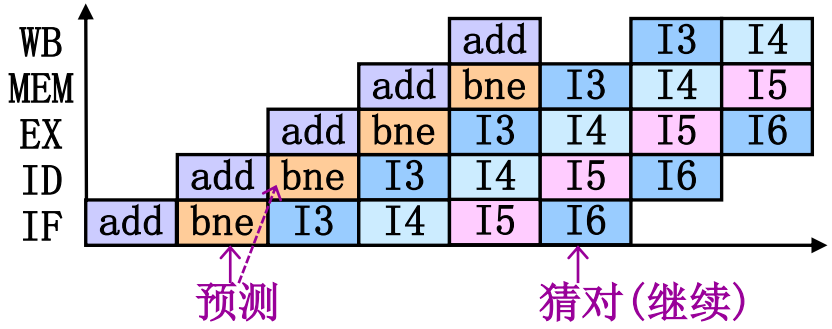
需用阻塞法处理的RAW冒险(停 0 拍)有： 无

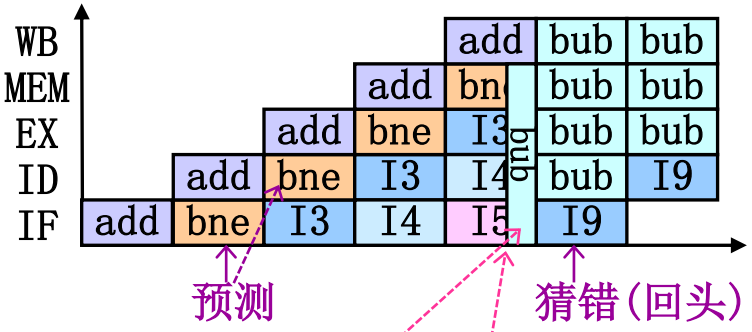
控制冒险用阻塞法处理时，I5每次使流水线停 3 拍；

执行时间＝[5t＋(402－1)t]＋3t×100＝706t

*预测法：预测转移方向(转/不转)，并执行该方向的指令；

猜对时继续执行后续指令，猜错时回头执行反方向指令





预测策略— (静态预测[盲猜]/动态预测[基于转移历史])

静态预测
- 预测不转移：无先决条件(PC+1已知)，可在IF段预测
- 预 测 转 移 ：需先计算目标地址，最早在ID段预测

动 态 预 测 ：已保存转移历史&目标地址，可在IF段预测

实现机制—猜对时不写PC，

猜错时重写PC、清空流水线(产生气泡)

可不占额外时间

停顿拍数—猜对时≥0拍，猜错时＝阻塞法(到指令执行完为止)

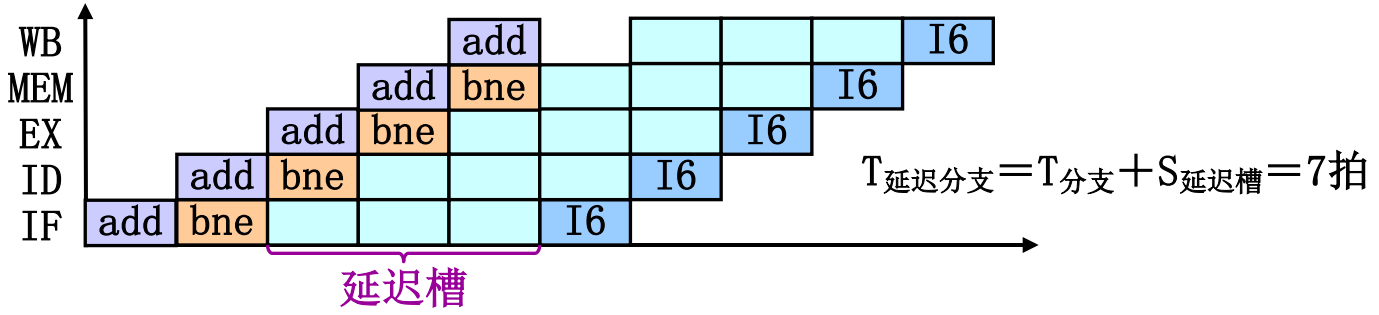
(ID预测时＝1)

└←无论是IF预测或ID预测←┘

*延迟分支法： 延迟槽中指令正常执行，使其尽量为有用指令

└←逻辑上延长了分支指令执行时间

延迟槽—分支指令执行完前，可流入流水线的指令位置



实现机制—软件实现(编译时重排序指令序列)

顺序指令I1～I5 转移指令I6 顺序指令I7～I9	⇨	顺序指令I1～I5中与I6有相关的指令 转移指令I6 延迟槽(I1～I5中与I6无相关的指令或nop指令) 顺序指令I7～I9

停顿拍数—延迟槽中均为nop指令时，＝阻塞法；

延迟槽中含有用指令时，＜阻塞法

适用场合—延迟槽＝1条指令时(即$T_{分支}=2T_C$) ←性能损失(为nop)小

└←≥2条时常用分支预测法 ←与延迟分支法不兼容

※指令流水线技术小结：

基本思想—各操作串行→各操作重叠　　（指不同指令间）

基本组成—

要求：操作分离、操作同步、操作无冲突

实现：增设段间REG，公共拍时钟，部件＋控制器

操作冲突的处理—

冲突类型：①结构冒险 ②数据冒险 ③控制冒险

处理方法：①重复设置部件、分时使用部件

└←部件不复用、使用时间无冲突

②阻塞法、转发法、乱序执行法

③阻塞法、分支预测法、延迟分支法

作业5-5：P238— 26、27

※附录1：μOPCmd序列效果图

